

Universidad de San Carlos de Guatemala
-Centro Universitario de Occidente-
Ingeniería en Ciencias y Sistemas
Ing. Mario Ramírez Tobar
Auxiliar: Luis Tax
Seminario de Sistemas 1

Multiservicios Wilson

Sistema de gestión y automatización de procesos

Documentación BPM - BPMN - BPMS
Fase 1 - Análisis, Modelado y Planificación
Fase 2 - Desarrollo e Integración Técnica
Fase 3 - Despliegue, Seguridad y Cierre

Código	Carnet	Nombre
21	201930672	Vicente Rocael Matías Osorio

Quetzaltenango octubre de 2026

ÍNDICE

1. Resumen Ejecutivo **7**
2. Introducción **8**
3. Marco Teórico y Metodológico **9**
3.1. Business Process Management (BPM) 9
3.2. Business Process Model and Notation (BPMN) 9
3.3. Business Process Management System (BPMS) 9
4. Objetivos **10**
4.1. Objetivo General 10
4.2. Objetivos Específicos 10
5. Descripción de la Empresa **10**
6. Mapa de Macroprocesos **10**
7. Priorización de Procesos **11**
8. Descripción de Procesos **12**
8.1. Proceso 1 - Gestión de inventario y reposición automática 12
8.2. Diccionario de actividades - Proceso 1 14
9. Proceso 2 - Generación automática de fotocopia de DPI **15**
9.1. Diccionario de actividades - Proceso 2 16
10. Proceso 3 - Fotografías tamaño cédula y pasaporte **17**
11. BPMS - Rol del sistema como motor de automatización **18**
12. Roles y Responsabilidades (RACI) **18**
13. Requisitos funcionales y no funcionales **19**
13.1. Requisitos Funcionales 19
13.2. Requisitos No Funcionales 20
14. Alcance excluído de esta fase **20**
15. Arquitectura, infraestructura y flujo de trabajo **21**
16. Gestión ágil del proyecto **21**
17. Glosario de términos **22**
18. Mockups **23**
18.1. Dashboard 23
18.2. Ventas 23
18.3. Generación de docs. 24
19. FASE 2 **24**
19.1. Historias de Usuario — Sistema Multiservicios Wilson **25**
19.1.1. Epic: Landing Publica 25
HU-01: Ver información del negocio sin iniciar sesión 25
HU-02: Acceso directo al login desde la landing 25
19.1.2. Epic: Autenticación y sesión 25
HU-03: Iniciar sesión 25

HU-04: Cerrar sesión 25
HU-05: Restricción de acciones sensibles por rol 26
19.1.3. Epic: Dashboard / Inicio (Administrador) 26
HU-06: Ver resumen general del negocio 26
HU-07: Ver alertas de stock recientes desde el dashboard 26
HU-08: Ver ventas recientes desde el dashboard 26
19.1.4. Epic: Perfil 26
HU-09: Ver mis datos de perfil 26
19.1.5. Epic: Productos 27
HU-10: Ver el listado de productos 27
HU-11: Buscar productos 27
HU-12: Registrar un nuevo producto 27
HU-13: Editar un producto existente 27
HU-14: Desactivar un producto 27
HU-15: Reactivar un producto 28
19.1.6. Epic: Proveedores 28
HU-16: Ver el listado de proveedores 28
HU-17: Registrar un nuevo proveedor 28
HU-18: Editar un proveedor 28
HU-19: Eliminar un proveedor 28
19.1.7. Epic: Ventas 28
HU-20: Registrar una venta 28
HU-21: Evitar vender más stock del disponible 29
HU-22: Descuento automático de stock al vender 29
HU-23: Alerta automática de stock bajo tras una venta 29
HU-24: Ver ventas recientes 29
19.1.8. Epic: Alertas de stock 29
HU-25: Ver alertas de stock pendientes 29
HU-26: Marcar una alerta como atendida 29
19.1.9. Epic: Documentos DPI 30
HU-27: Subir las fotos del DPI 30
HU-28: Detección automática del borde de la tarjeta 30
HU-29: Ajuste manual del recorte 30
HU-30: Rotar las fotos del DPI 30
HU-31: Ajustar brillo, contraste y nitidez 30
HU-32: Elegir el formato de salida del documento 30
HU-33: Generar y descargar el documento DPI 30
HU-34: Registro de notificación al generar un documento 31
19.1.10. Epic: Reportes 31
HU-35: Generar reporte de ventas por rango de fechas 31
HU-36: Generar reporte de inventario 31
HU-37: Exportar reportes en PDF o Excel 31
19.1.11. Epic: Usuarios (Administrador) 31
HU-38: Ver el listado de cuentas del sistema 31
HU-39: Crear una nueva cuenta de usuario 31

HU-40: Editar una cuenta de usuario 32
HU-41: Desactivar una cuenta de usuario 32
HU-42: Reactivar una cuenta de usuario 32
20. Diagramas UML **33**
20.1. Diagrama de clases 33
20.2. Diagrama de Infraestructura 34
20.3. Diagrama de Arquitectura 35
20.4. Diagrama de Secuencia 36
20.5. Diagrama de Entidad Relación 37
20.6. Diagrama de Paquetes 38
20.7. Diagrama de Actividades 39
21. Modelos de Procesos Técnicos **40**
21.1. 01- Inicio Sesión 40
21.2. 02- Cerrar Sesión 40
21.3. 03- Atender Alerta Stock 41
21.4. 04 - Crear Usuario 41
21.5. 05 - Editar Usuario 42
21.6. 06 - Activar/Desactivar Usuario 42
21.7. 07 - Crear Producto 43
21.8. 08 - Editar Producto 43
21.9. 09 - Dar Baja/Reactivar Producto 44
21.10. 10 - Registrar/Editar Proveedor 44
21.11. 11 - Eliminar Proveedor 45
21.12. 12 - Reporte Ventas 45
21.13. 13 - Reporte Inventario 46
21.14. 14 - Reporte Notificaciones 46
21.15. 15 - Renderización Pasaporte 47
21.16. 16 - Renderización Cédula 47
21.17. 16 - Proceso Completo 48
22. Casos de Uso **49**
22.1. 01 - Acceso Público y Autenticación 49
22.2. 02 - Gestión de Inventario 49
22.3. 03 - Operaciones Diarias 50
22.4. 04 - Administración 50
23. FASE 3 **51**
23.1. Despliegue, seguridad, automatización y cierre 51
23.2. Resumen técnico del sistema al cierre 51
23.3. Decisiones consolidadas respecto a las Fases 1 y 2 52
23.4. Actualización de requisitos 52
23.5. Historias de usuario nuevas 53
23.6. Glosario complementario 54

24. Manual Técnico Integral **55**
24.1. Introducción, audiencia y convenciones 55
24.2. Visión general de la arquitectura 55
24.3. Arquitectura detallada 58
24.4. Catálogo de la API REST 68
24.5. Modelado de datos 70
24.6. Trazabilidad 73
24.7. Calidad y pruebas 76
25. Guía de Despliegue y Runbook **77**
25.1. Propósito, alcance y convenciones 77
25.2. Topología de producción 78
25.3. Requisitos previos, cuentas y secretos 79
25.4. Despliegue 80
25.5. Rollback 84
25.6. Monitoreo y observabilidad 87
25.7. Respaldo y recuperación 88
25.8. Resolución de incidentes 91
25.9. Operación rutinaria 93
25.10. Lista de endurecimiento de seguridad 93
26. Documentación para Escalabilidad Futura **94**
26.1. Línea base actual y límites medidos 94
26.2. Cuellos de botella y deuda técnica identificados 94
26.3. Estrategia de escalado por componente 95
26.4. Arquitectura objetivo y equivalencias entre proveedores 95
26.5. Hoja de ruta por horizontes 98
26.6. Evolución funcional: multi-sucursal y nuevos canales 99
26.7. Capacidad y dimensionamiento orientativo 99
26.8. Criterios de activación 99
27. Procesos Identificados y Diagramados (BPMN) **100**
27.1. Inventario de procesos 100
27.2. Notación y convenciones 100
27.3. BPMN-01 · Venta, descuento de stock y alerta automática 102
27.4. BPMN-02 · Reposición: pedido al proveedor y cierre de la alerta 104
27.5. BPMN-03 · Fotocopia de DPI 106
27.6. BPMN-04 · Fotos tamaño cédula y pasaporte 108
27.7. BPMN-05 · Inicio de sesión con verificación en dos pasos 110
27.8. BPMN-06 · Recuperación de contraseña por correo 112
27.9. BPMN-07 · Reportes de ventas e inventario con exportación 114
27.10. BPMN-08 · Entrega continua con GitFlow 116
27.11. BPMN-09 · Gestión de incidentes en producción 118

27.12. Procesos propuestos 119
27.13. BPMN-F1 · Compras a proveedores con orden y recepción (propuesto) 120
27.14. BPMN-F2 · Apertura y cierre de caja diario (propuesto) 122
27.15. BPMN-F3 · Registro de servicios (propuesto) 124
28. Migraciones para la Evolución del Modelo de Datos 126
28.1. Política y convenciones 126
28.2. Historial aplicado y línea de tiempo 126
28.3. Hallazgos del modelo actual que motivan las migraciones 128
28.4. Plan de migraciones propuestas 128
28.5. V12 · Integridad e índices 129
28.6. V13 · Auditoría y estado de seguridad 129
28.7. V14 · Kardex y compras 130
28.8. V15 · Clientes, caja, servicios y documentos generados 131
28.9. V16 · Notificaciones leídas por usuario 132
28.10. V17 · Sucursales 133
28.11. V18 · Contraer las columnas heredadas 133
28.12. Estrategia de ejecución y pruebas 133
28.13. Modelo entidad-relación objetivo y trazabilidad de las migraciones 135
29. Tecnologías Nuevas por Utilizar y su Justificación 137
29.1. Criterios de selección 137
29.2. Línea base: lo que ya se utiliza y se mantiene 137
29.3. Tecnologías nuevas propuestas 137
29.4. Detalle de las decisiones principales 138
29.5. Tecnologías evaluadas y descartadas 140
29.6. Hoja de ruta de adopción 140
30. Conclusiones 141
31. Enlaces de interés 142

1. Resumen Ejecutivo

Multiservicios Wilson es una mini empresa de servicios múltiples orientada a la atención rápida de necesidades cotidianas de sus clientes: venta de celulares nuevos, venta de accesorios para celulares, fotocopias, emplasticados, fotografías tamaño cédula y pasaporte. El negocio opera hoy con procesos manuales, apoyados en cuadernos de control, hojas de cálculo sueltas y edición de documentos de Word para tareas repetitivas, lo que genera tiempos de atención más largos de lo necesario y expone al negocio a errores evitables, como quiebres de stock o inconsistencias en el inventario.

El presente documento desarrolla, bajo la metodología BPM (Business Process Management), el análisis, modelado y propuesta de automatización de los dos procesos de mayor impacto identificadores en el negocio: la gestión de inventario con reposición automática de accesorios y celulares, y la generación automática de documentos de identificación (DPI) en formato fotocopia. Ambos procesos se documentan bajo la notación BPMN (Business Process Management and Notation) en sus versiones AS-IS (situación actual) y TO-BE (situación propuesta), y se plantea el rol que cumplirá el sistema a desarrollar como motor de automatización de procesos (BPMS) para Multiservicios Wilson.

Esta primera fase corresponde al alcance, análisis y planificación del proyecto: aquí se definen el problema, los objetivos, los procesos priorizados, los requerimientos funcionales y no funcionales, la base metodológica y técnica sobre la cual se construirán las fases de desarrollo e integración (fase 2) y despliegue, seguridad y cierre (fase 3).

2. Introducción

Toda empresa, sin importar su tamaño, ejecuta procesos: secuencias repetitivas de actividades que transforman una necesidad del cliente en un servicio o producto entregado. En negocios pequeños como Multiservicios Wilson estos procesos casi nunca están documentados ni formalizados; existen únicamente en la experiencia y la memoria de quien atiende el mostrador. Esta informalidad funciona mientras el volumen de trabajo es bajo, pero se vuelve un cuello de botella cuando los procesos se repiten decenas de veces al día y dependen de tareas manuales lentas.

La gestión de procesos de negocio (BPM) ofrece un marco de trabajo para identificar, modelar, analizar, mejorar y automatizar estos procesos de forma sistemática. Aplicado a Multiservicios Wilson, este enfoque permite pasar de una operación basada en la memoria del encargado a una operación soportada por reglas de negocio explícitas, trazables y, en los casos que lo justifican, automatizadas mediante software.

Este documento presenta el resultado de ese análisis: el mapeo completo de los procesos del negocio, la priorización de los dos procesos con mayor retorno de automatización, su modelado BPMN en las versiones actual y propuesta, y la definición del alcance funcional técnico del sistema que se construirá en las siguientes fases del proyecto.

3. Marco Teórico y Metodológico

3.1. Business Process Management (BPM)

BPM es la disciplina de gestión que concibe a la organización como un conjunto de procesos interrelacionados, y que promueve su identificación, documentación, análisis, mejora, automatización y monitoreo continuo. No se trata de una herramienta ni de un software en particular, sino de una forma de gestionar el negocio centrada en el flujo de trabajo y en el valor entregado al cliente, más que en las funciones aisladas de cada área o persona.

3.2. Business Process Model and Notation (BPMN)

BPMN es el estándar internacional (mantenido por el Object Management Group, OMG) para representar gráficamente procesos de negocio. Utiliza un conjunto reducido y estandarizado de símbolos, eventos, actividades, compuertas de decisión, flujos de secuencia y carriles (pools y lanes) que representan a los actores del proceso que permiten que cualquier persona, técnica o no, comprenda el funcionamiento de un proceso con solo observar el diagrama. En este documento se utiliza BPMN para representar tanto la situación actual (AS-IS) como la situación propuesta (TO-BE) de cada proceso priorizado.

3.3. Business Process Management System (BPMS)

Un BPMS es la plataforma de software que ejecuta y automatiza los procesos previamente modelados en BPMN. Existen BPMS comerciales de propósito general, como Bizagi, Camunda, o Bonita, orientados a organizaciones que necesitan orquestar procesos complejos con muchos actores y sistemas. Multiservicios Wilson, por su tamaño y naturaleza, no requiere un BPMS de propósito general: en su lugar, el producto de software desarrollado en este proyecto cumplirá la función de un BPMS específico y a medida, capaz de ejecutar automáticamente las reglas de negocio de los dos procesos priorizados (descuento y reposición de inventario; generación de documentos de identificación). Esta decisión se documenta y justifica en la sección 8 de este documento.

4. Objetivos

4.1. Objetivo General

Diseñar y modelar, bajo el enfoque BPM y la notación BPMN, los procesos operativos de mayor impacto de Multiservicios Wilson, como base para el desarrollo de un sistema web que automatice la gestión de inventario y la generación de documentos de identificación, mejorando los tiempos de atención y reduciendo el margen de error humano en la operación diaria del negocio.

4.2. Objetivos Específicos

- Levantar y documentar el mapa de macroprocesos de Multiservicios Wilson, clasificándolos en estratégicos, misionales y de apoyo.
- Modelar en notación BPMN la situación actual (AS-IS) de los procesos de venta e inventario, y de generación de fotocopias de DPI.
- Diseñar la situación propuesta (TO-BE) de ambos procesos, incorporando las reglas de negocio de automatización identificadas y los requerimientos funcionales y no funcionales.

5. Descripción de la Empresa

Multiservicios Wilson es un negocio local de servicios múltiples que atiende a clientes de manera presencial en un único punto de atención, Su oferta de servicios actual incluye:

- Venta de Celulares nuevos, adquiridos a proveedores de telefonía nacional (TIgo, Claro)
- Venta de accesorios para celulares (protectores, vidrios templados, cargadores, audífonos, entre otros).
- Fotocopias de documentos en general.
- Emplasticado de documentos.
- Fotografías tamaño cédula y pasaporte.
- Elaboración de fotocopias de Documento Personal de Identificación (DPI), a partir de fotografías de anverso y reverso proporcionadas por el cliente.

La operación depende actualmente de un único encargado en mostrador, que atiende simultáneamente ventas, control de inventario y elaboración de documentos. Esta concentración de tareas es la principal fuente de ineficiencia que este proyecto busca atender: cualquier proceso que se automatice libera tiempo directamente aprovechable en la atención al cliente.

6. Mapa de Macroprocesos

Antes de priorizar qué automatizar, se levantó el mapa completo de procesos del negocio, clasificados según su naturaleza dentro de la cadena de valor.

Tipo	Procesos Identificados
Estratégicos	Gestión general del negocio - Relación con proveedores de telefonía
Misionales (core)	Venta de celulares y accesorios - Gestión de inventario - Fotografías tamaño cédula/pasaporte - Generación de fotocopia de DPI
De apoyo	Compras a proveedores - Cobro y caja - Atención al cliente

7. Priorización de Procesos

De los procesos misionales identificados, se priorizan dos para ser modelados en detalle y automatizados en este proyecto, por ser los que presentan mayor frecuencia de repetición, maor tiempo invertido por transacción y mayor riesgo operativo si continúan siendo manuales.

Proceso	¿Se automatiza?	Justificación
Gestión de inventario y reposición	Sí - Proceso 1	Regla de negocio clara y medible, riesgo real de pérdida de ventas por quiebre de stock, impacto directo en la relación con proveedores.
Generación de fotocopia de DPI	Sí - Proceso 2 (proceso complejo)	Proceso manual repetitivo (recorte y ajuste de imagen en Word), alto consumo de tiempo por transacción (4-6 minutos) buen candidato técnico por involucrar procesamiento de imágenes.
Fotografía tamaño cédula	Sí, Proceso 3 (proceso	Comparte tecnología de

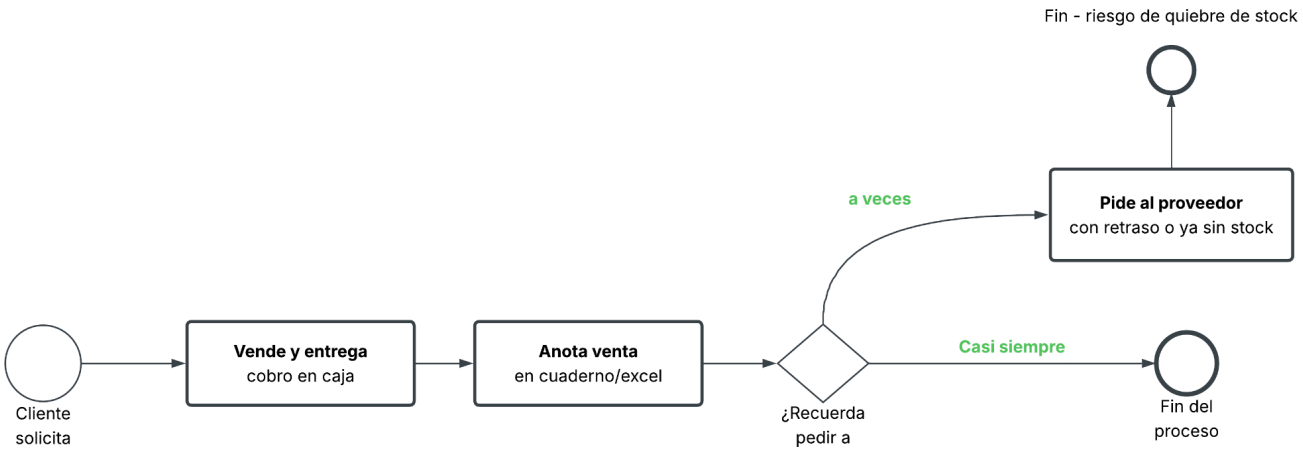
	complejo)	procesamiento de imagen con el proceso 2
Fotografía tamaño pasaporte	Sí, Proceso 4 (proceso complejo)	Comparte tecnología de procesamiento de imagen con el proceso 1 y 2.
Fotocopias generales y emplasticados	No	Procesos físicos sin lógica de negocio relevante que digitalizar, bajo retorno de automatización.
Compra-venta de celulares a proveedores	Parcialmente	Se integra como parte del mismo módulo de inventario del Proceso 1, en lugar de tratarse como proceso independiente.

8. Descripción de Procesos

8.1. Proceso 1 - Gestión de inventario y reposición automática

Situación Actual (AS-IS)

Actualmente, cada vez que se vende un producto (celular o accesorio), el encargado entrega el producto y anota la venta en un cuaderno, y posterior a ello en una hoja de cálculo. El control de las existencias restantes depende de que el encargado recuerde o revise manualmente cuántas unidades quedan de cada modelo. Como normalmente se mantienen 1 o 2 unidades en stock por modelo, el margen para reaccionar a tiempo es muy pequeño: en la práctica, el pedido de reposición al proveedor se hace tarde con frecuencia, a veces cuando el producto ya se ha agotado por completo.



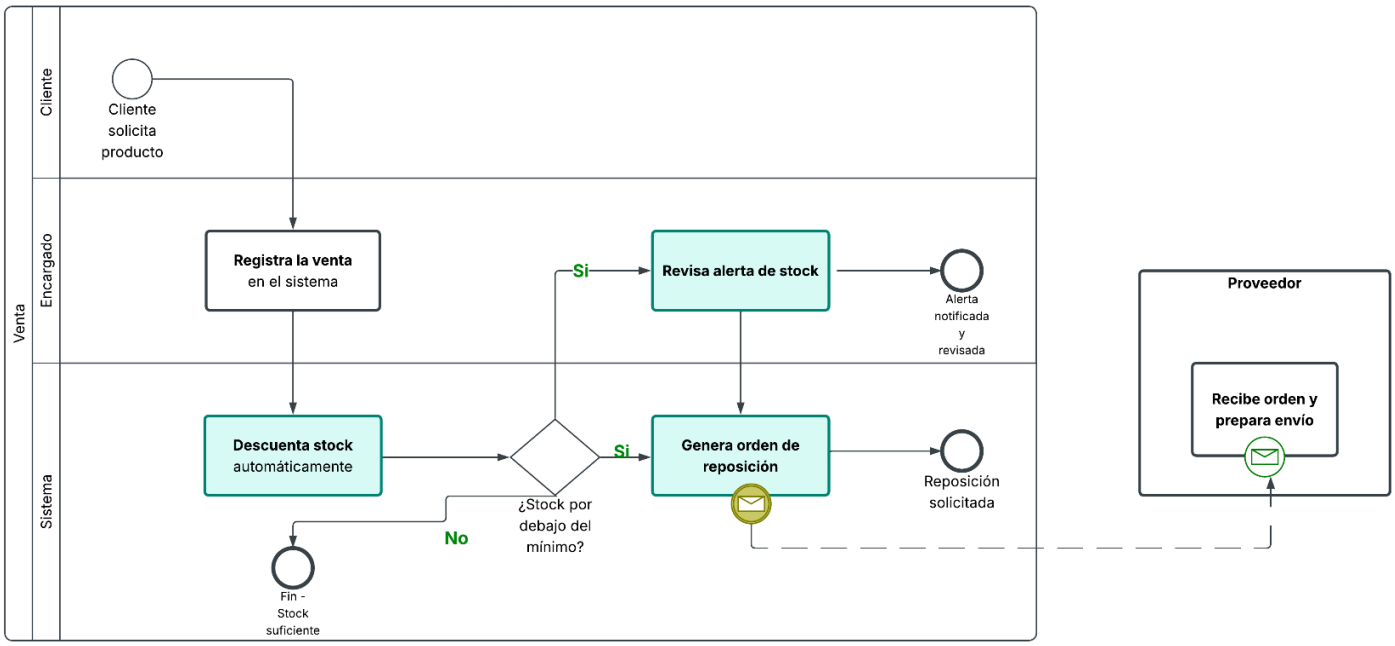
El principal problema de esta situación no es la venta en sí, sino la ausencia de un mecanismo confiable que dispare la reposición en el momento exacto en que el stock cae por debajo del mínimo operativo.

Regla de negocio identificada

Stock mínimo por modelo: 2 unidades para algunos modelos. Cuando el stock de un modelo de celular, desciende por debajo de este umbral tras una venta, el sistema debe generar automáticamente una notificación de reposición dirigida al encargado, referenciando al proveedor correspondiente.

Situación propuesta (TO-BE)

En la situación propuesta, el encargado únicamente registra la venta en el sistema: a partir de ahí, todo el control del inventario deja de depender de la memoria humana. El sistema descuenta el stock de forma automática en el momento de la venta y evalúa, también automáticamente, si el nuevo nivel de existencias está por debajo del mínimo definido para ese modelo. Si es así, genera una orden de reposición y notifica al encargado; si el stock aún es suficiente, el proceso simplemente concluye sin intervención adicional.



Con este rediseño, la responsabilidad de vigilar el stock deja de recaer en la memoria del encargado y pasa a ser una regla de negocio ejecutada de forma consistente por el sistema en cada transacción, eliminando el principal punto de falla del proceso actual.

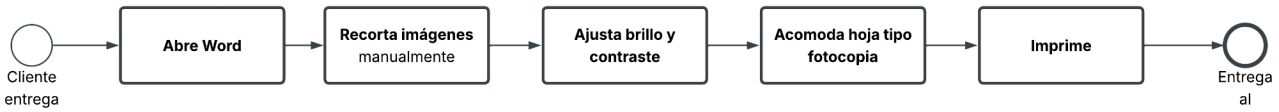
8.2. Diccionario de actividades - Proceso 1

Actividad	Responsable	Entrada	Salida
Registrar la venta	Encargado	Producto solicitado por el cliente	Venta registrada en el sistema
Descontar stock automáticamente	Sistema	Registro de venta	Nivel de stock actualizado
Evaluar stock mínimo	Sistema	Nivel de stock actualizado	Resultado de la evaluación (si/no)
Generar orden de reposición	Sistema	Resultado positivo de la evaluación	Orden de reposición y alerta al encargado
Recibir orden y preparar envio	Proveedor	Orden de reposición	Producto en camino al negocio

9. Proceso 2 - Generación automática de fotocopia de DPI

Situación actual (AS-IS)

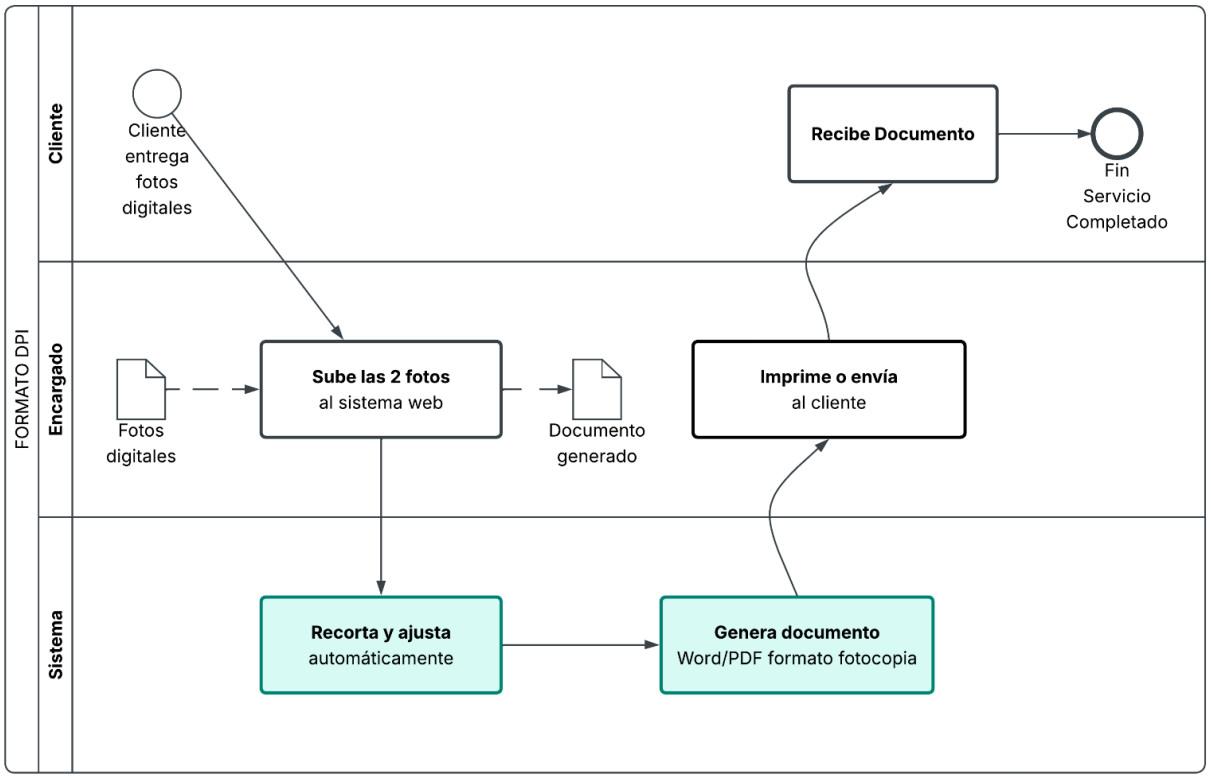
El cliente entrega al encargado las fotografías del anverso y reverso de su DPI(tomadas por el cliente o por el propio encargado). Con esas imágenes, el encargado abre un documento de Word, recorta manualmente cada fotografía, ajusta brillo y contraste a simple vista, y acomoda ambas imágenes dentro de una hoja tamaño carta imitando el formato tradicional de una fotocopia. El documento resultante se imprime y se entrega al cliente. Esta secuencia, aunque sencilla en apariencia, toma entre 10 y 15 minutos por cliente y depende completamente de la habilidad manual del encargado con las herramientas de edición de Word.



Al tratarse de un proceso puramente repetitivo, las mismas operaciones de recorte y ajuste se repiten cliente tras cliente sin variación en la lógica, es un candidato idóneo para automatizarse por completo mediante procesamiento de imágenes.

Situación propuesta (TO-BE)

En la propuesta, el encargado sube al sistema únicamente las dos fotografías originales(anverso y reverso). El sistema aplica de forma automática el recorte y el ajuste de imagen, una funcionalidad de procesamiento de imágenes que constituye el proceso técnico más complejo del proyecto, y genera directamente un documento en formato Word o PDF con ambas imágenes ya acomodadas en el formato de fotocopia estándar.
El encargado solo debe imprimir o enviar el documento resultante al cliente.



El impacto esperado es una reducción del tiempo de atención de 5-7 minutos a aproximadamente 2 minutos por cliente, además de eliminar la variabilidad de calidad asociada al recorte manual.

9.1. Diccionario de actividades - Proceso 2

Actividad	Responsable	Entrada	Salida
Entregar fotografías	Cliente	Fotos de anverso y reverso del DPI	Imágenes digitales disponibles
Subir las fotografías al sistema	Encargado	Imágenes digitales del cliente	Imágenes cargadas en el sistema
Recortar y ajustar automáticamente	Sistema	Imágenes cargadas	Imágenes procesadas y normalizadas
Generar documento formato fotocopia	Sistema	Imágenes procesadas	Documento PDF listo
Imprimir o enviar al cliente	Encargado	Documento generado	Documento entregado al cliente

10. Proceso 3 - Fotografías tamaño cédula y pasaporte

Este tercer proceso se documenta con el mismo nivel de detalle que los dos anteriores porque, aunque su implementación en código no forma parte del alcance de desarrollo de esta entrega, comparte por completo la lógica de negocio y la tecnología de procesamiento de imágenes del Proceso 2. Modelar desde ahora permite planificar el motor de recorte y ajuste como un componente reutilizable, parametrizado por tipo de documento, en lugar de construir nuevamente cuando se decida implementarlo.

Situación actual (AS-IS)

El cliente solicita una fotografía tamaño cédula o pasaporte. El encargado forma la fotografía (o recibe una ya tomada), la abre en un editor de imagen, la recorta y ajusta manualmente al tamaño estándar correspondiente y acomoda varias copias dentro de una misma hoja fotográfica antes de imprimir. Al igual que en el Proceso 2, esta secuencia es puramente repetitiva y depende de la habilidad manual del encargado, con un tiempo estimado de 8 a 10 minutos por cliente.

Situación propuesta (TO-BE)

En la propuesta, el encargado sube la fotografía original al sistema e indica el tipo de documento solicitado. El sistema reutiliza el mínimo motor de recorte y ajuste automático construido para el Proceso 2, mediante un parámetro que define el tamaño estándar de salida, y genere automáticamente una hoja con varias copias de la fotografía lista para imprimir.

11. BPMS - Rol del sistema como motor de automatización

Como se explicó en el marco metodológico, Multiservicios Wilson no requiere ni justifica la adopción de un BPMS comercial de propósito general. En su lugar, el sistema web desarrollado en este proyecto asumirá el rol de BPMS a medida para los procesos priorizados: será la plataforma que reciba los eventos de negocio (una venta, la carga de dos fotografías), ejecute automáticamente las reglas asociadas (evaluación de stock mínimo, recorte y ajuste de imagen) sin intervención manual adicional.

12. Roles y Responsabilidades (RACI)

Matriz de responsabilidades para las actividades clave de los procesos priorizados.

R: Responsable – A: Aprueba – C: Consultado – I: Informado

Actividad	Cliente	Encargado	Sistema	Proveedor
Registrar venta	I	R/A	C	-
Descontar stock	-	I	R/A	-
Generar orden de reposición	-	I	R/A	C
Recibir y preparar envío	-	I	I	R/A
Entregar fotografías DPI	R/A	C	-	-
Subir fotografías al sistema	I	R/A	C	-
Recortar y generar documento	-	I	R/A	-
Entregar documento final	I	R/A	-	-

13. Requisitos funcionales y no funcionales

13.1. Requisitos Funcionales

Código	Requisito
RF-01	El sistema debe permitir autenticación mediante usuario y contraseña, con al menos dos roles: Administrador y Encargado
RF-02	El sistema debe permitir el registro, edición, consulta y baja de productos(celulares y accesorios) en el inventario.
RF-03	El sistema debe descontar automáticamente el stock del producto vendido al registrar una venta.
RF-04	El sistema debe generar una alerta y una orden de reposición cuando el stock de un modelo caiga por debajo del mínimo configurado (2 unidades)
RF-05	El sistema debe permitir cargar dos imágenes (anverso y reverso de DPI) y generar automáticamente un documento en formato PDF con ambas imágenes recortadas y ajustadas en formato de fotocopia.
RF-06	El sistema debe notificar al Administrador o Encargado ante eventos críticos (stock bajo, documento generado) mediante un módulo de notificaciones dentro del sistema.
RF-07	El sistema debe generar reportes de ventas e inventario, exportables en formato Excel (.xlsx) y PDF.

13.2. Requisitos No Funcionales

Código	Requisito
RNF-01	El sistema debe estar mediante una URL pública desplegado en un proveedor cloud
RNF-02	El tiempo de generación del documento de fotocopia de DPI no debe superar los 10 segundos por solicitud.
RNF-03	El sistema debe aplicar cifrado en tránsito (HTTPS) y gestión segura de credenciales de acceso.
RNF-04	El esquema de base de datos debe versionarse mediante una herramienta de migraciones.
RNF-05	El código fuente debe gestionarse bajo un flujo Gitflow(main, develop, feature/, hotfix) con revisión obligatoria de Pull request

14. Alcance excluído de esta fase

Con el fin de mantener el proyecto enfocado y viable dentro del tiempo disponible, los siguientes procesos del negocio quedan fuera del alcance automatizado de este proyecto, aunque se documentan como posibles extensiones futuras:

- Fotocopias generales: Proceso puramente físico(impresión), sin lógica de negocio que digitalizar.
- Emplasticado de documentos: Proceso físico sin componente de información relevante para el sistema.
- Implementación de landing page:
    - El sistema está diseñado exclusivamente para uso interno de la empresa.
    - No es un producto B2C (Business to Consumer): No está orientado a clientes finales ni al público en general.
    - Es un sistema B2E (Business to Employee): Su único propósito es optimizar los procesos internos de inventario, ventas, gestión de productos y servicios.
    - Acceso restringido: Solo personal autorizado (administradores y empleados) tiene acceso al sistema mediante credenciales y roles

Conclusión: Una landing page está diseñada para captar clientes externos y generar leads, lo cual no aplica a un sistema interno.

15. Arquitectura, infraestructura y flujo de trabajo

El sistema se desplegará sobre un proveedor de nube (a definir entre AWS, Azure o GCP durante la Fase 2), separando frontend, backend e infraestructura de base de datos como componentes independientes para facilitar su escalabilidad y mantenimiento. La automatización de integración y despliegue (CI/CD) se configurará mediante pipelines que se disparen automáticamente en cada cambio confirmado sobre las ramas relevantes del repositorio.
La gestión de versiones del código seguirá el modelo GitFlow: la rama main contendrá únicamente código estable, develop concentrará la integración de nuevas funcionalidades, las ramas feature/ se usarán para el desarrollo de cada funcionalidad de forma aislada, y las ramas hotfix/ para corregir errores críticos detectados en producción. Todo cambio hacia develop o main se incorporará exclusivamente mediante Pull Requests revisados minuciosamente.

16. Gestión ágil del proyecto

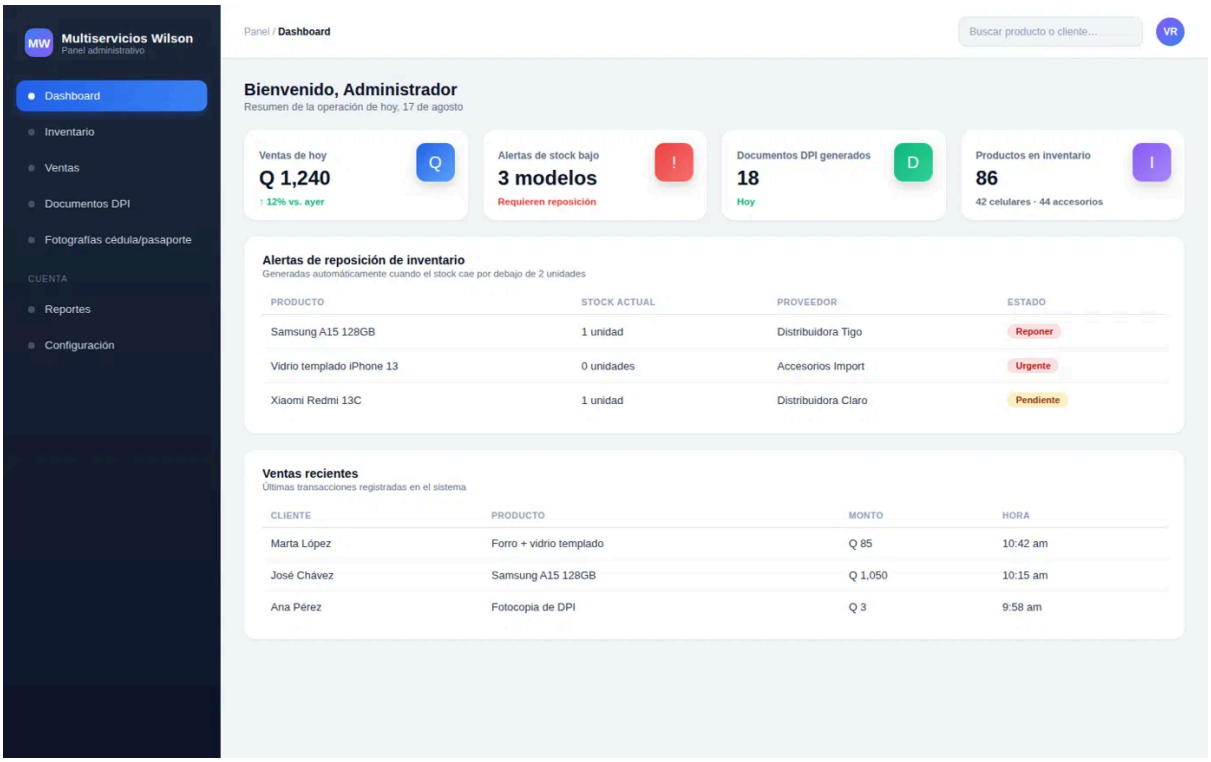
El proyecto se gestionará bajo el marco de trabajo Scrum, utilizando Jira como herramienta central de planificación, backlog y seguimiento. Las historias de usuario correspondientes a los Procesos 1 y 2 descritos en este documento constituirán el contenido inicial del Product Backlog, priorizado para los primeros sprints del proyecto. Se realizarán las ceremonias estándar de Scrum, planificación de sprint, sincronización semanal, revisión y retrospectiva, con una periodicidad de tres semanas por sprint.

17. Glosario de términos

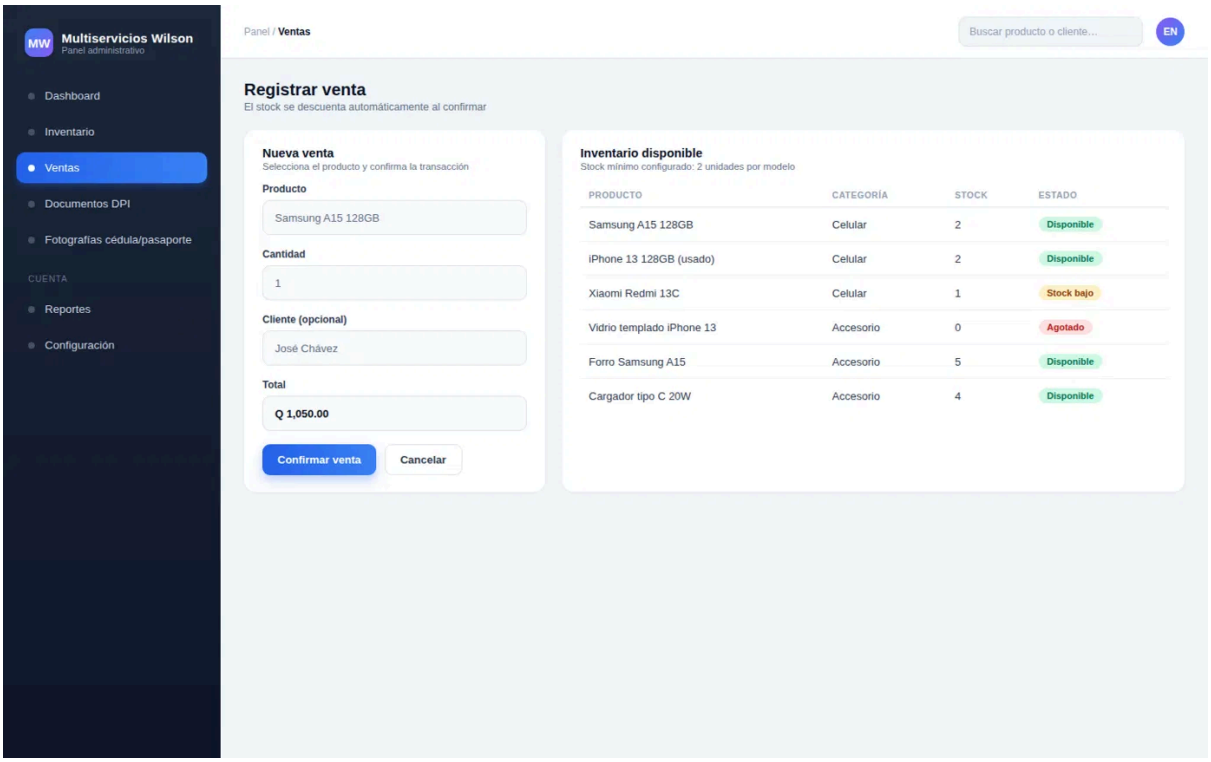
Tèrmino	Definición
BPM	Business Process Management, Disciplina de gestión centrada en identificar, modelar, mejorar y automatizar los procesos de negocio.
BPMN	Business Process Model and Notation. Estándar gráfico para modelar procesos de negocio mediante eventos, actividades, compuertas y carriles.
BPMS	Business Process Management System. Plataforma de software que ejecuta y automatiza procesos modelados en BPMN.
AS-IS	Representación del proceso en su situación actual, antes de cualquier mejora o automatización.
TO-BE	Representación del proceso en su situación propuesta, después de la mejora o automatización planteada.
Gateway (compuerta)	Elemento BPMN que representa un punto de decisión o divergencia en el flujo de un proceso.
Pool/ Lane	Contenedor BPMN que representa a un actor o rol responsable de un conjunto de actividades dentro del proceso.
RACI	Matriz de asignación de responsabilidades: Responsable, Aprueba, Consultado, Informado.

18. Mockups

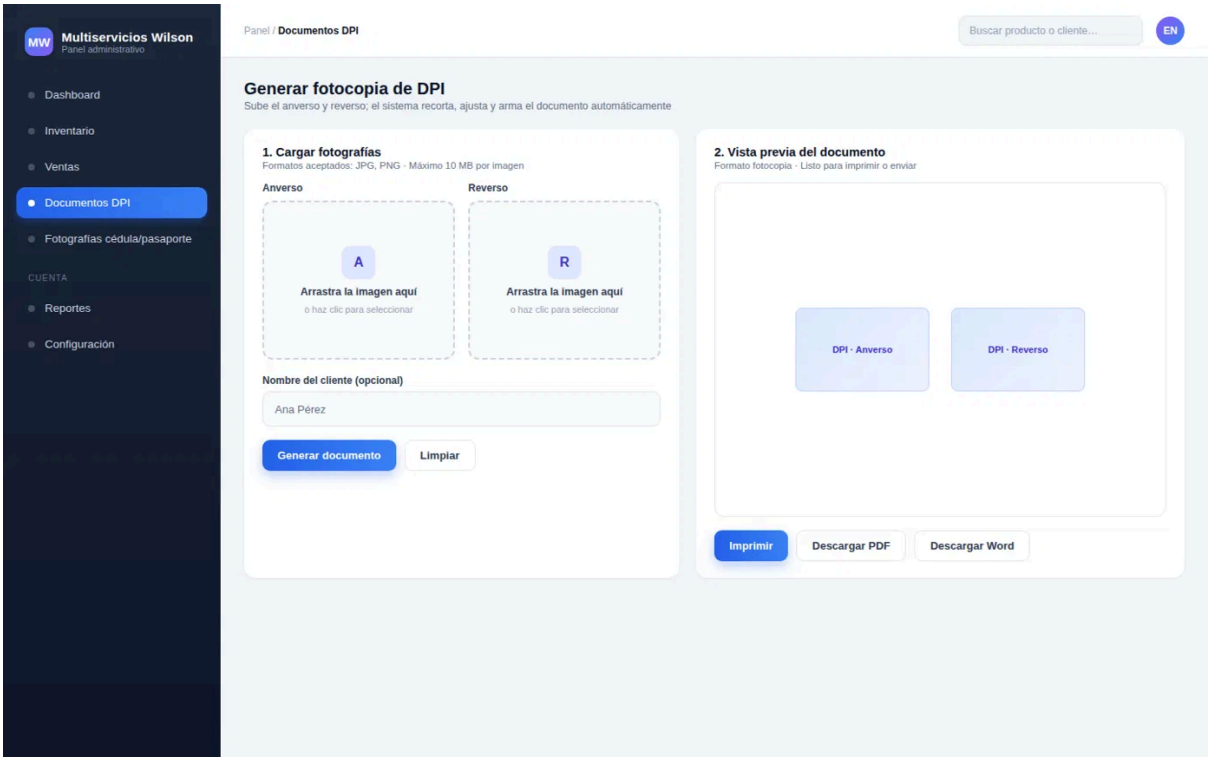
18.1. Dashboard



18.2. Ventas



18.3. Generación de docs.



19. FASE 2

19.1. Historias de Usuario — Sistema Multiservicios Wilson

Este documento recopila las historias de usuario de las funcionalidades ya construidas en el sistema (inventario, ventas, proveedores, alertas de stock, generación de documentos DPI, reportes, landing pública y administración de usuarios), organizadas por Epic.

19.1.1. Epic: Landing Publica

HU-01: Ver información del negocio sin iniciar sesión

Como visitante, quiero ver información del negocio (servicios, horario, ubicación y contacto) en una página pública, para decidir si visitar el negocio sin necesitar una cuenta.

- La página es accesible en la ruta raíz sin necesidad de iniciar sesión.
- Muestra nombre del negocio, eslogan, lista de servicios, horario, dirección y teléfono.
- Incluye un enlace a Google Maps con la ubicación del negocio.

HU-02: Acceso directo al login desde la landing

Como miembro del equipo que visita la página pública, quiero un acceso directo al inicio de sesión, para entrar al sistema interno sin buscar la URL por separado.

- Existe un botón/enlace visible que lleva a la pantalla de inicio de sesión.
- El acceso está visualmente diferenciado del contenido informativo público.

19.1.2. Epic: Autenticación y sesión

HU-03: Iniciar sesión

Como Encargado o Administrador, quiero iniciar sesión con usuario y contraseña, para acceder a las funciones del sistema según mi rol.

- Formulario de usuario y contraseña en la pantalla de inicio de sesión.
- Al autenticar correctamente se crea una sesión válida por 7 días.
- Tras iniciar sesión, soy redirigido a la pantalla de inicio correspondiente a mi rol (Administrador → Dashboard, Encargado → Ventas).
- Credenciales inválidas muestran un mensaje de error genérico.

HU-04: Cerrar sesión

Como usuario autenticado, quiero cerrar mi sesión, para proteger el acceso al sistema cuando termino de usarlo.

- Un botón de cerrar sesión invalida la sesión activa.

- Tras cerrar sesión, cualquier intento de acceder a una ruta protegida redirige al inicio de sesión.

HU-05: Restricción de acciones sensibles por rol

Como Administrador, quiero que las acciones sensibles del sistema (eliminar, desactivar, atender alertas, gestionar usuarios) estén restringidas únicamente a mi rol, para mantener el control operativo del negocio.

- Las pantallas de Dashboard/Inicio y Usuarios solo son visibles y accesibles para el rol Administrador.
- Los endpoints de desactivar/reactivar producto, eliminar proveedor, atender alerta y gestión de usuarios responden con error si el usuario autenticado no tiene el rol Administrador.

19.1.3. Epic: Dashboard / Inicio (Administrador)

HU-06: Ver resumen general del negocio

Como Administrador, quiero ver un resumen del estado del negocio al entrar al sistema, para tener una visión rápida sin revisar cada módulo por separado.

- Muestra el total y la cantidad de ventas del día actual.
- Muestra la cantidad de alertas de stock pendientes.
- Muestra la cantidad de notificaciones no leídas de documentos generados.
- Muestra el total de productos registrados y cuántos están agotados.

HU-07: Ver alertas de stock recientes desde el dashboard

Como Administrador, quiero ver las alertas de stock más recientes desde el dashboard, para identificar rápidamente qué productos reabastecer.

- Lista las 6 alertas pendientes más recientes con producto, stock actual y proveedor.

HU-08: Ver ventas recientes desde el dashboard

Como Administrador, quiero ver las ventas más recientes desde el dashboard, para monitorear la actividad del negocio sin entrar al módulo de Ventas.

- Lista las 6 ventas más recientes con cliente, fecha/hora y total.

19.1.4. Epic: Perfil

HU-09: Ver mis datos de perfil

Como usuario autenticado, quiero ver mis datos de perfil (nombre, usuario, correo y rol), para confirmar con qué cuenta y permisos estoy trabajando.

- Muestra nombre, nombre de usuario, correo electrónico y rol del usuario en sesión.

19.1.5. Epic: Productos

HU-10: Ver el listado de productos

Como Encargado o Administrador, quiero ver la lista de todos los productos con su stock y estado, para conocer el inventario disponible.

- La tabla muestra nombre, descripción, código de barras, categoría, precio, proveedor, stock y estado (Disponible / Stock Bajo / Agotado).
- Los productos inactivos se muestran diferenciados visualmente.

HU-11: Buscar productos

Como Encargado o Administrador, quiero buscar productos por nombre o código de barras, para encontrar un producto rápidamente sin recorrer toda la lista.

- Un campo de búsqueda filtra la tabla en tiempo real por nombre o código de barras.

HU-12: Registrar un nuevo producto

Como Encargado o Administrador, quiero registrar un nuevo producto (nombre, código de barras, descripción, categoría, precio, stock inicial, stock mínimo y proveedor opcional), para incorporarlo al inventario.

- Valida que nombre, código de barras y categoría estén presentes, y que stock y precio no sean negativos.
- Rechaza un código de barras duplicado.
- Si no se especifica un stock mínimo, se usa un valor por defecto configurado en el sistema.

HU-13: Editar un producto existente

Como Encargado o Administrador, quiero editar los datos de un producto existente, para corregir o actualizar su información.

- El formulario se muestra prellenado con los datos actuales del producto.
- Los cambios se guardan sobre el mismo producto, sin crear uno nuevo.

HU-14: Desactivar un producto

Como Administrador, quiero desactivar un producto, para retirarlo del catálogo activo sin perder su historial de ventas y alertas.

- La acción solo está disponible para el rol Administrador, tanto en la interfaz como en el servidor.

- El producto pasa a estado inactivo sin eliminarse físicamente ni perder su historial asociado.

HU-15: Reactivar un producto

Como Administrador, quiero reactivar un producto previamente desactivado, para volver a ponerlo disponible en el inventario.

- La acción solo está disponible para el rol Administrador.
- El producto vuelve a mostrarse como activo en el listado.

19.1.6. Epic: Proveedores

HU-16: Ver el listado de proveedores

Como Encargado o Administrador, quiero ver la lista de proveedores registrados, para saber a quién contactar para reabastecer productos.

HU-17: Registrar un nuevo proveedor

Como Encargado o Administrador, quiero registrar un nuevo proveedor (nombre, contacto y teléfono), para asociarlo a los productos que surte.

HU-18: Editar un proveedor

Como Encargado o Administrador, quiero editar los datos de contacto de un proveedor, para mantener su información actualizada.

HU-19: Eliminar un proveedor

Como Administrador, quiero eliminar un proveedor, para depurar proveedores que ya no trabajan con el negocio.

- La acción está restringida al rol Administrador tanto en la interfaz como en el servidor.

19.1.7. Epic: Ventas

HU-20: Registrar una venta

Como Encargado o Administrador, quiero registrar una venta con uno o varios productos, cantidades y precios, para dejar constancia de cada transacción realizada.

- Requiere el nombre del cliente.
- Permite agregar varias líneas de producto, cada una con cantidad y precio unitario (autocompletado desde el precio del producto, pero editable).
- Calcula el subtotal por línea y el total general de la venta.

- La venta queda asociada al Encargado o Administrador que la registró.

HU-21: Evitar vender más stock del disponible

Como Administrador, quiero que el sistema rechace una venta si la cantidad solicitada de algún producto supera el stock disponible, para evitar inconsistencias en el inventario.

- Si algún artículo de la venta excede el stock disponible, la venta completa se rechaza y no se descuenta stock de ningún producto.

HU-22: Descuento automático de stock al vender

Como Administrador, quiero que el stock de cada producto se descuente automáticamente al confirmarse una venta, para mantener el inventario siempre actualizado sin trabajo manual.

HU-23: Alerta automática de stock bajo tras una venta

Como Administrador, quiero que se genere una alerta automáticamente cuando una venta deja a un producto por debajo de su stock mínimo, para saber cuándo reabastecer sin revisar el inventario manualmente.

- No se genera una alerta duplicada si ya existe una pendiente para ese mismo producto.

HU-24: Ver ventas recientes

Como Encargado o Administrador, quiero ver un listado de las ventas recientes, para consultar el historial reciente de transacciones.

- La lista muestra cliente, fecha, cantidad de artículos y total de cada venta.

19.1.8. Epic: Alertas de stock

HU-25: Ver alertas de stock pendientes

Como Encargado o Administrador, quiero ver las alertas de stock pendientes, para saber qué productos es necesario reabastecer.

- Cada alerta muestra producto, stock actual, proveedor y fecha de creación.
- El nivel de stock se resalta visualmente según qué tan crítico es.

HU-26: Marcar una alerta como atendida

Como Administrador, quiero marcar una alerta de stock como atendida, para llevar control de qué reabastecimientos ya se gestionaron.

- La acción solo está disponible para el rol Administrador.
- La alerta pasa a estado "Atendida" con la fecha correspondiente; no modifica el stock del producto de forma automática.

19.1.9. Epic: Documentos DPI

HU-27: Subir las fotos del DPI

Como Encargado o Administrador, quiero subir las fotos de anverso y reverso de un DPI, para generar una copia digital del documento.

- Solo se aceptan imágenes en formato JPEG o PNG.
- Ambas fotos (anverso y reverso) son obligatorias para generar el documento.

HU-28: Detección automática del borde de la tarjeta

Como Encargado o Administrador, quiero que el sistema detecte automáticamente el borde de la tarjeta en cada foto, para no tener que recortarla a mano en la mayoría de los casos.

HU-29: Ajuste manual del recorte

Como Encargado o Administrador, quiero poder ajustar manualmente las 4 esquinas del recorte cuando la detección automática se equivoque, para asegurar que el documento final se vea correcto.

- Un overlay muestra las 4 esquinas detectadas sobre la imagen, cada una arrastrable de forma independiente.
- Existe una opción para restaurar las esquinas a la detección automática original.
- Rotar la imagen limpia cualquier ajuste manual previo, mostrando una nueva detección automática.

HU-30: Rotar las fotos del DPI

Como Encargado o Administrador, quiero rotar cada foto 90° de forma independiente, para corregir la orientación de la imagen antes de generar el documento.

HU-31: Ajustar brillo, contraste y nitidez

Como Encargado o Administrador, quiero ajustar el brillo, contraste y nitidez de las fotos, para mejorar la legibilidad del documento antes de generarlo.

- La vista previa procesada se actualiza automáticamente al cambiar los ajustes.

HU-32: Elegir el formato de salida del documento

Como Encargado o Administrador, quiero elegir el formato de salida del documento (PDF o Word), para adaptarme a lo que necesite el cliente o el negocio.

HU-33: Generar y descargar el documento DPI

Como Encargado o Administrador, quiero generar y descargar el documento final con el

anverso y reverso del DPI, para entregárselo al cliente.

- El documento generado se puede descargar.
- Cuando el formato es PDF, también se puede imprimir directamente desde el sistema.

HU-34: Registro de notificación al generar un documento

Como Administrador, quiero que se registre una notificación cada vez que se genera un documento DPI, para llevar control de cuántos documentos se han producido.

19.1.10. Epic: Reportes

HU-35: Generar reporte de ventas por rango de fechas

Como Encargado o Administrador, quiero generar un reporte de ventas en un rango de fechas, para analizar el desempeño de ventas en un periodo específico.

- El rango por defecto es el día de hoy.
- El sistema rechaza un rango donde la fecha inicial es posterior a la fecha final.

HU-36: Generar reporte de inventario

Como Encargado o Administrador, quiero generar un reporte del inventario actual, para tener una fotografía completa del estado del stock.

HU-37: Exportar reportes en PDF o Excel

Como Encargado o Administrador, quiero exportar cualquier reporte en formato PDF o Excel, para compartirlo o analizarlo fuera del sistema.

19.1.11. Epic: Usuarios (Administrador)

HU-38: Ver el listado de cuentas del sistema

Como Administrador, quiero ver la lista de todas las cuentas del sistema (nombre, usuario, rol y estado), para tener control de quién tiene acceso.

HU-39: Crear una nueva cuenta de usuario

Como Administrador, quiero crear una nueva cuenta (nombre, usuario, contraseña y rol), para dar acceso al sistema a un nuevo Encargado o Administrador.

- Rechaza un nombre de usuario duplicado.
- No existe registro público: solo un Administrador puede crear cuentas nuevas.

HU-40: Editar una cuenta de usuario

Como Administrador, quiero editar el nombre y el rol de un usuario existente, para corregir datos o cambiar sus permisos.

- El nombre de usuario y la contraseña no se pueden modificar desde esta pantalla.

HU-41: Desactivar una cuenta de usuario

Como Administrador, quiero desactivar la cuenta de un usuario, para revocarle el acceso sin eliminar su historial de acciones en el sistema.

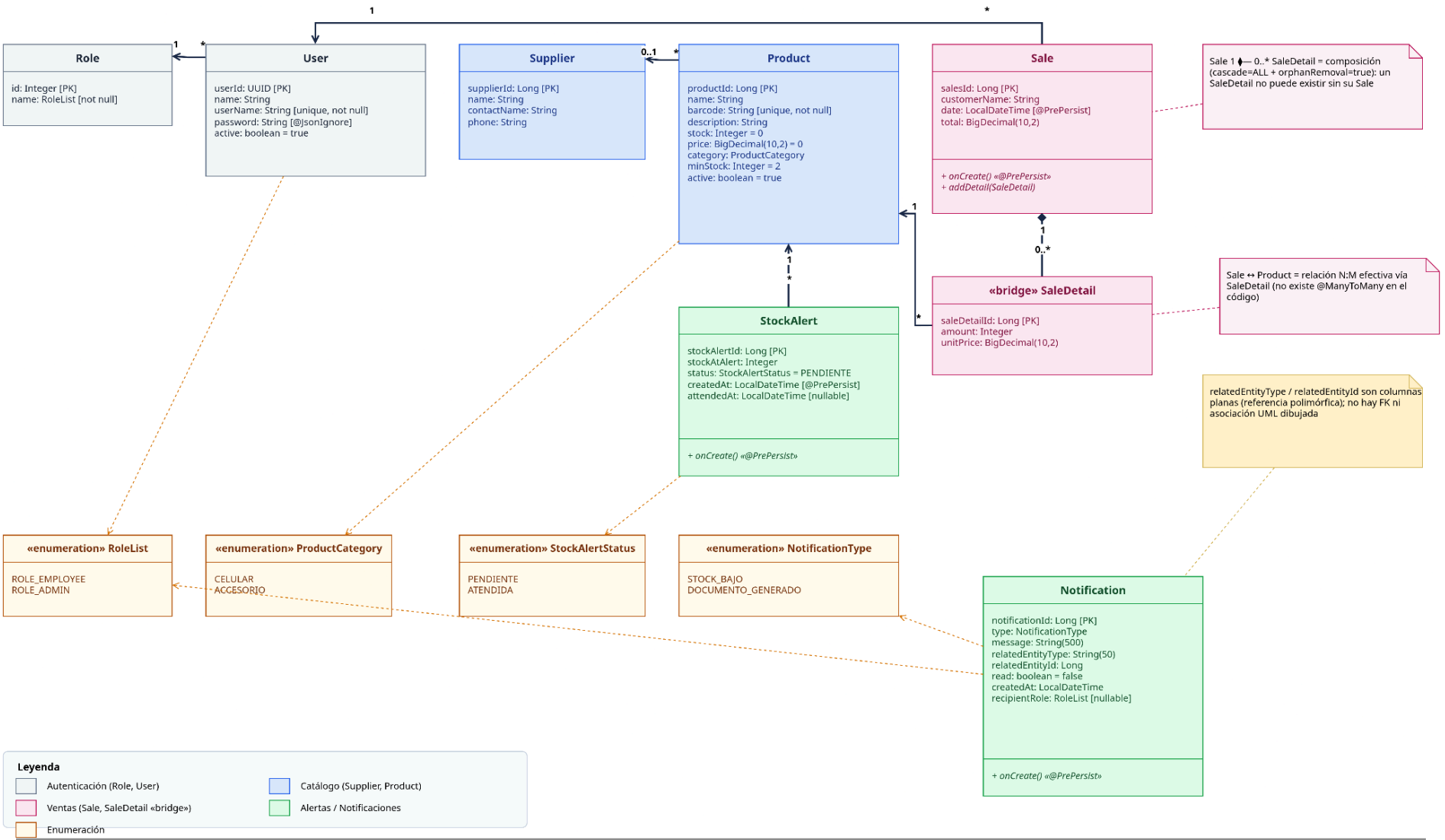
- Un usuario desactivado no puede iniciar sesión.

HU-42: Reactivar una cuenta de usuario

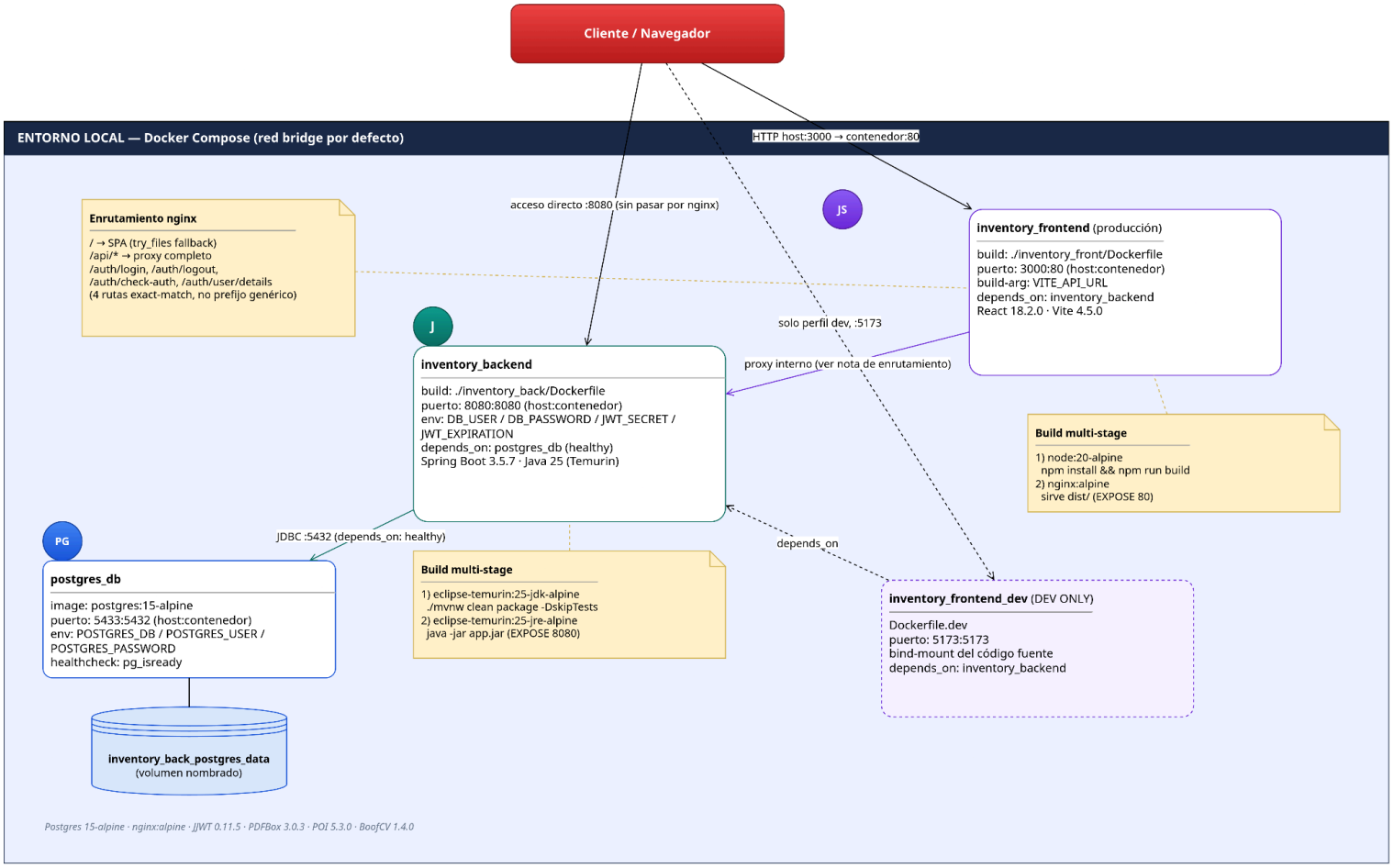
Como Administrador, quiero reactivar una cuenta de usuario previamente desactivada, para restaurarle el acceso cuando corresponda.

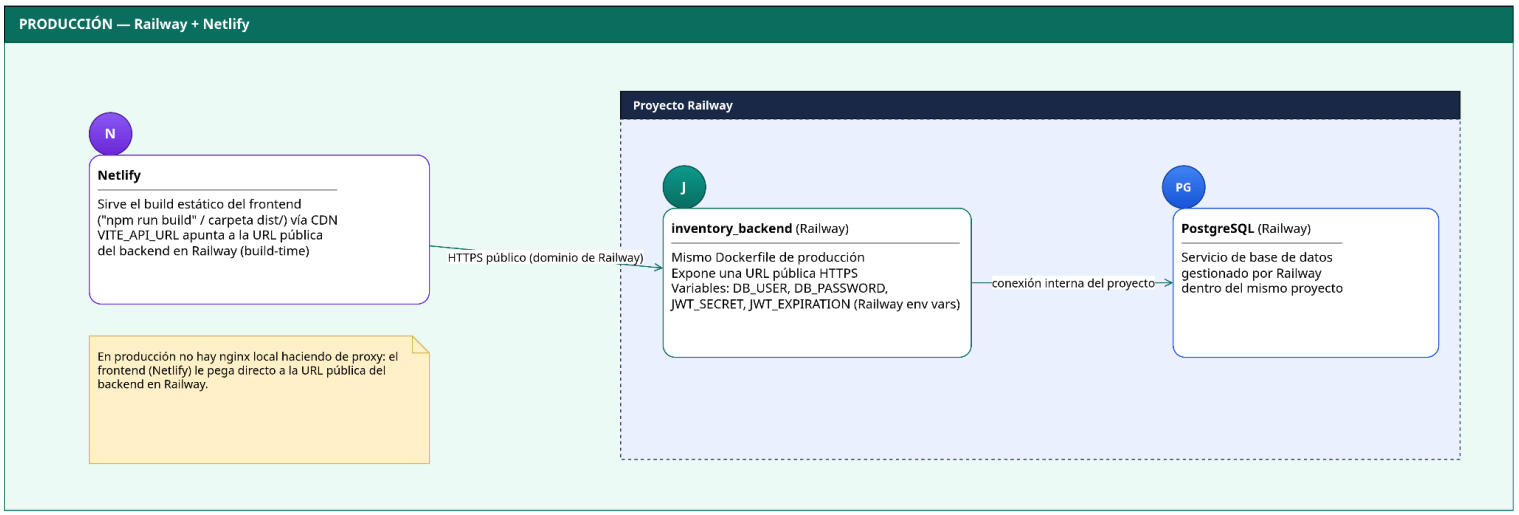
20. Diagramas UML

20.1. Diagrama de clases

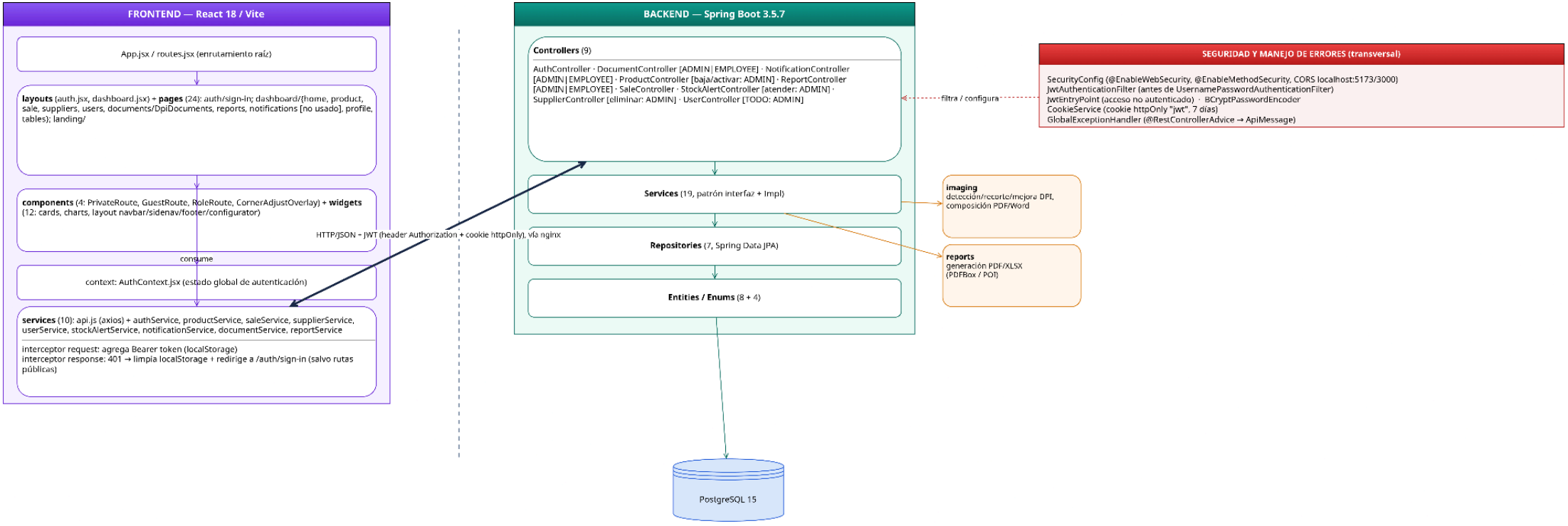


20.2. Diagrama de Infraestructura

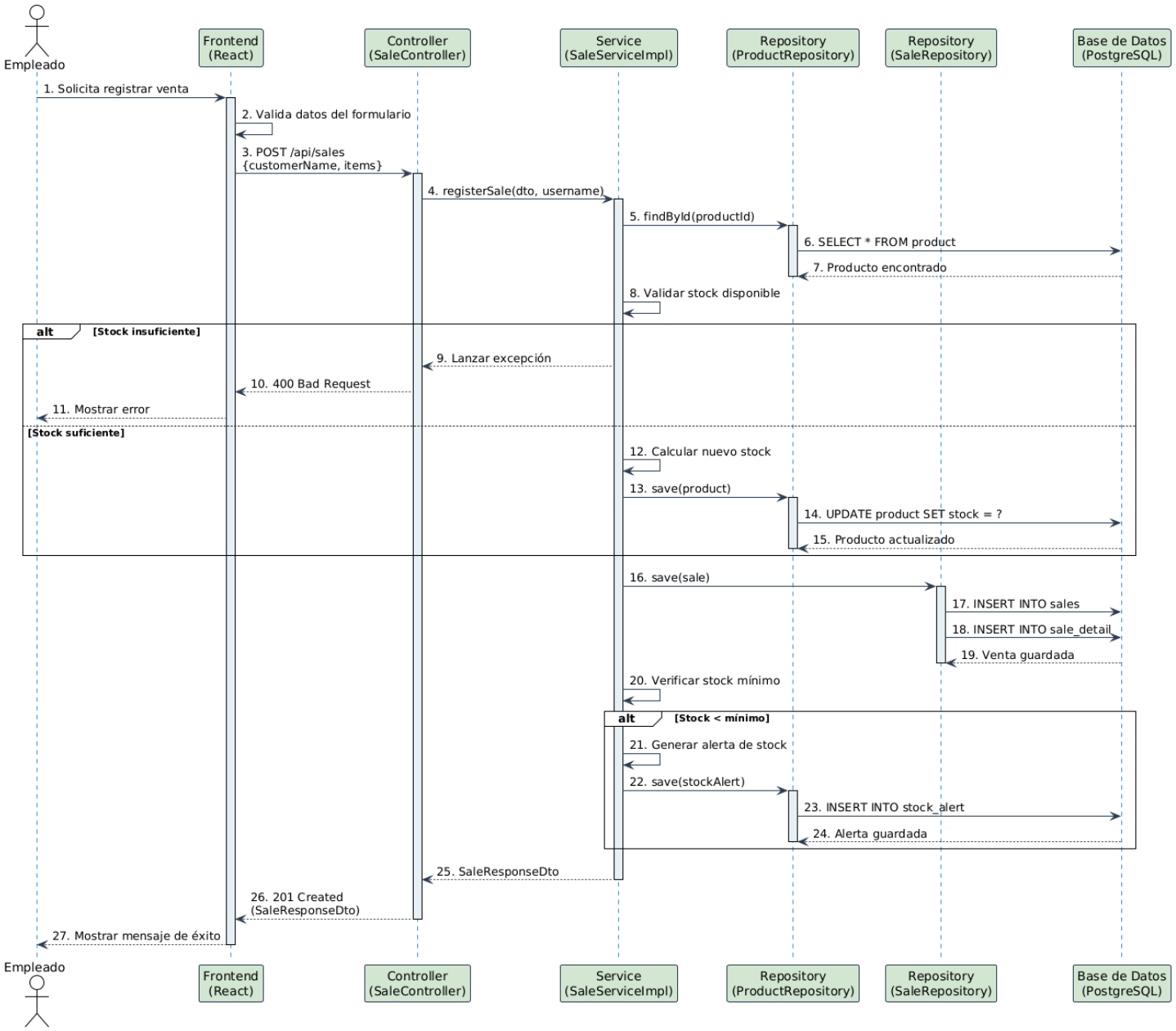




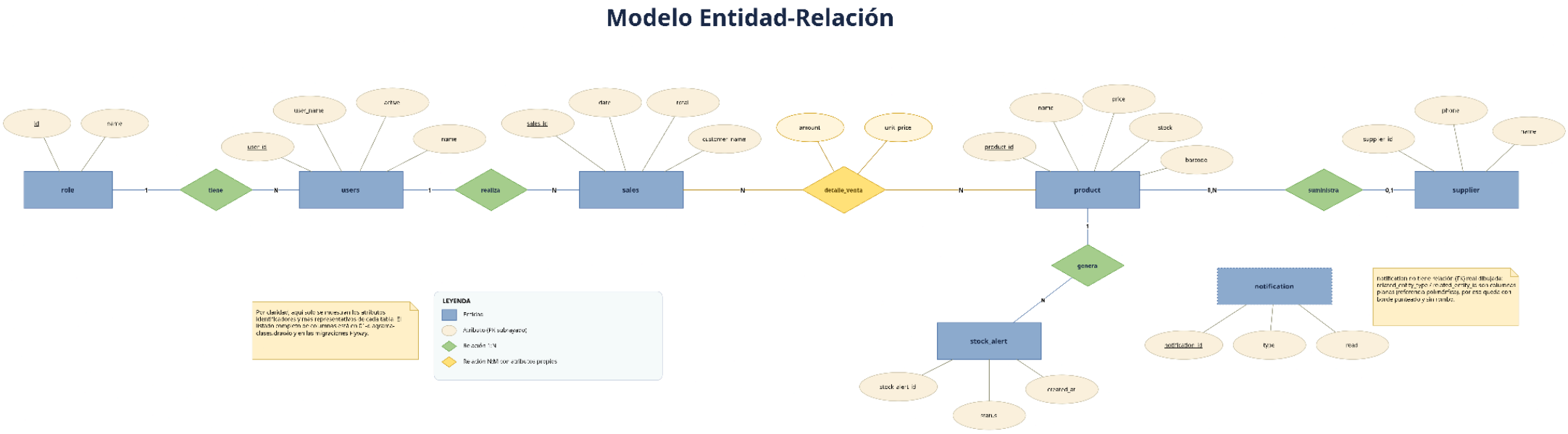
20.3. Diagrama de Arquitectura



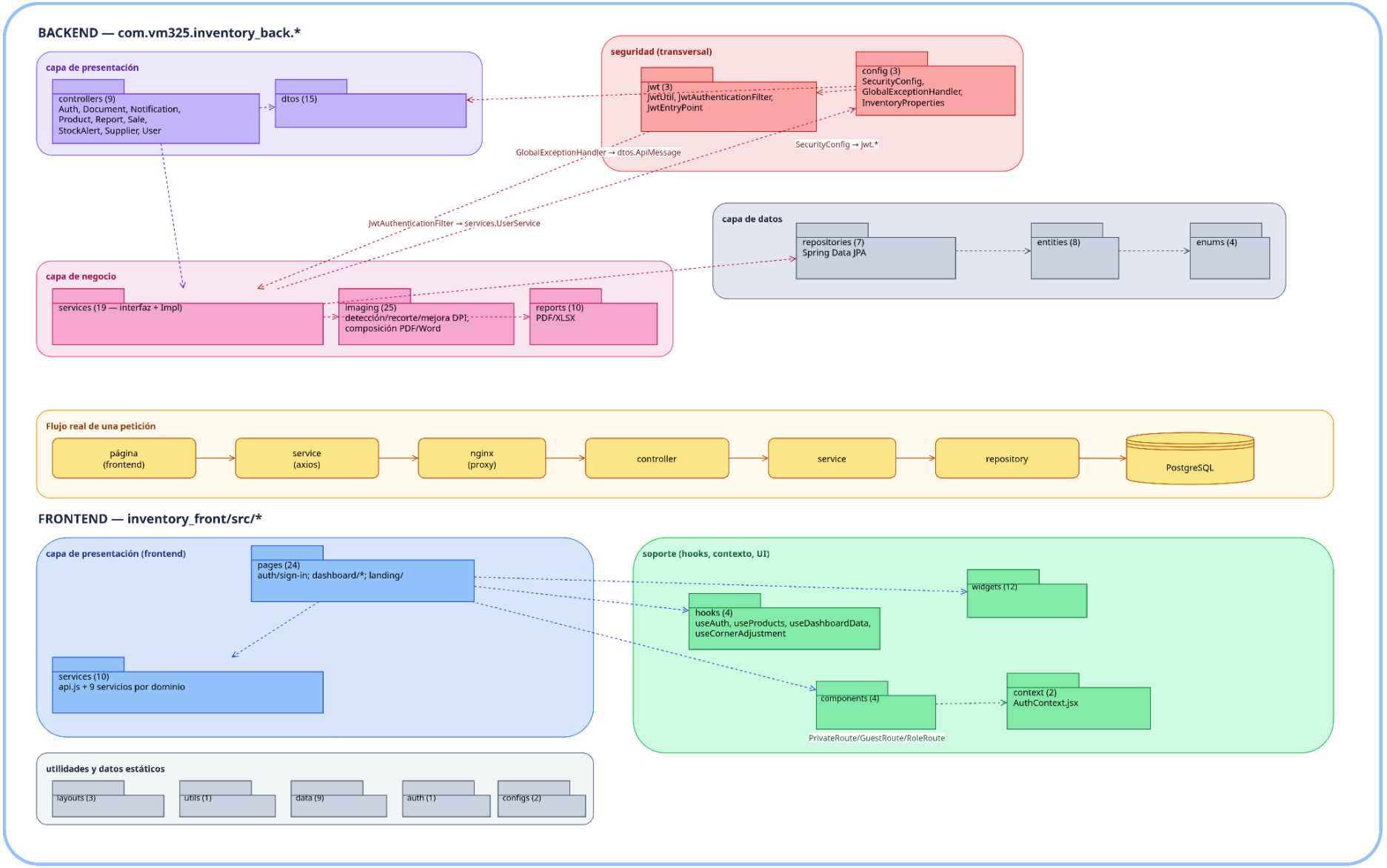
20.4. Diagrama de Secuencia



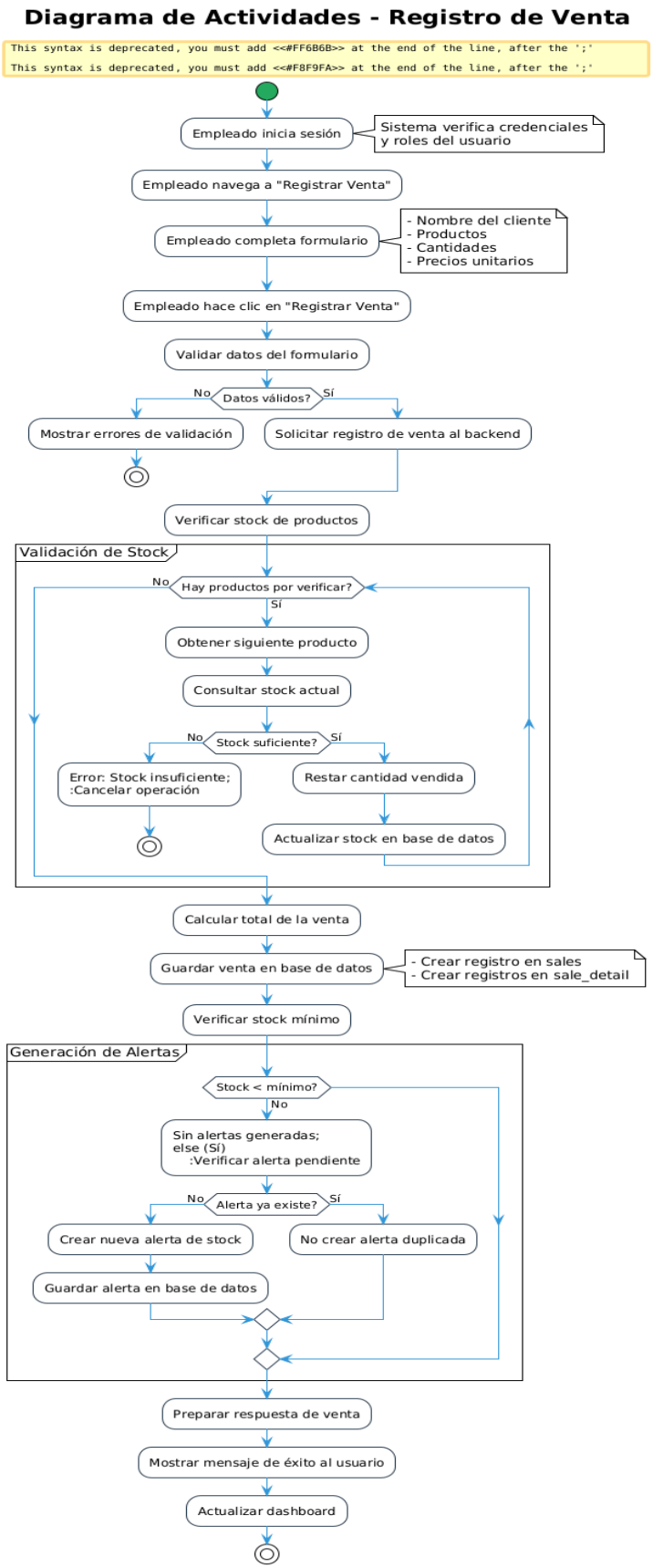
20.5. Diagrama de Entidad Relación



20.6. Diagrama de Paquetes

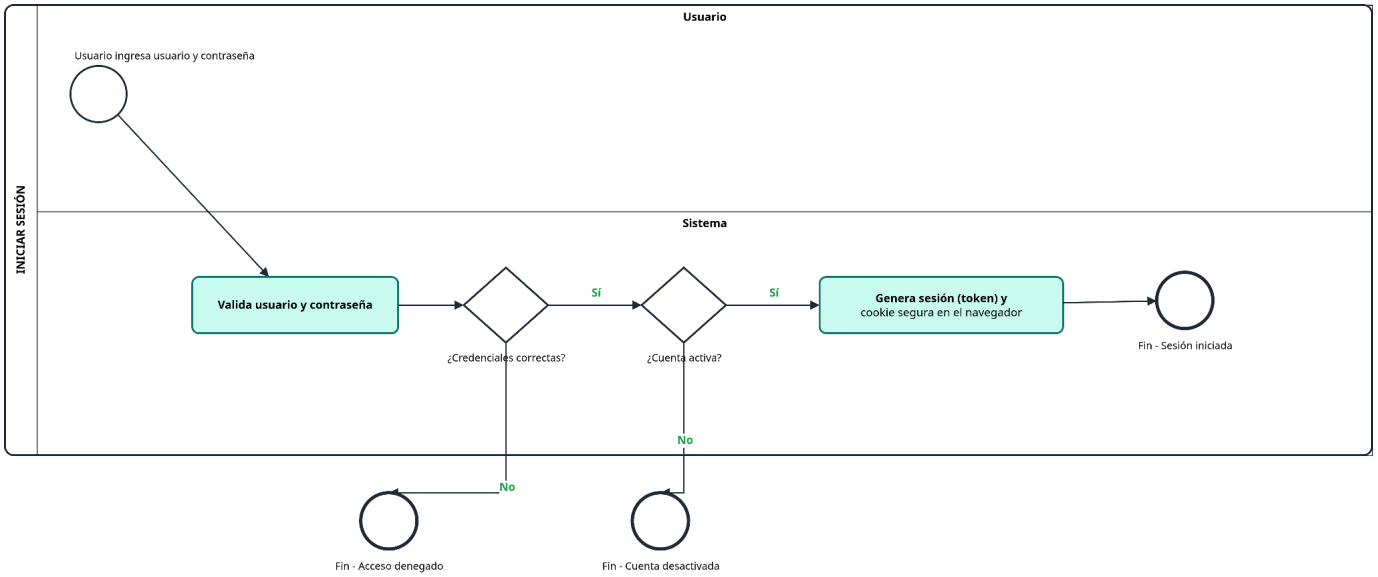


20.7. Diagrama de Actividades

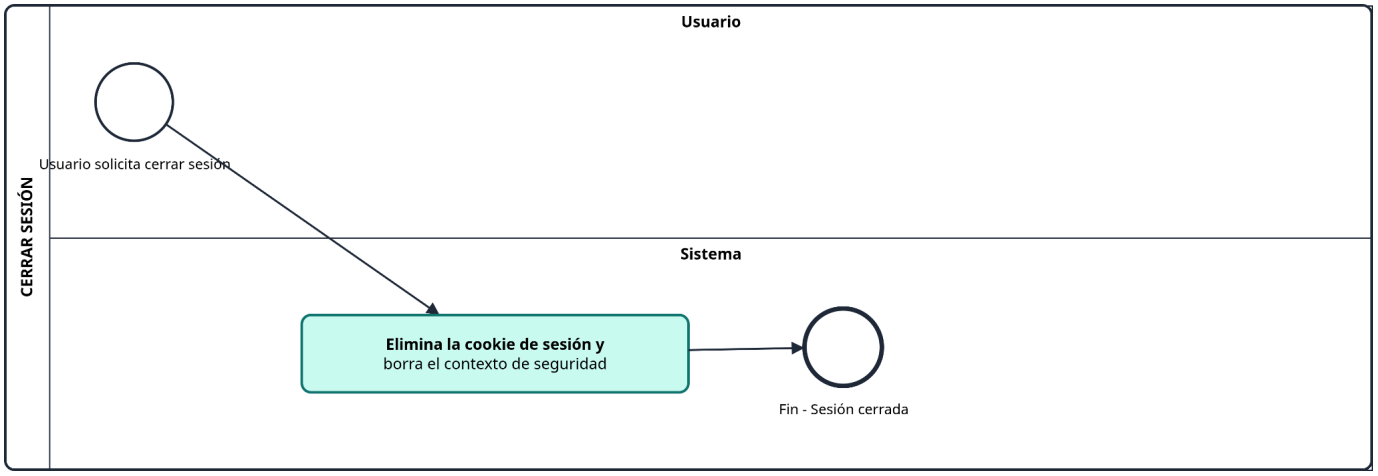


21. Modelos de Procesos Técnicos

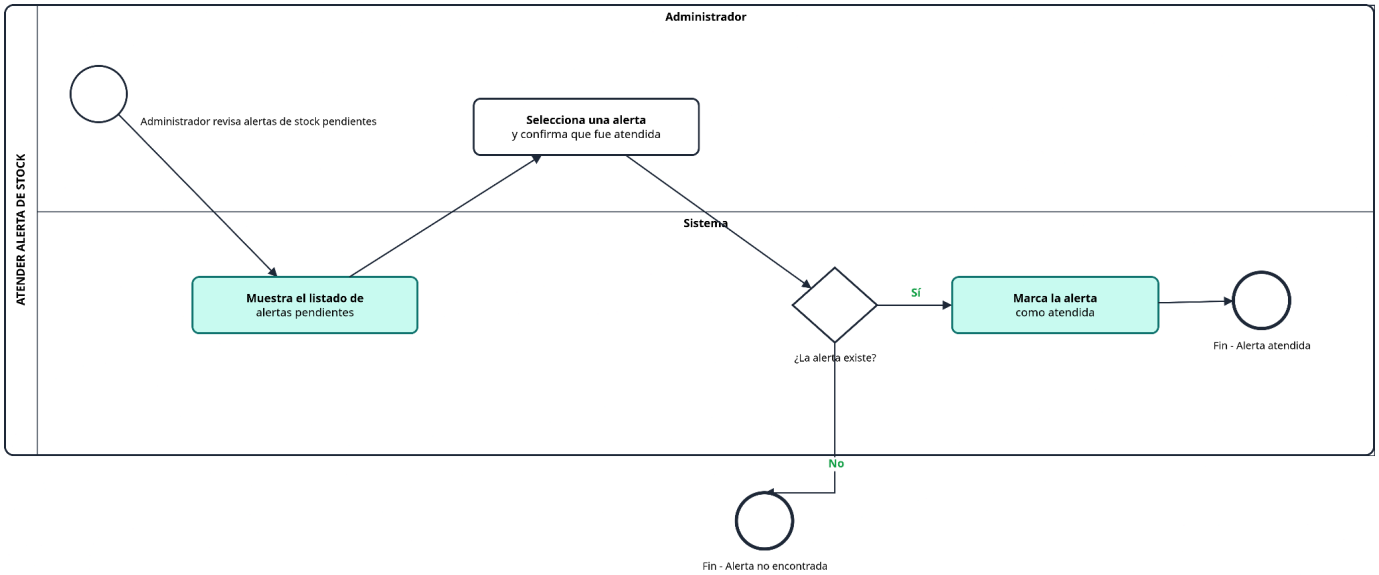
21.1. 01- Inicio Sesión



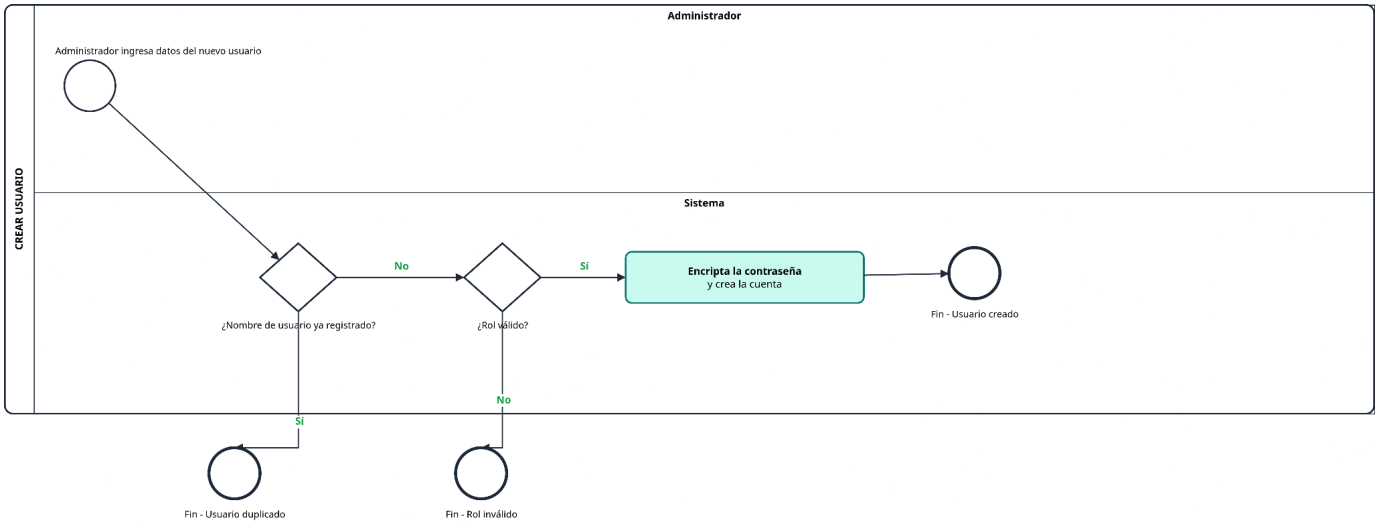
21.2. 02- Cerrar Sesión



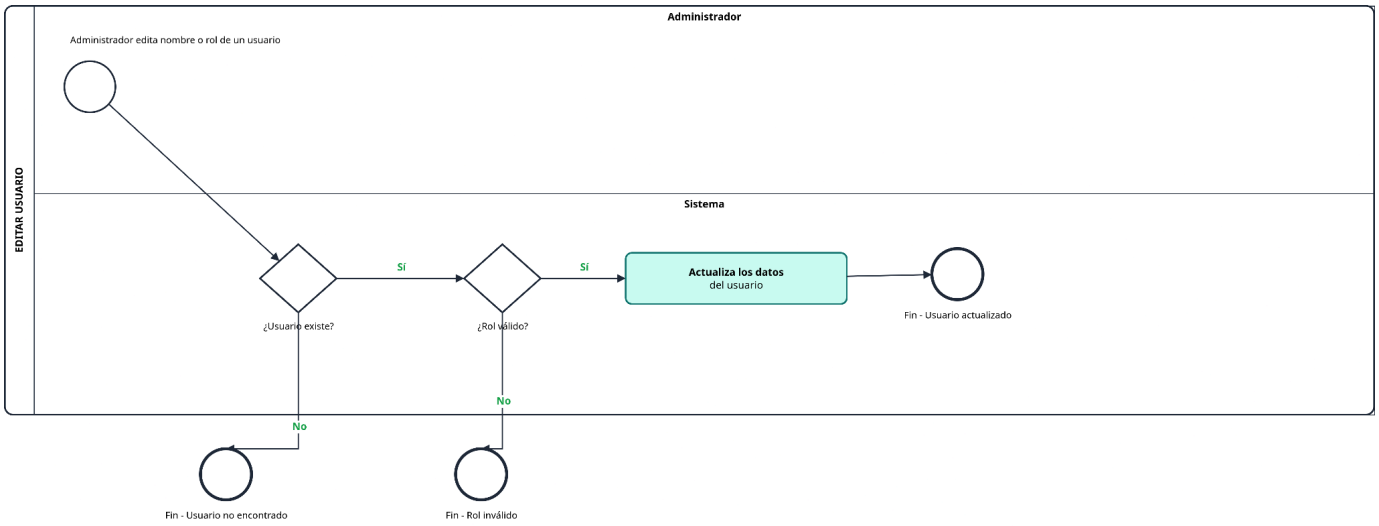
21.3. 03- Atender Alerta Stock



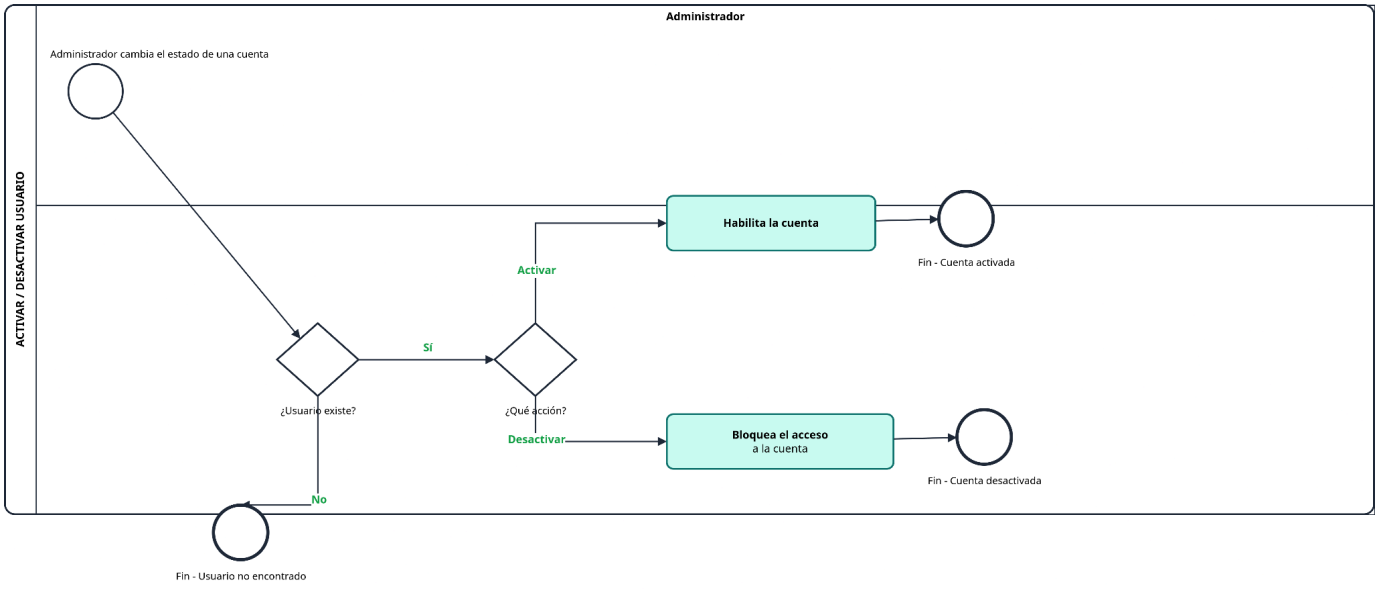
21.4. 04 - Crear Usuario



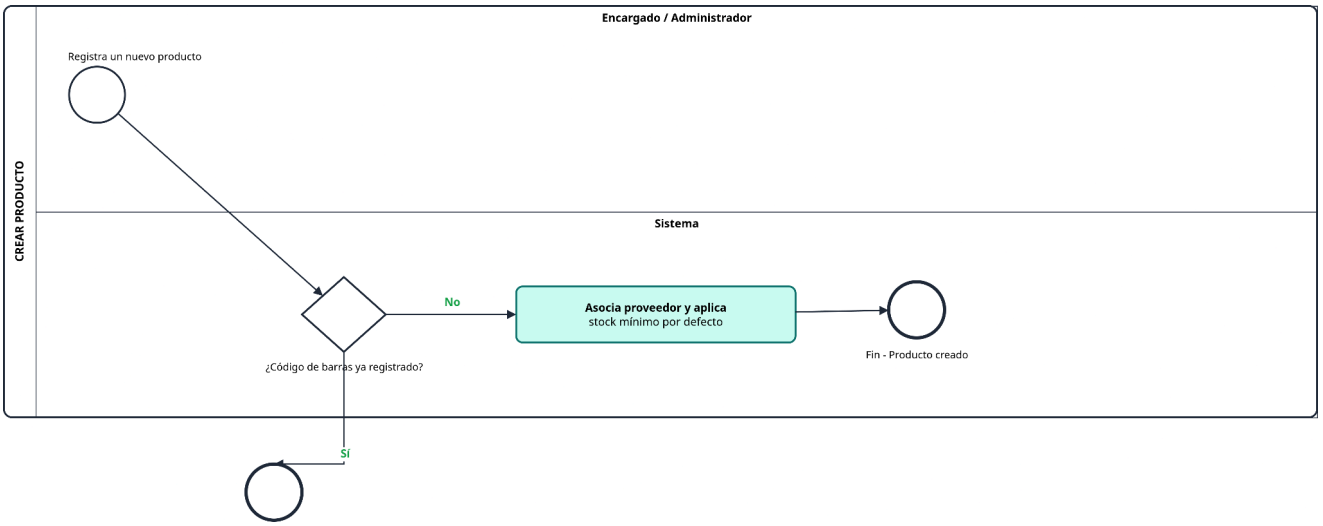
21.5. 05 - Editar Usuario



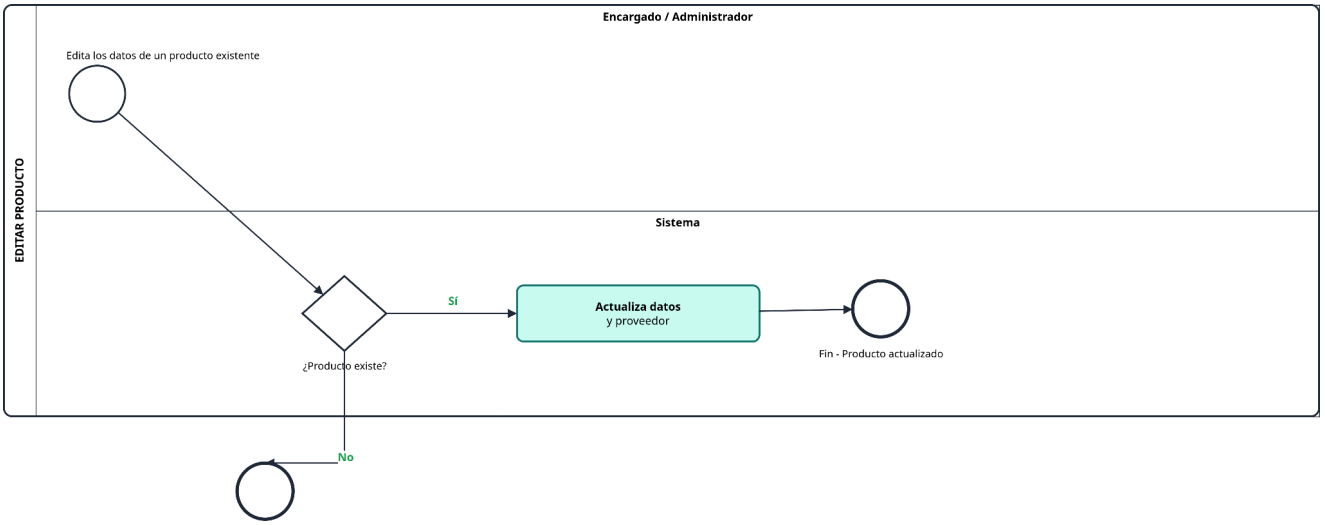
21.6. 06 - Activar/Desactivar Usuario



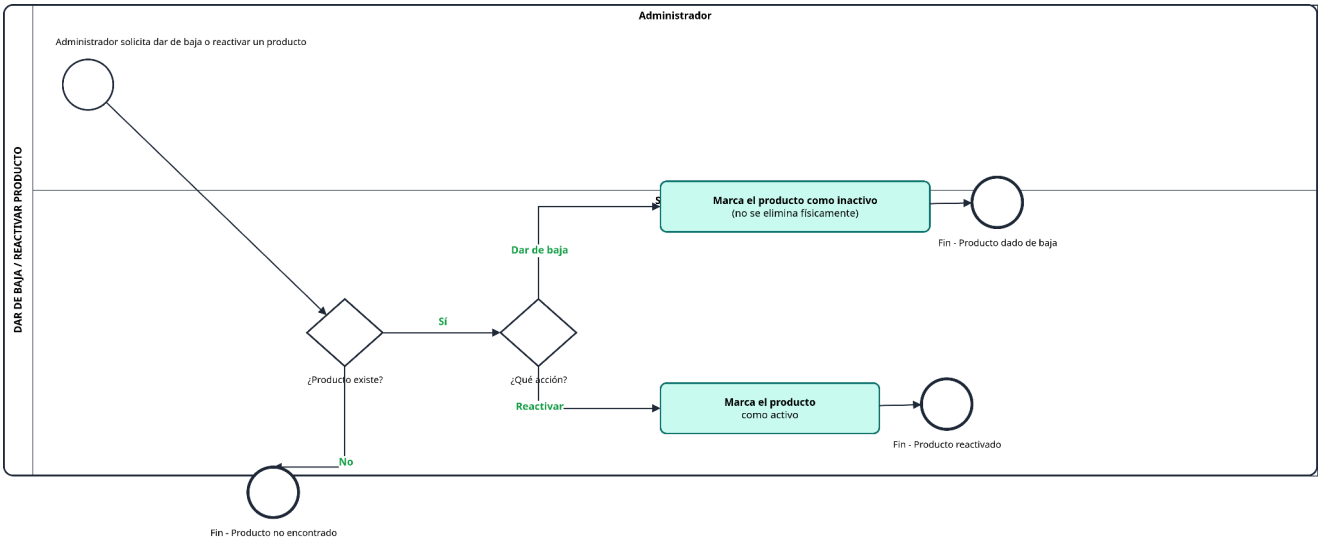
21.7. 07 - Crear Producto



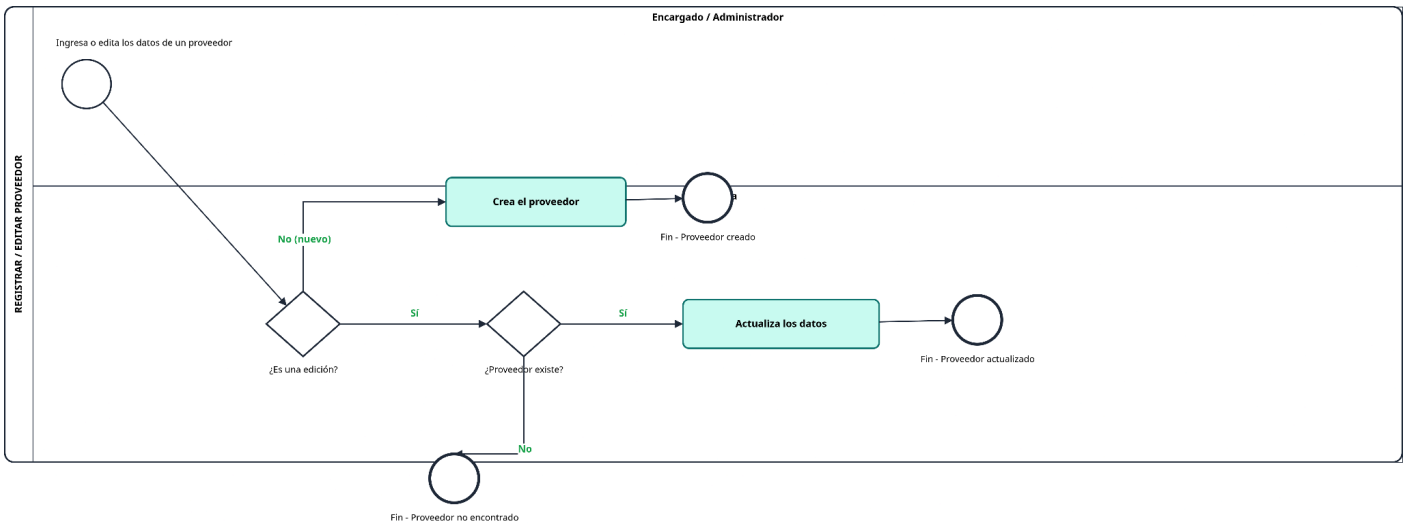
21.8. 08 - Editar Producto



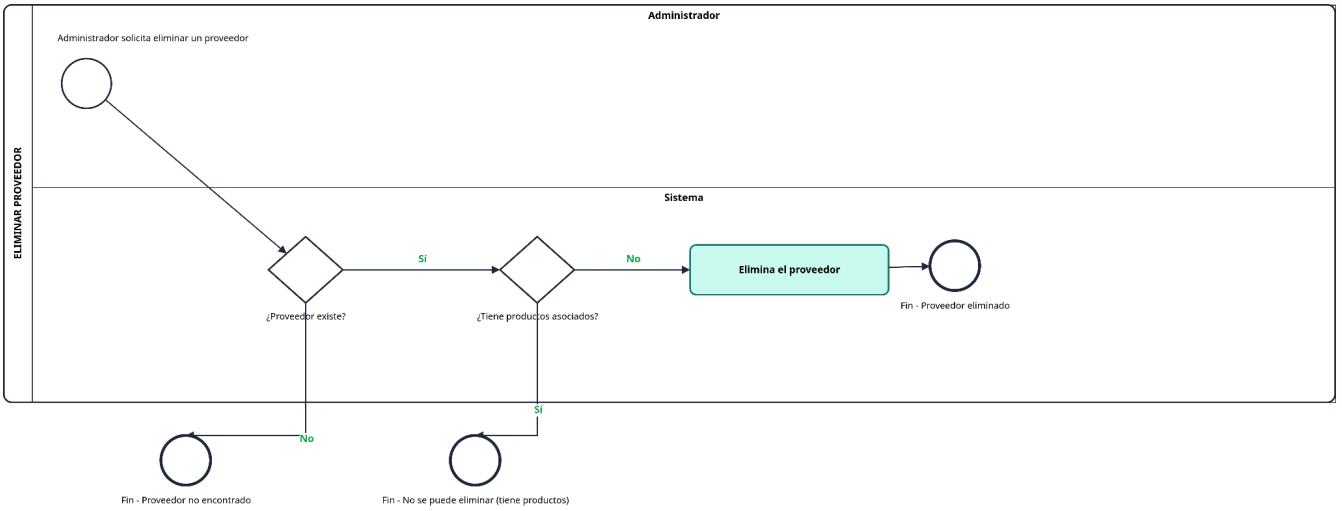
21.9. 09 - Dar Baja/Reactivar Producto



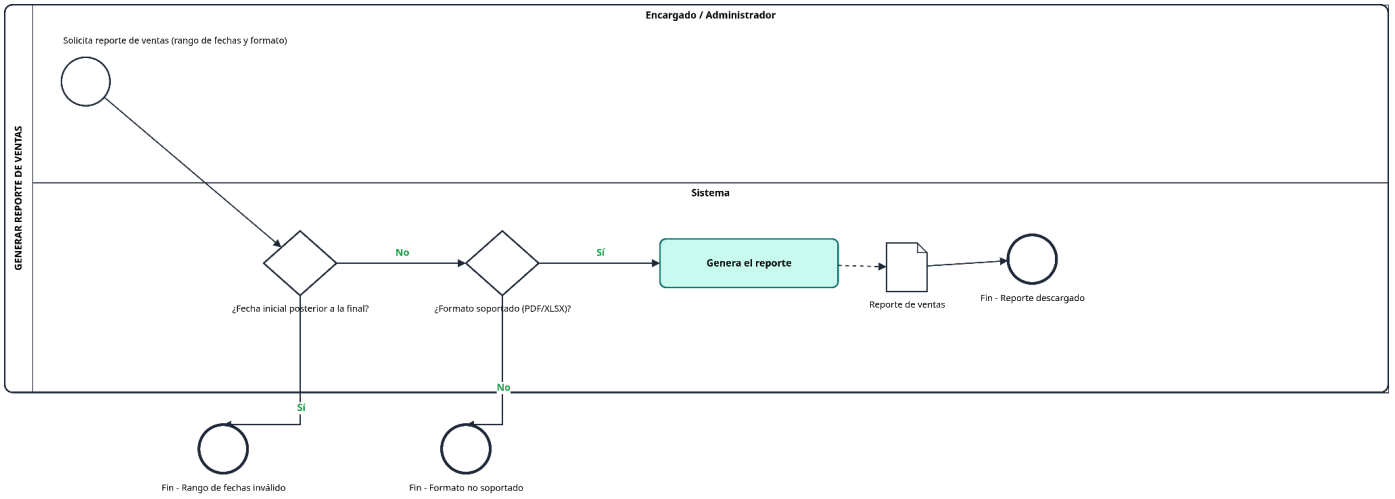
21.10. 10 - Registrar/Editar Proveedor



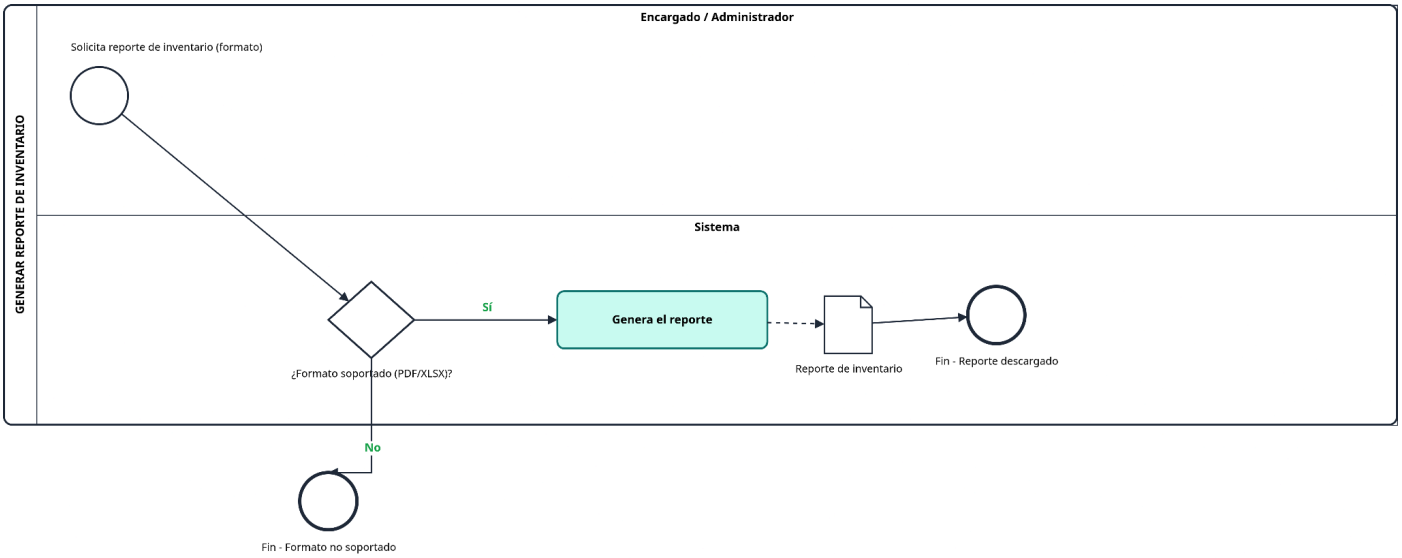
21.11. 11 - Eliminar Proveedor



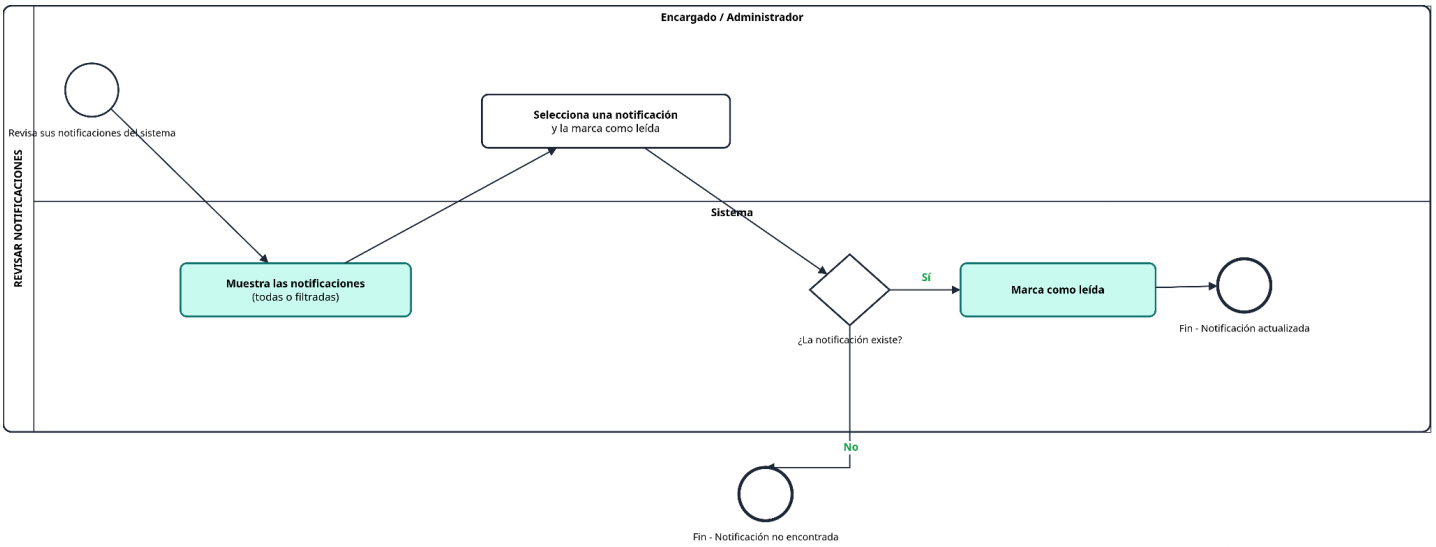
21.12. 12 - Reporte Ventas



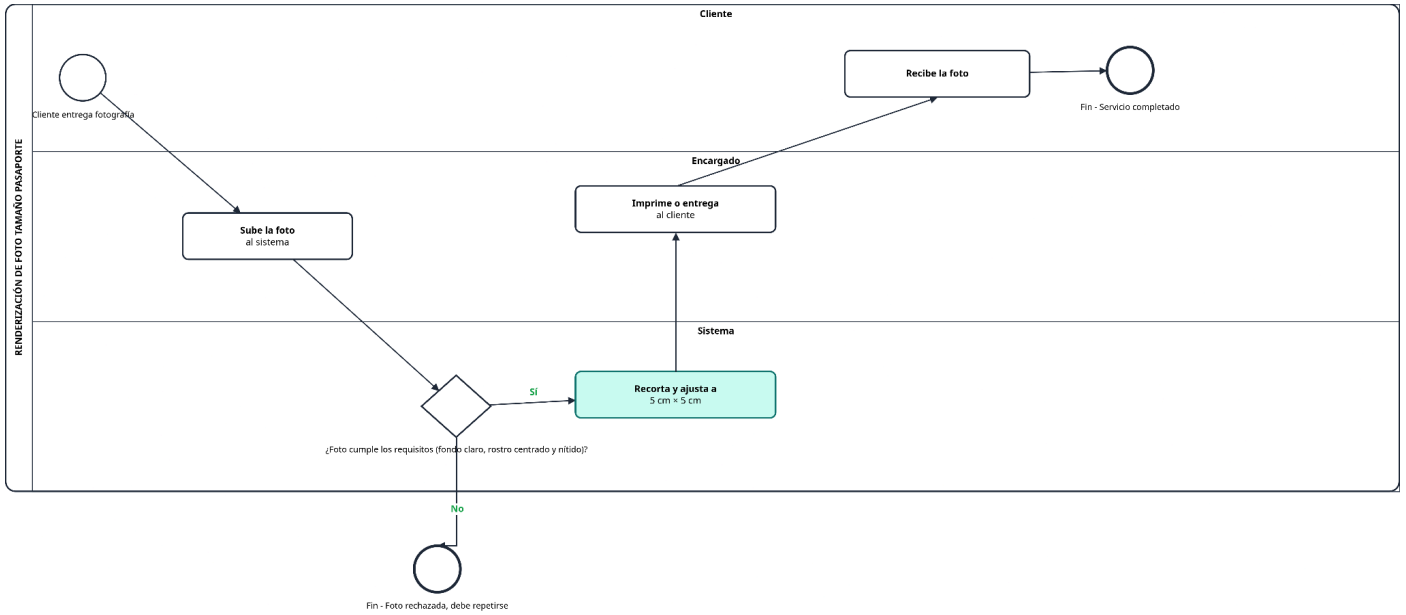
21.13. 13 - Reporte Inventario



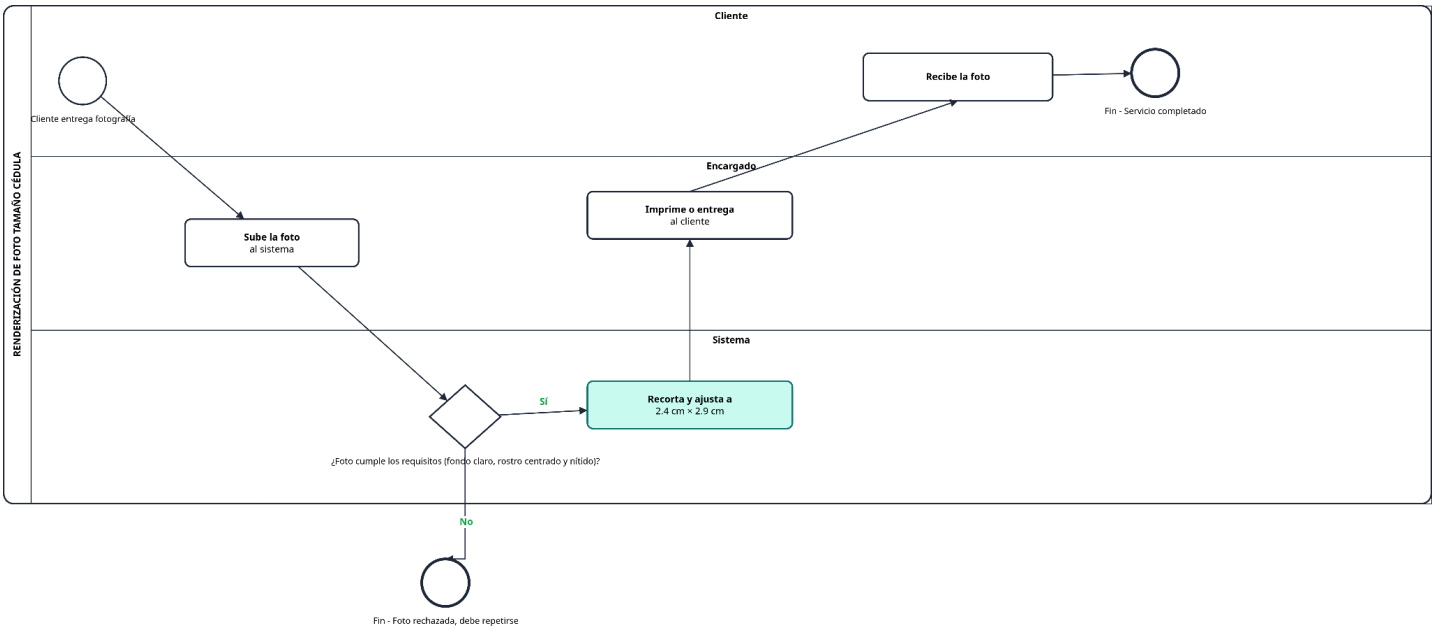
21.14. 14 - Reporte Notificaciones



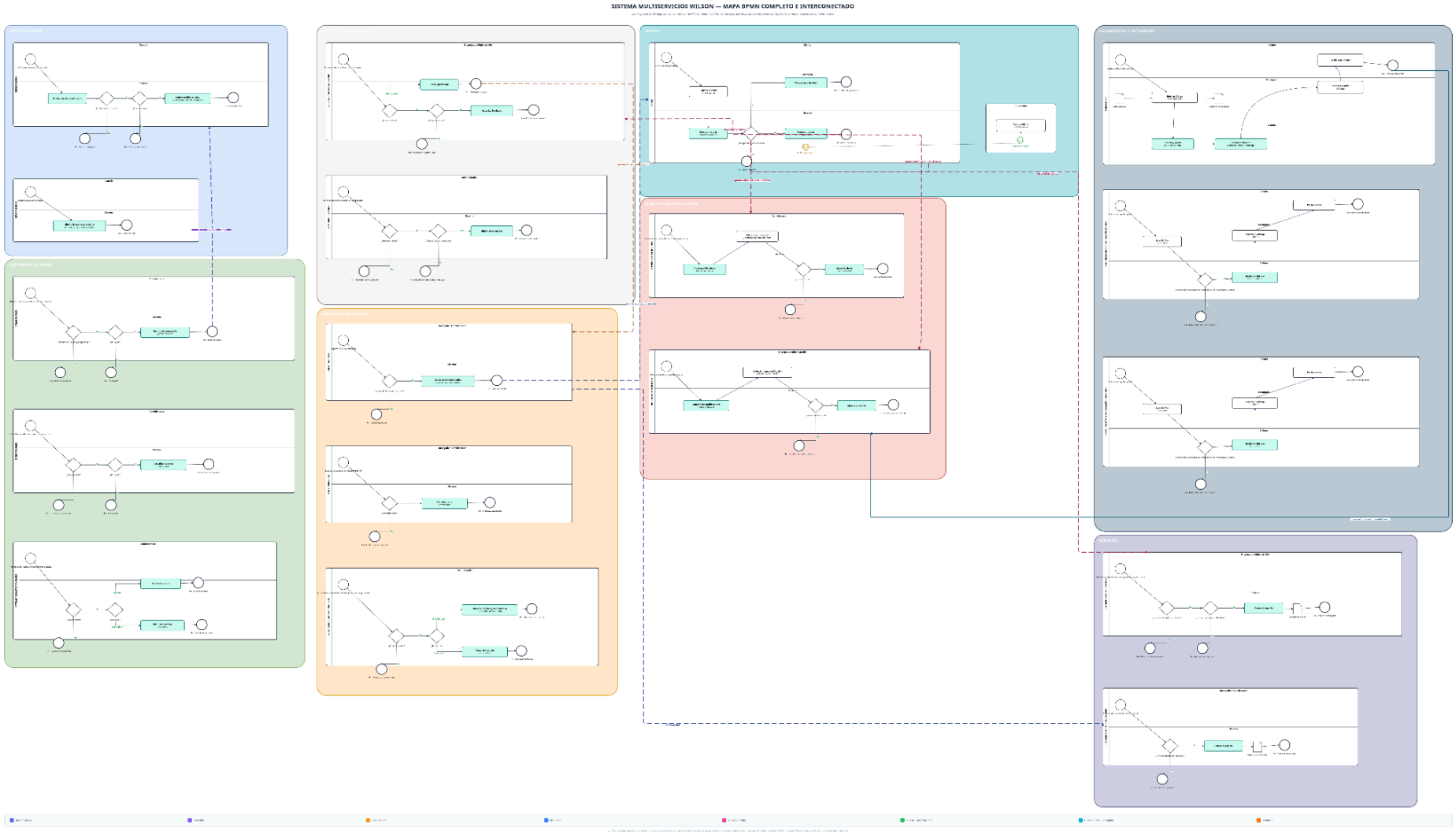
21.15. 15 - Renderización Pasaporte



21.16. 16 - Renderización Cédula



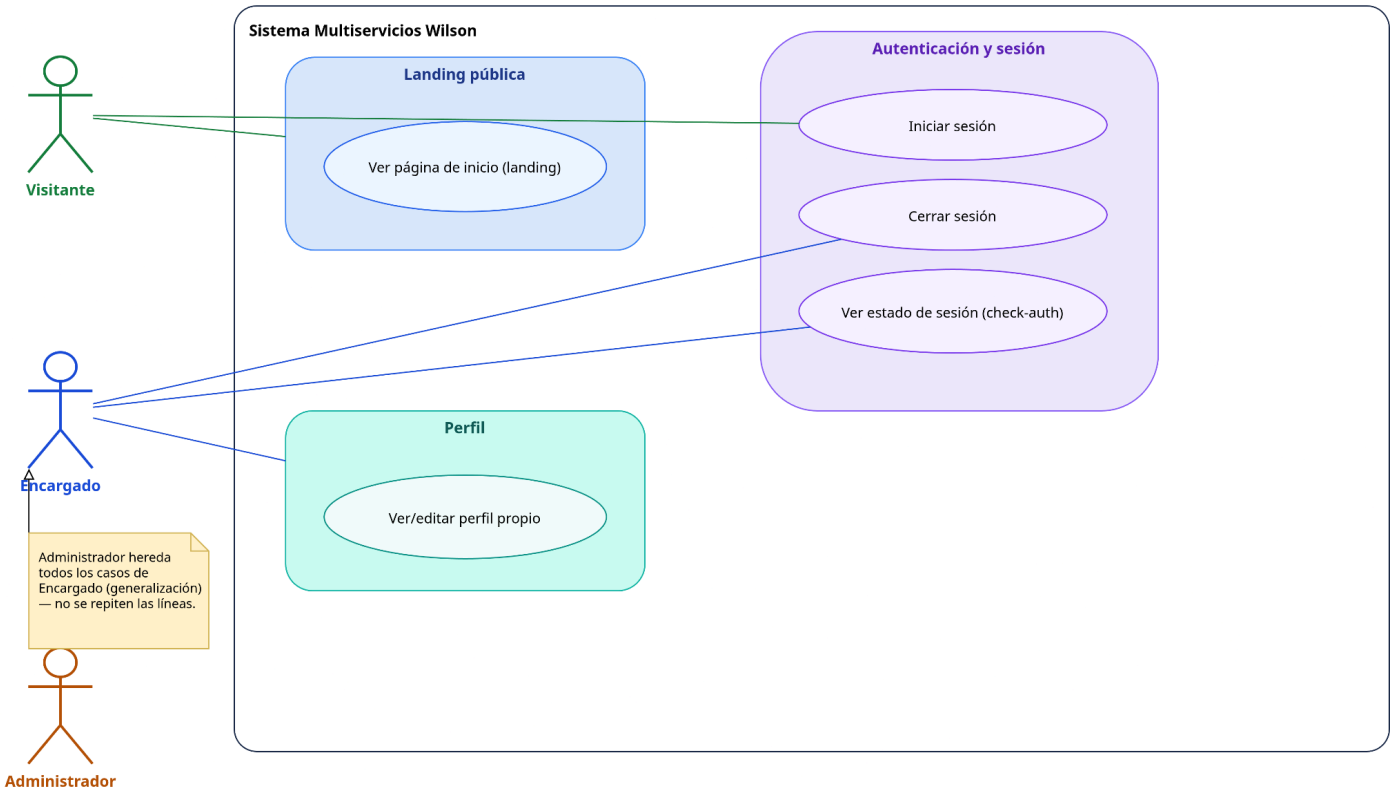
21.17. 16 - Proceso Completo



22. Casos de Uso

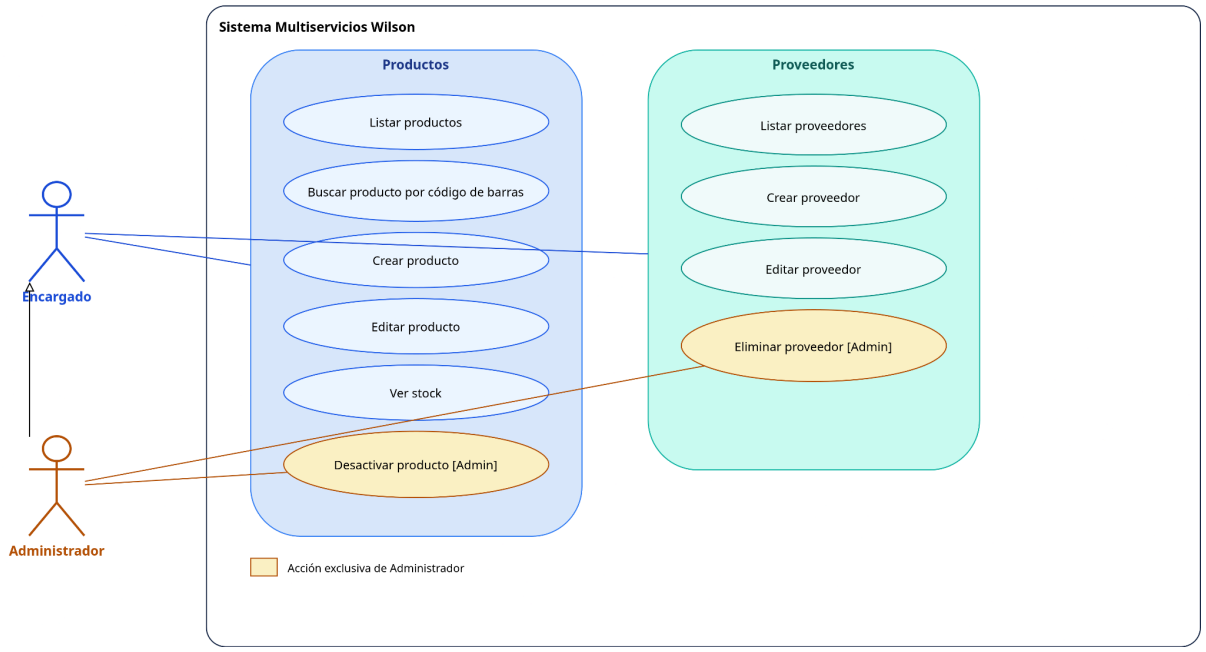
22.1. 01 - Acceso Público y Autenticación

Casos de Uso — Acceso público y Autenticación



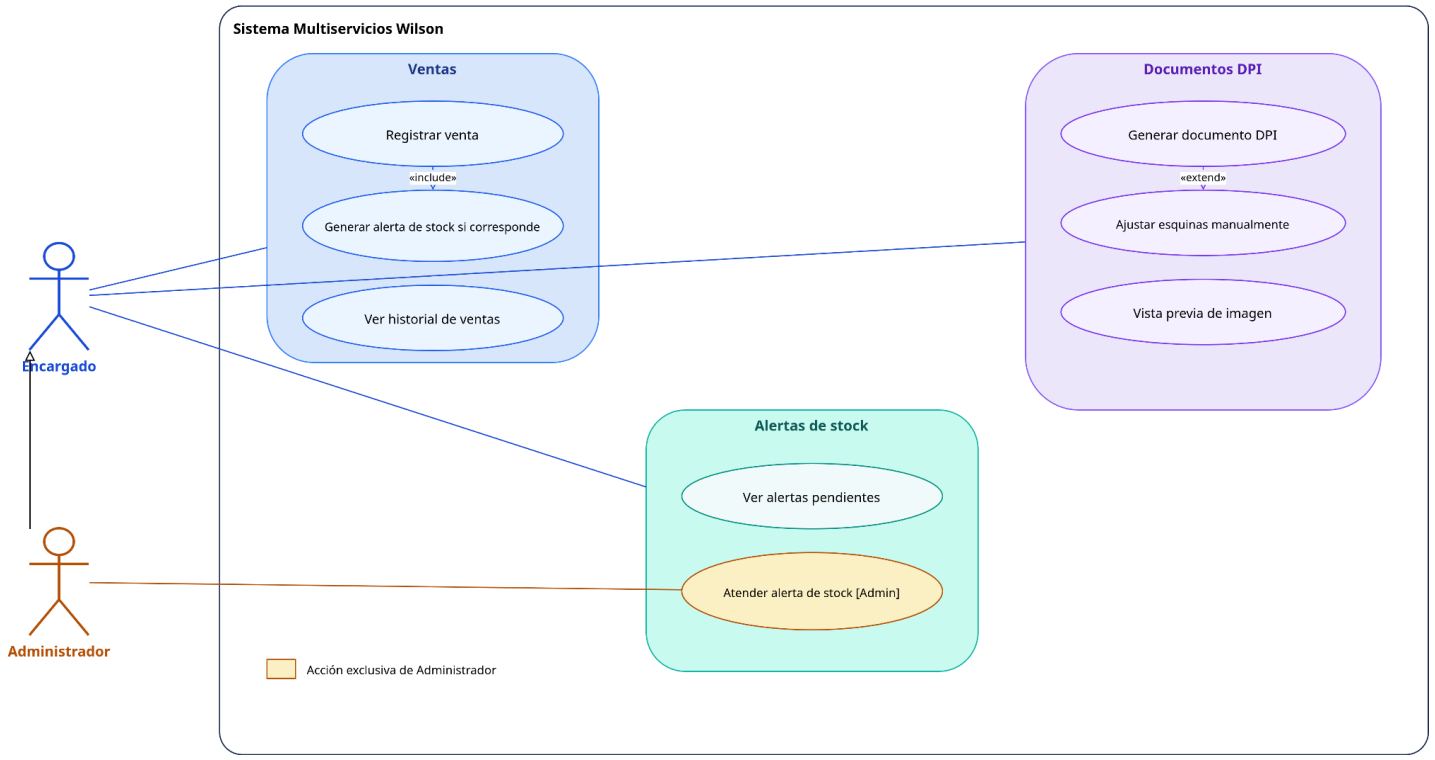
22.2. 02 - Gestión de Inventario

Casos de Uso — Inventario (Productos y Proveedores)



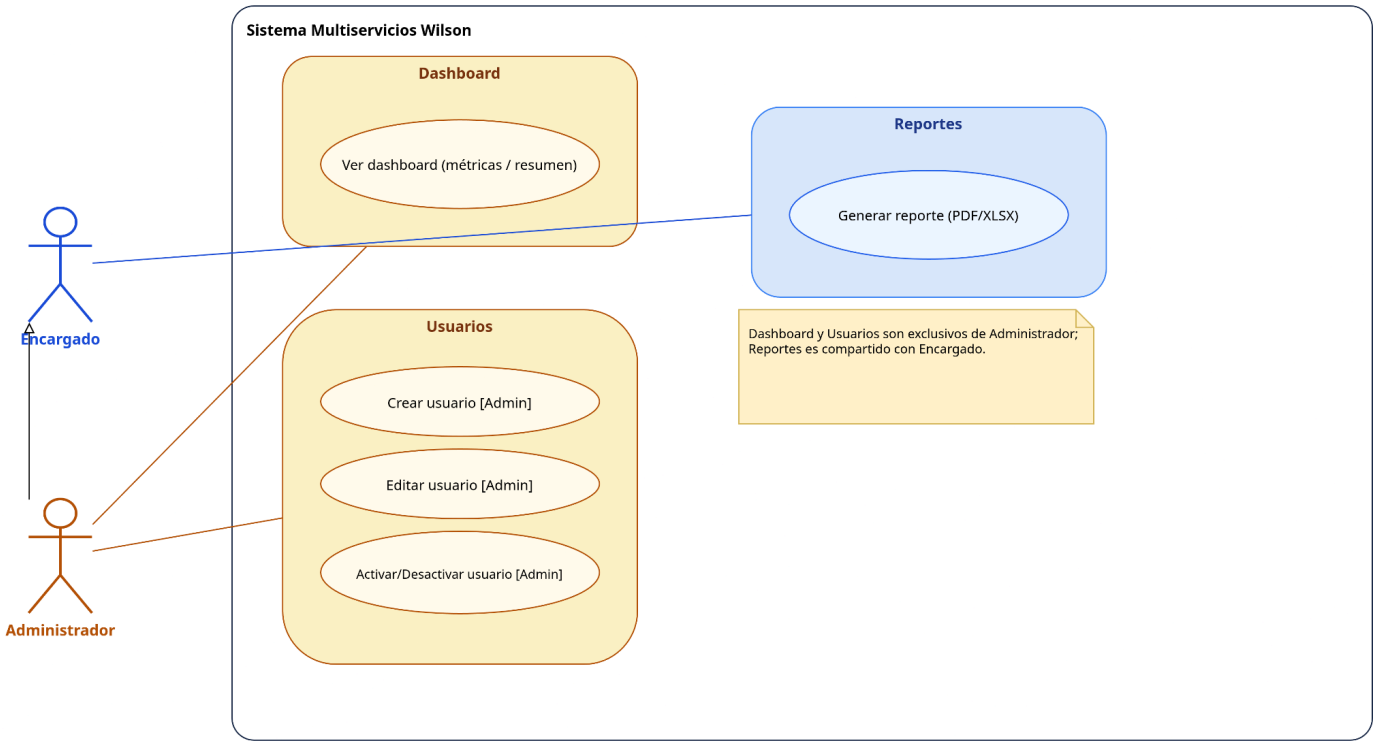
22.3. 03 - Operaciones Diarias

Casos de Uso — Operación diaria (Ventas, Alertas, Documentos DPI)



22.4. 04 - Administración

Casos de Uso — Administración (Dashboard, Reportes, Usuarios)



23. FASE 3

23.1. Despliegue, seguridad, automatización y cierre

Esta tercera fase corresponde a la puesta en producción del sistema, al endurecimiento de su seguridad y a la entrega final del portafolio técnico. Con ella el documento deja de describir únicamente lo que se analizó y se construyó (Fases 1 y 2) y pasa a explicar cómo está hecho, cómo se opera, cómo se recupera ante fallas y cómo puede crecer, de modo que cualquier persona con conocimientos de Java, React y PostgreSQL pueda mantener el sistema sin depender de quien lo desarrolló.

Todo lo que se afirma en las secciones 23 a 29 se verificó contra el código fuente de los repositorios `InventoryFW-backend` e `InventoryFW-frontend`, contra sus flujos de GitHub Actions y contra las migraciones Flyway. Cuando algo es una propuesta y todavía no existe en el sistema, el texto lo indica con la palabra «propuesto» y, en el caso de los scripts SQL, se validó su ejecución sobre una base PostgreSQL real con datos de ejemplo.

Entregables de la fase y dónde se documentan

Entregable (enunciado, sección 8.1)	Sección de este documento	Estado
Manual técnico integral: arquitectura detallada, modelado de datos y trazabilidad	24	Completo
Guía de despliegue y Runbook: despliegue, rollback, monitoreo y resolución de incidentes	25	Completo
Documentación para escalabilidad futura	26	Completo
Procesos identificados y diagramados (BPMN)	27	9 procesos vigentes y 3 propuestos
Migraciones necesarias para la evolución del modelo de datos	28	11 aplicadas y 7 propuestas, validadas en PostgreSQL
Tecnologías nuevas por utilizar, con su justificación	29	Completo
Finalización de reportes en PDF y Excel	24.3.7 y BPMN-07	En producción
Autenticación avanzada: 2FA por correo, recuperación de contraseña y gestión de secretos	24.3.5 y BPMN-05 y 06	En producción
Manual de usuario	Enlaces de interés (sección 31)	Documento externo
Código fuente, repositorios e infraestructura	Repositorios GitHub; `docker-compose.yml`, `Dockerfile` y workflows versionados	Entregado; Terraform propuesto en la sección 29
Sistema funcional en la nube con URL pública	Sección 25.2	En producción (Netlify y Railway)

23.2. Resumen técnico del sistema al cierre

Aspecto	Valor al cierre de la Fase 3
Backend	Spring Boot 3.5.7 sobre Java 25; API REST sin estado con 10 controladores y 9 entidades; autenticación JWT en cookie HttpOnly
Frontend	React 18.2 con Vite 4.5, Tailwind CSS 3.3 y Material Tailwind 2.1 (plantilla de Creative Tim, licencia MIT)
Base de datos	PostgreSQL 17 administrada por Flyway: 11 migraciones aplicadas (V1 a V11), `ddl-auto=validate`
Infraestructura	Frontend en Netlify (CDN y proxy) y API más PostgreSQL en Railway; correo transaccional con la API HTTP de Brevo

Aspecto	Valor al cierre de la Fase 3
Automatización	Dos pipelines de GitHub Actions (CI en cada PR y push; despliegue a producción solo desde `main`, con prueba de humo)
Calidad	180 pruebas JUnit en el backend y 71 pruebas Vitest en el frontend; bitácoras de pruebas empíricas contra PostgreSQL y navegador reales
Código y flujo	143 commits en el backend y 91 en el frontend; ramas `main`, `develop`, `feature/*`, `fix/*` e `integration/*`; Pull Requests fusionados desde ramas de funcionalidad
Roles	Administrador y Encargado; no existe registro público de cuentas
Procesos automatizados	Inventario y reposición (Proceso 1), fotocopia de DPI (Proceso 2), fotos de cédula (Proceso 3) y de pasaporte (Proceso 4)

23.3. Decisiones consolidadas respecto a las Fases 1 y 2

Varias decisiones quedaron abiertas en las fases anteriores («a definir», «a configurar»). La tabla siguiente registra cómo se resolvieron y dónde se justifican, para que la documentación sea coherente de principio a fin.

Tema	Planteamiento anterior	Decisión final	Referencia
Proveedor cloud	A definir entre AWS, Azure o GCP (sección 15)	Plataforma como servicio: Railway (API y PostgreSQL) y Netlify (SPA y CDN). La equivalencia con AWS está documentada como ruta de migración	25.2, 26.4
Base de datos	Sin definir; en desarrollo se usó `ddl-auto=update`	PostgreSQL 17 con Flyway; Hibernate solo valida el esquema	24.5, 28
Sesión (HU-03)	Sesión de 7 días	JWT en cookie HttpOnly con vigencia configurable (3600 s por defecto) y segundo factor opcional por correo	24.3.5
Notificaciones (RF-06)	Módulo interno dentro del sistema	Campana por rol dentro del sistema y, además, correo con Brevo para 2FA, recuperación, recibos y pedidos	24.3.7
Fotos de cédula y pasaporte	Procesos 3 y 4 documentados pero fuera del alcance de implementación	Implementados: hoja de papel fotográfico 4×6 in en PDF o Word	24.3.6, BPMN-04
Landing pública	Excluida del alcance (sección 14)	Se incluyó una página informativa sin datos sensibles; el sistema sigue siendo de uso interno (B2E)	HU-01, HU-02
CI/CD	A configurar con pipelines	GitHub Actions: CI en PR y push; despliegue automático solo desde `main` con prueba de humo	25.4
Secretos	«Gestión segura de credenciales»	Variables de entorno en Railway y en el environment `production` de GitHub; secretos de más de 32 bytes; códigos de un solo uso guardados como HMAC	24.3.5, 25.3
BPMS	Sistema a medida como motor de automatización	Se mantiene; los procesos vigentes se formalizan en BPMN-01 a BPMN-09	27

23.4. Actualización de requisitos

Entre el cierre de la Fase 2 y la entrega final se incorporaron funcionalidades que ya forman parte del sistema en producción. Los requisitos originales RF-01 a RF-07 y RNF-01 a RNF-05 siguen vigentes; se agregan los siguientes.

Requisitos funcionales adicionales

Código	Requisito
RF-08	El sistema debe ofrecer autenticación reforzada: verificación en dos pasos por correo (2FA), recuperación de contraseña mediante enlace de un solo uso, cambio de contraseña y de correo, y bloqueo temporal tras intentos fallidos consecutivos
RF-09	El sistema debe generar hojas de fotografías tamaño cédula (9 copias de 3.1×3.9 cm) y pasaporte (2 copias de 5×5 cm) sobre papel fotográfico 4×6 in, en PDF o Word, a partir de una sola foto
RF-10	El sistema debe permitir al Administrador pedir la reposición de un producto a su proveedor por correo desde una alerta de stock, con vista previa del mensaje y registro de la cantidad y la fecha del pedido
RF-11	El sistema debe poder enviar el recibo de una venta al correo del cliente, sin afectar la venta si el correo falla
RF-12	El sistema debe permitir buscar y registrar productos escaneando el código de barras con la cámara del dispositivo

Código	Requisito
RF-13	Los reportes de ventas e inventario deben mostrar indicadores, gráficos y la comparación contra el periodo anterior, y exportarse en PDF y Excel

Requisitos no funcionales adicionales

Código	Requisito
RNF-06	Cada despliegue a producción debe ser automático, verificarse con una prueba de humo y poder revertirse siguiendo el Runbook (sección 25)
RNF-07	Los códigos de un solo uso deben guardarse únicamente como HMAC-SHA256, vencer a los 10 minutos (30 minutos en el caso del enlace de recuperación) y bloquearse tras 5 intentos fallidos
RNF-08	Las fotografías de los clientes no deben almacenarse: se procesan en memoria y solo se devuelve el documento generado
RNF-09	Las ventas simultáneas no deben dejar el stock negativo ni duplicar alertas: el descuento se hace con bloqueo de filas y en una sola transacción
RNF-10	El sistema debe responder con códigos HTTP y mensajes claros (400, 401, 403, 404, 409, 429) y sin filtrar detalles internos en los errores 500

23.5. Historias de usuario nuevas

Continúan la numeración de la Fase 2 (HU-01 a HU-42). Todas están implementadas y en producción; la trazabilidad completa con componentes y pruebas está en la sección 24.6.

ID	Historia de usuario	Criterios de aceptación
HU-43	Como Encargado o Administrador con 2FA activo, quiero confirmar mi inicio de sesión con un código enviado a mi correo, para que mi cuenta esté protegida aunque alguien conozca mi contraseña	El código tiene 6 dígitos y vence a los 10 minutos; se permiten 5 intentos; el correo aparece enmascarado; puedo pedir un reenvío cada 60 segundos
HU-44	Como usuario, quiero activar o desactivar la verificación en dos pasos desde mi perfil, para decidir el nivel de protección de mi cuenta	Activar exige tener correo registrado y confirmar un código; desactivar exige mi contraseña actual; cambiar el correo desactiva el 2FA hasta confirmarlo de nuevo
HU-45	Como usuario que olvidó su contraseña, quiero recibir un enlace para restablecerla, para recuperar el acceso sin intervención del administrador	La respuesta es la misma exista o no el correo; el enlace vence a los 30 minutos y solo sirve una vez; la contraseña nueva exige de 8 a 72 caracteres con letra y número
HU-46	Como usuario, quiero cambiar mi contraseña y mi correo desde el perfil, para mantener mis datos al día	Ambos cambios piden la contraseña actual; no se aceptan correos ya usados por otra cuenta
HU-47	Como Administrador, quiero que el sistema bloquee temporalmente una cuenta tras varios intentos fallidos, para frenar ataques de fuerza bruta	Tras 5 fallos en 15 minutos se bloquea 15 minutos y se informa el tiempo restante; un acceso correcto reinicia el contador
HU-48	Como Encargado o Administrador, quiero enviar el recibo de la venta al correo del cliente, para entregarle un comprobante digital	El correo del cliente es opcional; la venta se registra aunque el envío falle; el sistema informa ENVIADO, NO_ENVIADO u OMITIDO; hay un tope de 20 recibos por hora por vendedor y 3 por destinatario
HU-49	Como Administrador, quiero pedir la reposición al proveedor desde la alerta de stock, para reabastecer sin salir del sistema	Elijo la cantidad y veo la vista previa exacta del correo; si el proveedor no tiene correo válido se me indica completarlo; el pedido guarda cantidad y fecha; una alerta con proveedor solo se atiende después de enviar el pedido
HU-50	Como Encargado o Administrador, quiero escanear el código de barras con la cámara del móvil, para registrar ventas y productos sin escribirlo	En ventas el escáner agrega productos a la venta de forma continua; en el alta y la edición de productos completa el código de barras; se usa la cámara del dispositivo (biblioteca ZXing)
HU-51	Como Administrador, quiero ver indicadores y gráficos con la comparación contra el periodo anterior, para entender la evolución del negocio	Rangos rápidos (hoy, 30 días, este mes); rango máximo de 366 días; ranking de empleados y de productos; detalle paginado con búsqueda; exportación a PDF y Excel
HU-52	Como Encargado o Administrador, quiero generar una hoja de fotos tamaño cédula, para entregar fotografías listas para imprimir	Una sola foto con marco de recorte ajustable; hasta 9 copias de 3.1×3.9 cm; vista previa de una copia; salida en PDF o Word
HU-53	Como Encargado o Administrador, quiero generar una hoja de fotos tamaño pasaporte, para entregar fotografías listas para imprimir	Hasta 2 copias de 5×5 cm sobre papel 4×6 in; las medidas se obtienen del servidor y no están fijas en la interfaz
HU-54	Como usuario, quiero una campana de notificaciones con eventos de mi rol, para enterarme de stock bajo, productos agotados, cambios de estado y documentos generados	Las notificaciones pueden marcarse como leídas una por una o todas juntas; cada rol solo ve las suyas y las generales

ID	Historia de usuario	Criterios de aceptación
HU-55	Como usuario, quiero un modo oscuro, para trabajar con comodidad en ambientes de poca luz	Se respeta la preferencia del sistema y la elegida queda guardada en el navegador
HU-56	Como Administrador, quiero que el sistema impida dejar al negocio sin administradores, para no perder el control de la plataforma	Nadie puede desactivarse ni quitarse el rol de administrador a sí mismo; no se puede desactivar al último administrador activo; se advierte si `admin` conserva la contraseña sembrada

23.6. Glosario complementario

Término	Definición
Runbook	Guía operativa paso a paso para desplegar, vigilar, recuperar y resolver incidentes de un sistema en producción
Rollback	Volver a la versión anterior de la aplicación (o de los datos) cuando un despliegue falla
Smoke test	Prueba de humo: verificación mínima automática tras un despliegue para confirmar que el sistema responde
CI/CD	Integración continua y entrega continua: pipelines que prueban y despliegan el código sin pasos manuales
IaC	Infraestructura como código: describir servidores, redes y servicios en archivos versionados
JWT	JSON Web Token: credencial firmada que identifica al usuario sin guardar sesión en el servidor
OTP	Contraseña de un solo uso: aquí, el código de 6 dígitos que se envía por correo
HMAC	Resumen criptográfico con clave secreta; permite guardar un código sin poder leerlo después
Flyway	Herramienta que versiona y aplica las migraciones SQL del esquema de la base de datos
Expandir y contraer	Patrón de migración en dos pasos: primero se agrega lo nuevo (compatible con la versión anterior) y después se retira lo viejo
PITR	Recuperación a un punto en el tiempo, usando un respaldo completo más el registro de transacciones
RPO y RTO	Pérdida máxima de datos tolerable y tiempo máximo de recuperación tras un desastre
Healthcheck	Consulta periódica (`/actuator/health`) con la que la plataforma decide si una instancia puede recibir tráfico
GitFlow	Modelo de ramas con `main`, `develop`, `feature/*` y `hotfix/*`
SLI, SLO	Indicador de nivel de servicio (lo que se mide) y objetivo de nivel de servicio (la meta)
Kardex	Registro cronológico de entradas, salidas y ajustes de cada producto del inventario

24. Manual Técnico Integral

24.1. Introducción, audiencia y convenciones

Este manual describe el sistema de Multiservicios Wilson tal como está construido y desplegado al cierre de la Fase 3: su arquitectura, sus componentes, su modelo de datos, las reglas de negocio que implementa, los controles de seguridad, el catálogo de la API y la trazabilidad entre lo que se pidió y lo que se entregó. Complementa a la Guía de despliegue y Runbook (sección 25), que explica cómo operarlo.

Si usted es…	Empiece por…	Después consulte…
Desarrollador nuevo en el proyecto	24.2 y 24.3 (arquitectura y paquetes)	24.4 (API), 24.5 (datos) y 28 (migraciones)
Operador o administrador técnico	25 (Runbook)	24.3.9 (variables) y 26 (límites de capacidad)
Auditor, docente o evaluador	24.6 (trazabilidad) y 24.7 (calidad)	23 (alcance) y 27 (procesos BPMN)

Convenciones: los nombres de clases, rutas, tablas y variables aparecen en `tipografía monoespaciada`; los códigos RF, RNF y HU provienen de las Fases 1 y 2 y de la sección 23; «Encargado» es el rol `ROLE_EMPLOYEE` y «Administrador» es `ROLE_ADMIN`.

24.2. Visión general de la arquitectura

El sistema es una aplicación web de uso interno (B2E) organizada en tres capas físicas: una aplicación de página única (SPA) servida desde una CDN, una API REST sin estado y una base de datos relacional. No se eligió una arquitectura de microservicios: el negocio es una sola sucursal con un único encargado en mostrador, el equipo es de una persona y los procesos automatizados comparten datos transaccionales (una venta descuenta stock y crea alertas en la misma transacción). Un monolito modular, con paquetes de responsabilidad clara y un motor de imágenes aislado, ofrece la menor complejidad operativa y deja la puerta abierta a separar componentes cuando una métrica lo justifique (sección 26).

Principios de diseño

- **Una sola fuente de verdad para el esquema:** Flyway crea y modifica las tablas; Hibernate solo valida (`ddl-auto=validate`) y nunca genera DDL.
- **La regla de negocio vive en el servidor:** la interfaz valida para ayudar al usuario, pero el servidor vuelve a validar; por ejemplo, el precio de una venta siempre es el del catálogo.
- **Sin estado en el servidor de aplicación:** la identidad viaja en un JWT dentro de una cookie HttpOnly; las únicas excepciones son dos contadores en memoria (intentos de inicio de sesión y recibos por hora), documentados como límite de escalabilidad en la sección 26.
- **Mínimo privilegio y denegación por defecto:** toda ruta exige sesión salvo una lista corta y explícita; las acciones destructivas exigen el rol Administrador en el servidor, no solo en la interfaz.
- **Fallar de forma clara:** los errores esperados devuelven 400, 401, 403, 404, 409 o 429 con un mensaje legible; los inesperados devuelven 500 genérico y el detalle va al registro.
- **Configuración por entorno:** ningún secreto está en el repositorio; todo se inyecta por variables de entorno.
- **Procesamiento de imágenes en memoria:** las fotos de los clientes no se guardan (RNF-08).

Pila tecnológica

Capa	Tecnología	Versión	Uso en el sistema
Lenguaje y plataforma	Java (Temurin)	25	Backend; imagen Docker de compilación y de ejecución
Framework backend	Spring Boot (Web, Security, Data JPA, Validation, Mail, Actuator)	3.5.7	API REST, seguridad, persistencia y healthcheck
Base de datos	PostgreSQL y driver JDBC	17	Datos transaccionales; extensión `pgcrypto`
Migraciones	Flyway (`flyway-core` y `flyway-database-postgresql`)	Gestionada por Spring Boot	Versionado del esquema V1 a V11

Capa	Tecnología	Versión	Uso en el sistema
Sesión	JJWT (`jjwt-impl`, `jjwt-jackson`)	0.11.5	Firma HS256 de JWT
Imágenes	BoofCV (`boofcv-core`, `boofcv-io`) y `metadata-extractor`	1.4.0 y 2.21.0	Detección de bordes de la tarjeta y orientación EXIF
Documentos	Apache PDFBox y Apache POI (`poi-ooxml`)	3.0.3 y 5.3.0	PDF y Word de documentos y reportes; Excel de reportes
Frontend	React, React Router, Vite	18.2, 6.17 y 4.5	SPA con rutas protegidas por rol
Interfaz	Tailwind CSS, Material Tailwind, Heroicons, ApexCharts	3.3, 2.1, 2.0 y 3.44	Diseño, componentes y gráficos
Comunicación	axios	1.13	Cliente HTTP con cookie (`withCredentials`)
Escáner	`@zxing/browser` y `@zxing/library`	0.1.5 y 0.21.3	Lectura de códigos de barras con la cámara
Pruebas	JUnit 5 y Spring Test; Vitest	Boot 3.5; 0.34	180 y 71 pruebas
Contenedores	Docker y Docker Compose; nginx	Alpine	Entorno local completo y servidor del frontend en contenedor
Entrega	GitHub Actions, Railway CLI y Netlify CLI	Acciones v6 a v8	CI/CD
Hospedaje	Railway (API y PostgreSQL) y Netlify (SPA y CDN)	Plataforma	Producción
Correo	Brevo (API HTTP) y Mailtrap (desarrollo)	v3	2FA, recuperación, recibos y pedidos

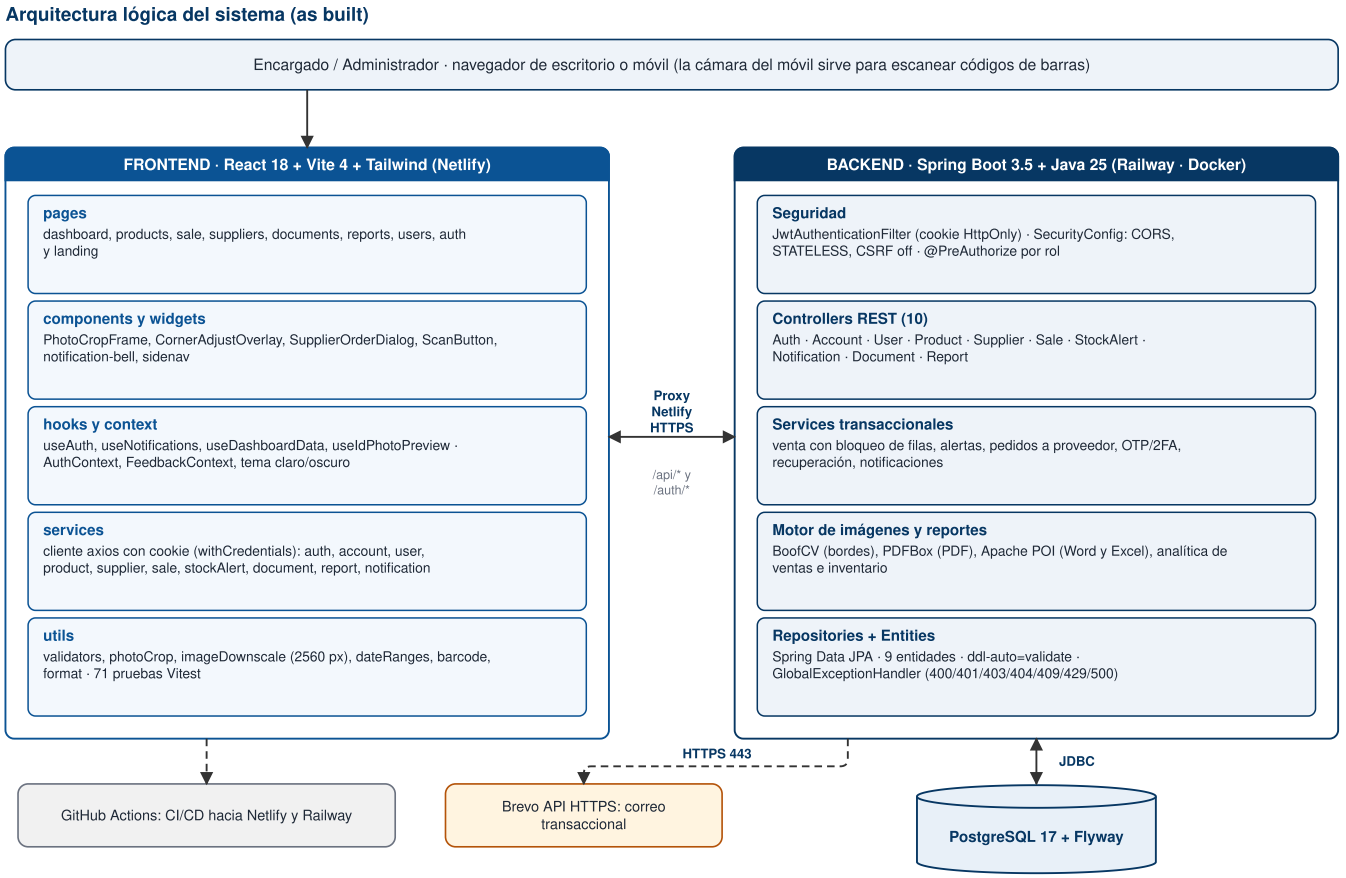


Figura 24.1 — Arquitectura lógica del sistema: SPA, API por capas, base de datos y servicios externos

24.3. Arquitectura detallada

24.3.1. Backend: capas y paquetes

El backend sigue la estructura clásica de Spring: **controladores → servicios → repositorios → entidades**, con paquetes transversales para seguridad, configuración, imágenes y reportes. Las dependencias solo van hacia abajo: un controlador nunca toca un repositorio y una entidad nunca conoce a un servicio.

Paquete ([redacted])	Responsabilidad	Clases principales
`config`	Seguridad HTTP y CORS, propiedades de negocio, reloj inyectable, manejo global de errores y verificación de la contraseña sembrada	`SecurityConfig`, `InventoryProperties`, `ClockConfig`, `GlobalExceptionHandler`, `DefaultAdminPasswordCheck`
`controllers`	10 controladores REST: traducen HTTP a llamadas de servicio y aplican `@PreAuthorize`	`AuthController`, `AccountController`, `UserController`, `ProductController`, `SupplierController`, `SaleController`, `StockAlertController`, `NotificationController`, `DocumentController`, `ReportController`
`dtos`	Objetos de petición y respuesta con validaciones de Jakarta Validation	`SaleRequestDto`, `ProductRequestDto`, `SupplierRequestDto`, `LoginUserDto`, `ResetPasswordDto`, `SaleResponseDto` y otros
`entities`	9 entidades JPA mapeadas a las tablas del esquema	`User`, `Role`, `Product`, `Supplier`, `Sale`, `SaleDetail`, `StockAlert`, `Notification`, `VerificationCode`
`enums`	Dominios cerrados	`RoleList`, `ProductCategory` (CELULAR, ACCESORIO), `StockAlertStatus`, `NotificationType`, `VerificationType`
`jwt`	Emisión, firma y validación del token; filtro por petición; respuesta 401	`JwtUtil`, `JwtAuthenticationFilter`, `JwtEntryPoint`
`services`	Lógica de negocio transaccional, autenticación, correo y notificaciones	`SaleServiceImpl`, `StockAlertServiceImpl`, `StockStatusCalculator`, `SupplierOrderService`, `SaleReceiptService`, `AuthService`, `LoginAttemptService`, `VerificationCodeService`, `PasswordRecoveryServiceImpl`, `AccountServiceImpl`, `MailServiceImpl`, `BrevoApiClient`, `NotificationServiceImpl`, `DpiDocumentServiceImpl`, `IdPhotoDocumentServiceImpl`, `UserManagementServiceImpl`
`repositories`	Acceso a datos con Spring Data JPA; consultas con bloqueo y con carga anticipada	`ProductRepository.findByIdForUpdate` (bloqueo pesimista), `SaleRepository.findForReport` (join fetch), `NotificationRepository.findVisibleTo`
`imaging`	Motor de imágenes: orientación, detección de bordes, recorte, mejora y composición de documentos	`CardDetectionServiceImpl`, `ImageCropServiceImpl`, `ImageEnhancementServiceImpl`, `ExifOrientationNormalizerImpl`, `DocumentLayoutRegistry`, compositores `Pdf*` y `Word*`
`reports`	Analítica de ventas e inventario y exportación a PDF y Excel	`SalesAnalyticsServiceImpl`, `InventoryAnalyticsServiceImpl`, `PdfReportWriter`, `XlsxReportWriter`, `ReportWriterFactory`
`validation`	Regla única de contraseña	`PasswordRules` (8 a 72 caracteres, con letra y número)
`resources`	Configuración, migraciones y plantillas de correo	`application.properties`, `application-prod.properties`, `db/migration/V1…V11`, `mail/*.html`

24.3.2. Frontend: estructura

Carpeta (`src/`)	Contenido
`pages`	Pantallas por módulo: `auth` (inicio de sesión, 2FA, recuperación), `dashboard` (inicio, perfil y seguridad, productos, ventas, alertas, proveedores, documentos DPI y fotos, reportes, usuarios) y `landing` (página informativa)
`components`	Piezas reutilizables: rutas protegidas (`PrivateRoute`, `RoleRoute`, `GuestRoute`), edición de recortes (`PhotoCropFrame`, `CornerAdjustOverlay`, `ImageAdjustmentSliders`), `SupplierOrderDialog`, `GeneratedDocumentCard`, `ScanButton` y `BarcodeScannerDialog`
`layouts` y `widgets`	Estructura de las vistas (autenticación y panel), barra lateral, barra superior, campana de notificaciones, configurador de tema, tarjetas y gráficos
`context`	`AuthContext` (sesión y rol), `FeedbackContext` (avisos) y tema claro u oscuro con preferencia guardada
`hooks`	`useAuth`, `useNotifications`, `useProducts`, `useDashboardData`, `useIdPhotoSpecs`, `useIdPhotoPreview`, `useCornerAdjustment`
`services`	Un servicio por módulo sobre una instancia de axios con cookie; un 401 fuera del flujo de autenticación cierra la sesión local
`utils`	Validadores, formato, rangos de fechas, recorte de fotos, reducción a 2560 px, códigos de barras, descarga de archivos; con 71 pruebas
`auth/menuByRole.js` y `routes.jsx`	Rutas y menú filtrados por rol: Dashboard y Usuarios solo para el Administrador
`public`	`_redirects` (proxy hacia Railway y SPA) y `_headers` (caché y cabeceras de seguridad)

24.3.3. Flujo de una petición de punta a punta

1. El navegador carga la SPA desde Netlify y llama a rutas relativas (`/api/...`, `/auth/...`) porque `VITE_API_URL=/`.
2. Netlify aplica `public/_redirects`: las rutas `/api/*` y los endpoints de `/auth/` listados se reenvían con estado 200 al dominio del servicio en Railway; cualquier otra ruta devuelve `index.html` para React Router. Así la cookie `jwt` es de primera parte y el navegador no necesita CORS.
3. En Railway, el filtro `JwtAuthenticationFilter` lee la cookie, valida el token y carga al usuario; si falta o es inválido, la petición sigue como anónima y `JwtEntryPoint` responde 401 cuando la ruta lo exige.
4. `SecurityConfig` decide el acceso y `@PreAuthorize` refina por rol; después actúa el controlador, el servicio transaccional y el repositorio contra PostgreSQL.
5. Cualquier excepción pasa por `GlobalExceptionHandler`, que la convierte en un `ApiMessage` con el código HTTP correcto.

> **Importante:** el proxy de Netlify corta cada petición a los 26 segundos, contando la subida del archivo. Por eso las fotos se reducen a 2560 px en el navegador antes de enviarse y el backend aplica el mismo límite como defensa.

24.3.4. Reglas de negocio y transaccionalidad

El corazón del sistema es el Proceso 1 (inventario y reposición automática). `SaleServiceImpl.registerSale` ejecuta todo en una sola transacción: bloquea las filas de los productos involucrados en orden de identificador (evita interbloqueos entre ventas simultáneas), valida, descuenta, guarda la venta y sus líneas y, si corresponde, crea la alerta y la notificación. Si cualquier línea falla, nada se descuenta.

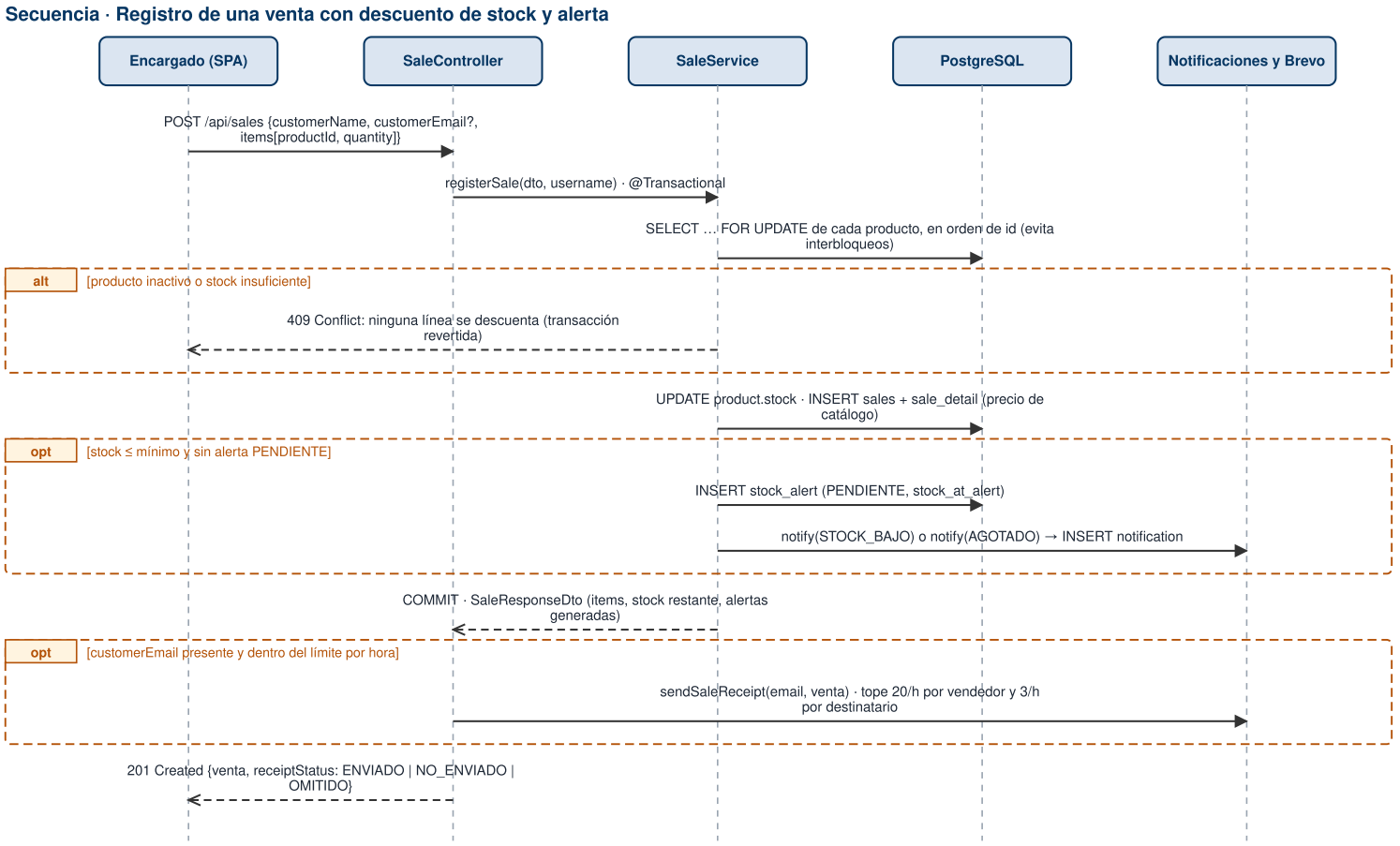


Figura 24.2 — Secuencia del registro de una venta con descuento de stock, alerta y recibo

ID	Regla de negocio implementada
RN-01	Cada producto tiene su propio stock mínimo (`min_stock`); al crearlo sin valor se usa `inventory.stock-minimo` (2 por defecto). Un producto está en STOCK_BAJO cuando `stock ≤ min_stock` y AGOTADO cuando `stock ≤ 0`
RN-02	Tras una venta, si el stock queda en el mínimo o por debajo y no existe una alerta PENDIENTE para ese producto, se crea una (no se duplican alertas)
RN-03	Una venta es atómica: si algún artículo es inactivo, no existe o supera el stock, la venta completa se rechaza con 409 y no se descuenta nada
RN-04	El precio de cada línea es el del catálogo; el precio que envíe el cliente HTTP se ignora
RN-05	Un producto con historial no se borra: se da de baja (`active=false`) y se notifica; un producto inactivo no se puede vender
RN-06	Atender una alerta no cambia el stock. Si el producto tiene proveedor, la alerta solo se puede atender después de enviar el pedido; sin proveedor se atiende directamente
RN-07	Un pedido de reposición exige un proveedor con correo válido (el valor provisional `.invalid` de la migración V10 se rechaza) y una cantidad de al menos 1; si el correo falla no se guarda nada
RN-08	El recibo por correo nunca bloquea la venta; hay un tope de 20 por hora por vendedor y 3 por hora por destinatario
RN-09	Se notifica STOCK_BAJO cuando se crea la alerta con stock mayor que 0, y AGOTADO cuando el stock llega a 0
RN-10	Un usuario no puede desactivarse ni quitarse el rol de administrador a sí mismo, ni se puede desactivar al último administrador activo; una cuenta creada con correo nace con 2FA activo
RN-11	Un proveedor con productos asociados no se puede eliminar (409)
RN-12	Los reportes usan los últimos 30 días por defecto y rechazan rangos invertidos o de más de 366 días
RN-13	Las fotos del DPI solo pueden ser JPEG o PNG, de máximo 10 MB cada una y 20 MB por solicitud; ambas caras son obligatorias
RN-14	En las hojas de cédula y pasaporte las copias deben estar entre 1 y la capacidad de la hoja

24.3.5. Seguridad

Autenticación. `POST /auth/login` valida usuario y contraseña con BCrypt. Si la cuenta no tiene 2FA, el servidor emite un JWT HS256 (sujeto, emisión y expiración) y lo entrega en la cookie `jwt` con los atributos `HttpOnly`, `Secure` (por defecto) y `SameSite=Lax` (configurable); el frontend nunca ve ni guarda el token. La vigencia por defecto es de 3600 segundos y se cambia con `JWT_EXPIRATION`. El secreto debe tener al menos 32 bytes: si no, la aplicación no arranca. Una cuenta desactivada no puede autenticarse, y el mensaje de error es siempre el mismo («Usuario o contraseña incorrectos») para no revelar si el usuario existe.

Verificación en dos pasos (2FA). Con 2FA activa, el primer paso devuelve un token de reto de 10 minutos (con la marca `purpose=2fa`, que el filtro rechaza como sesión) y el correo enmascarado. El código de 6 dígitos se genera con `SecureRandom`, se guarda solo como HMAC-SHA256 con el secreto del servidor como clave, vence a los 10 minutos, permite 5 intentos y se invalida al emitir otro. Hay una espera de 60 segundos entre envíos. El mismo mecanismo confirma la activación del 2FA y protege la recuperación de contraseña, donde el enlace contiene un token aleatorio de 256 bits, de un solo uso y con vigencia de 30 minutos.

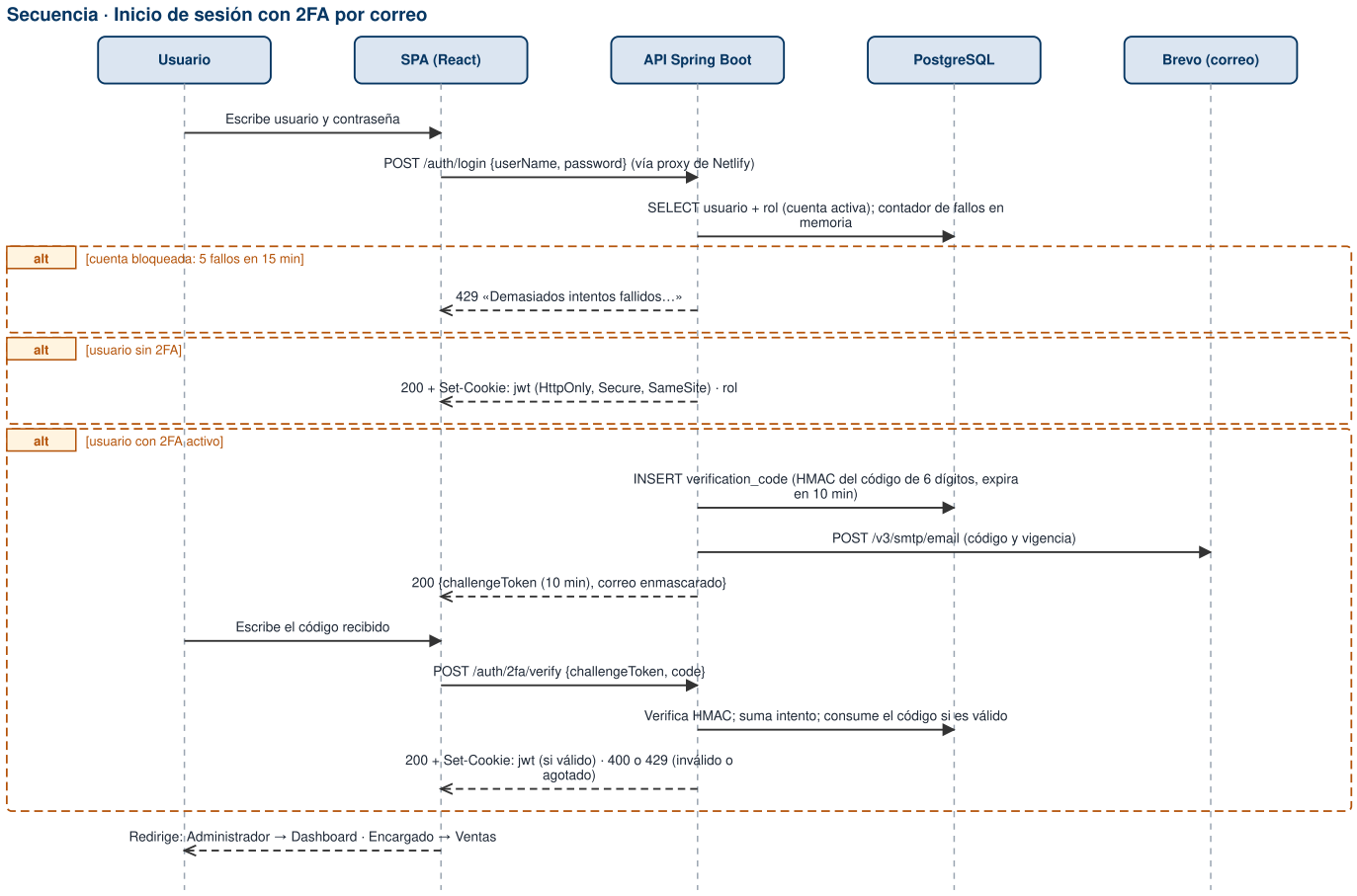


Figura 24.3 — Secuencia del inicio de sesión con 2FA por correo

Autorización. `SecurityConfig` deniega por defecto: son públicos solo `/auth/login`, `/auth/logout`, `/auth/2fa/**`, `/auth/forgot-password`, `/auth/reset-password` y `/actuator/health`. El resto exige sesión. Los métodos sensibles llevan `@PreAuthorize`.

Función	Administrador	Encargado
Ventas, productos (alta y edición), proveedores (alta y edición), reportes, documentos DPI y fotos, notificaciones, perfil	Sí	Sí
Dashboard e inicio	Sí	No
Dar de baja o reactivar productos; eliminar proveedores	Sí	No
Atender alertas y pedir reposición al proveedor	Sí	No (puede ver las alertas)
Gestión de usuarios (crear, editar, activar y desactivar)	Sí	No

Protecciones adicionales.

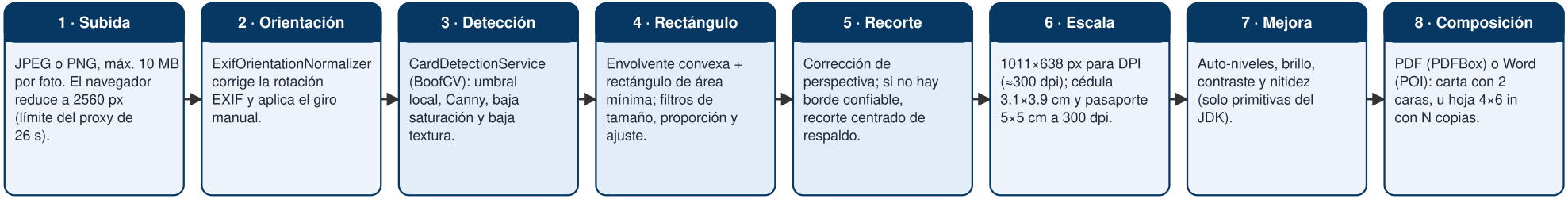
- Bloqueo temporal de la cuenta tras 5 contraseñas incorrectas en 15 minutos (15 minutos de bloqueo; responde 429 con el tiempo restante). Se aplica por cuenta, no por IP, para que un atacante detrás de varios proxys no lo evada.
- CSRF deshabilitado de forma deliberada: la API es sin estado, la cookie usa `SameSite` y CORS solo admite los orígenes de `CORS_ALLOWED_ORIGINS` con credenciales.
- Respuestas uniformes en recuperación de contraseña (no permite enumerar cuentas) y mensajes genéricos en errores 500.
- Cabeceras de `public/_headers`: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin` y caché inmutable para `/assets/*`.
- Al arrancar, `DefaultAdminPasswordCheck` avisa si `admin` conserva la contraseña sembrada por la migración V2 (pública en el repositorio) y, con `SECURITY_FAIL_ON_DEFAULT_ADMIN_PASSWORD=true`, impide el arranque. En cada base nueva debe crearse otro Administrador y desactivarse `admin`.
- Actuator expone únicamente `/actuator/health`, sin detalles; el indicador de correo está desactivado para que un fallo de Brevo no tumbe el despliegue.

> **Riesgo:** el cierre de sesión borra la cookie pero no invalida el JWT ya emitido, que seguiría siendo válido hasta su expiración si alguien lo hubiera copiado. La mitigación propuesta (columna `token_version`) está en la migración V13 de la sección 28.

24.3.6. Motor de imágenes y documentos

Los procesos 2, 3 y 4 comparten un motor parametrizado por tipo de documento: `DocumentLayoutRegistry` es el único lugar donde viven los tamaños de hoja, márgenes, medidas de foto y resolución. Agregar un documento nuevo consiste en registrar una especificación y, si hace falta, un compositor.

Pipeline del motor de imágenes (paquete imaging)



Todo se procesa en memoria: las fotos del cliente no se guardan en disco ni en la base de datos (RNF-08). Cada generación crea una notificación DOCUMENTO_GENERADO.

Figura 24.4 — Pipeline del motor de imágenes

Elemento	Especificación vigente
Fotocopia de DPI	Hoja carta (612×792 pt), márgenes de 0.5 in; cada cara se recorta a 1011×638 px (≈ 300 dpi) y se apila: anverso arriba y reverso abajo
Hoja de fotos	Papel fotográfico 4×6 in (288×432 pt), margen de 3 mm y 2 pt entre copias; cédula 3.1×3.9 cm (9 copias) y pasaporte 5×5 cm (2 copias) a 300 dpi
Detección de bordes	BoofCV 100 % Java: umbral de brillo local en ambas polaridades, bordes Canny, baja saturación y baja textura; el mejor contorno se reduce a envolvente convexa y rectángulo de área mínima y se filtra por tamaño (15 % a 98 % de la imagen), proporción y ajuste a los lados
Respaldo	Si no hay un contorno confiable, recorte centrado con la proporción de la tarjeta; el usuario puede ajustar las 4 esquinas a mano
Mejora	Auto-niveles, brillo, contraste y nitidez (por defecto 0, 0 y 15) con primitivas del JDK
Salida	PDF con PDFBox (borde gris fino como guía de corte en las hojas de fotos) o Word con Apache POI (tabla de celdas de tamaño fijo)
Memoria	Las fotos se reducen a 2560 px; con 1 vCPU se miden unos 230 MiB en reposo y unos 720 MiB de pico con dos fotocopias simultáneas de fotos de 12 MP; JVM con `-Xmx512m` y `SerialGC`

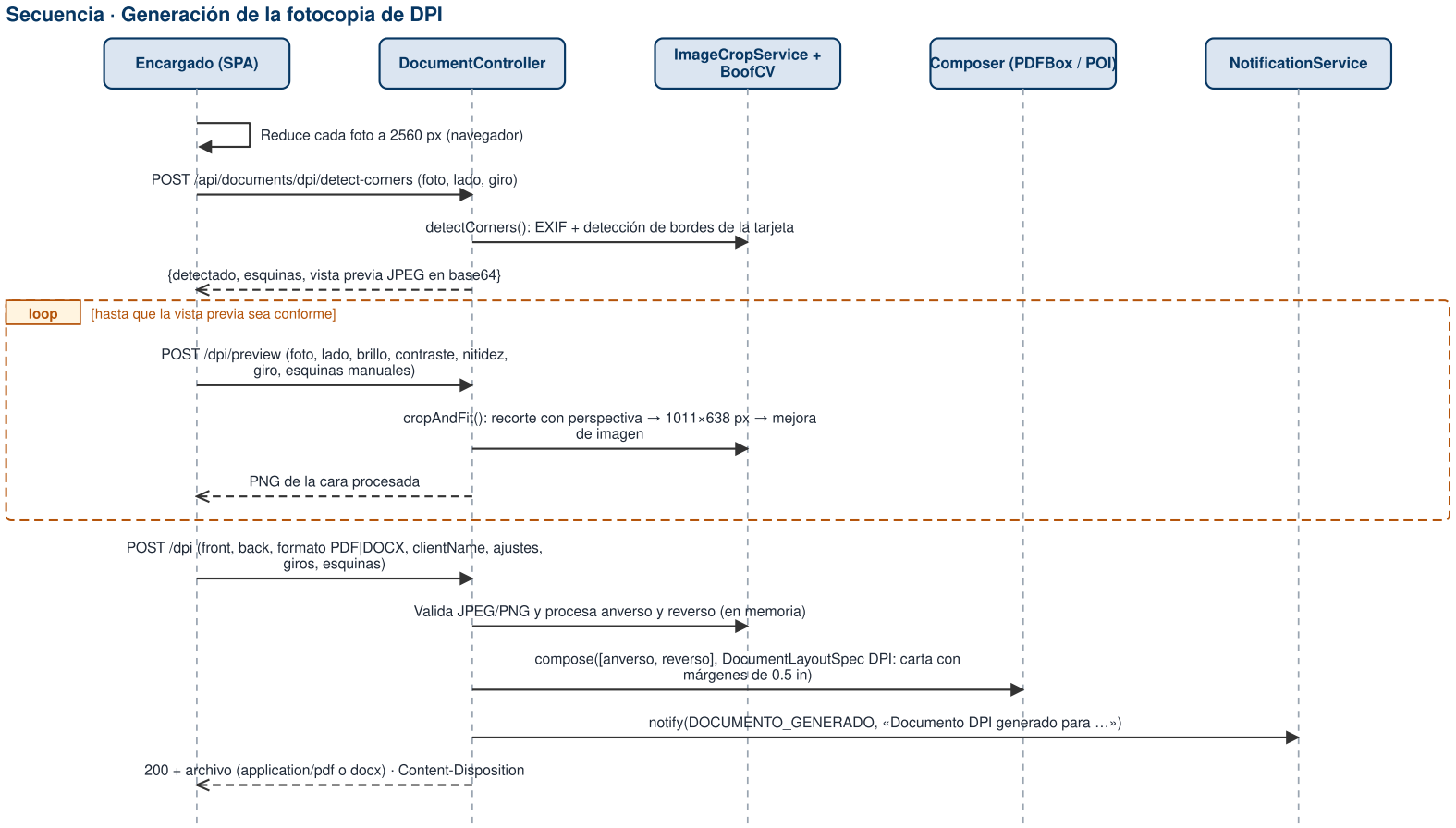


Figura 24.5 — Secuencia de la generación de la fotocopia de DPI

> **Nota:** el Word se verificó por su estructura XML y no con Microsoft Word. Antes de depender de un `.docx` generado conviene abrir uno real. Las hojas deben imprimirse al 100 % en papel 4×6 in, sin «ajustar a la página», para que la foto mida lo indicado.

24.3.7. Correo, notificaciones y reportes

Correo. `MailServiceImpl` renderiza plantillas HTML (`layout`, `code`, `reset`, `receipt`, `supplier-order`) y envía por la API HTTP de Brevo si `BREVO_API_KEY` tiene valor (Railway bloquea SMTP); si no, usa SMTP con las variables `MAIL_*` (Mailtrap en desarrollo). El cliente HTTP tiene un tiempo máximo de 10 segundos. Se envían cinco tipos de mensaje: código de inicio de sesión, código de activación del 2FA, enlace de recuperación, recibo de venta y pedido de reposición.

Notificaciones. La entidad `Notification` guarda el tipo (STOCK_BAJO, DOCUMENTO_GENERADO, AGOTADO, PRODUCTO_ESTADO), el mensaje, la entidad relacionada y el rol destinatario (nulo significa que la ven todos los roles). Se generan al crear alertas, al agotar o cambiar de estado un producto y al generar un documento. La campana del frontend las consulta por rol y permite marcarlas como leídas.

Reportes. `ReportController` ofrece resúmenes para la pantalla y archivos descargables. Los resúmenes de ventas comparan el periodo con el inmediatamente anterior de la misma duración; las exportaciones se generan con PDFBox (con gráficos) y Apache POI (Excel). Para no disparar consultas adicionales, `findForReport` trae ventas, empleado, detalles y productos con un solo `join fetch`.

24.3.8. Manejo de errores y validaciones

Código	Cuándo ocurre	Ejemplos reales
400	Datos inválidos	Validación de DTO (nombre vacío, stock negativo); imagen que no es JPEG o PNG; archivo mayor al límite; falta el campo `front` o `back`; código OTP incorrecto o vencido
401	Sin sesión o credenciales inválidas	Cookie ausente o vencida; usuario o contraseña incorrectos; reto 2FA vencido
403	Rol insuficiente	Un Encargado intenta crear un usuario o atender una alerta (`AccessDeniedException`)
404	No existe el recurso	Producto, proveedor o alerta con un identificador inexistente
409	Conflicto con los datos	Stock insuficiente; código de barras duplicado; proveedor con productos; pedido sin correo de proveedor; violación de integridad de la base
429	Demasiados intentos	Cuenta bloqueada por fallos de contraseña; reenvío de código antes de 60 segundos; 5 intentos de OTP agotados
500	Error no previsto	Mensaje genérico «Error interno del servidor»; el detalle queda en el registro

24.3.9. Configuración y variables de entorno

Variable	Valor por defecto	Descripción
`DB_URL`, `DB_USER`, `DB_PASSWORD`	URL de Docker Compose; `postgres`; obligatoria	Conexión JDBC a PostgreSQL
`JWT_SECRET`	Obligatoria (≥ 32 bytes)	Firma del JWT y clave del HMAC de los códigos; generar con `openssl rand -base64 48`
`JWT_EXPIRATION`	`3600`	Vigencia de la sesión en segundos (también `Max-Age` de la cookie)
`CORS_ALLOWED_ORIGINS`	Orígenes locales	Orígenes del frontend separados por coma; admite comodines
`FRONTEND_URL`	`http://localhost:5173`	Base de los enlaces de los correos
`COOKIE_SECURE`, `COOKIE_SAME_SITE`	`true`, `Lax`	Atributos de la cookie de sesión
`BREVO_API_KEY`, `MAIL_FROM`, `MAIL_FROM_NAME`, `MAIL_REPLY_TO`	vacías	Envío por la API HTTP de Brevo; el remitente debe estar verificado
`MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`	Mailtrap Sandbox	SMTP alternativo para desarrollo
`BUSINESS_NAME`, `BUSINESS_PHONE`	Multiservicios Wilson; vacío	Firma de los correos de pedido

Variable	Valor por defecto	Descripción
`OTP_TTL_MINUTES`, `OTP_MAX_ATTEMPTS`, `OTP_RESEND_COOLDOWN_SECONDS`, `RESET_TTL_MINUTES`	10, 5, 60, 30	Vigencia, intentos y espera de los códigos de un solo uso
`LOGIN_MAX_FAILURES`, `LOGIN_FAILURE_WINDOW_MINUTES`, `LOGIN_LOCK_MINUTES`	5, 15, 15	Bloqueo por fuerza bruta
`SECURITY_FAIL_ON_DEFAULT_ADMIN_PASSWORD`	`false`	`true` impide arrancar si `admin` conserva la contraseña sembrada (recomendado en producción)
`SPRING_PROFILES_ACTIVE`	vacío	`prod` desactiva el SQL en el registro, sube el nivel de Security a INFO y reintenta la primera conexión a la base durante 60 s
`PORT`, `SHOW_SQL`, `SECURITY_LOG_LEVEL`	8080, `false`, `WARN`	Puerto HTTP y trazas bajo demanda
`VITE_API_URL` (frontend)	`/` en producción	Se incrusta en el bundle al compilar; nunca debe contener secretos

24.4. Catálogo de la API REST

Todas las rutas devuelven JSON salvo las de descarga y las de vista previa. «Sesión» significa cualquier usuario autenticado (Administrador o Encargado). Los endpoints con «Administrador» responden 403 a un Encargado.

Método	Ruta	Acceso	Descripción
POST	`/auth/login`	Público	Valida credenciales; abre la sesión o devuelve el reto 2FA
POST	`/auth/2fa/verify`	Público (token de reto)	Verifica el código y abre la sesión
POST	`/auth/2fa/resend`	Público (token de reto)	Reenvía el código (espera de 60 s)
POST	`/auth/forgot-password`	Público	Envía el enlace de recuperación; respuesta uniforme
POST	`/auth/reset-password`	Público (token)	Define la nueva contraseña
GET	`/auth/check-auth`	Sesión	Confirma que la sesión sigue vigente
GET	`/auth/user/details`	Sesión	Datos del usuario en sesión (sin contraseña)
POST	`/auth/logout`	Público	Borra la cookie `jwt`
PUT	`/api/account/password`	Sesión	Cambia la contraseña (pide la actual)
PUT	`/api/account/email`	Sesión	Cambia el correo (pide la contraseña; desactiva el 2FA)
POST	`/api/account/2fa/enable`	Sesión	Envía el código de activación del 2FA
POST	`/api/account/2fa/confirm`	Sesión	Confirma el código y activa el 2FA
POST	`/api/account/2fa/disable`	Sesión	Desactiva el 2FA (pide la contraseña)
GET	`/api/users`	Administrador	Lista las cuentas
POST	`/api/users`	Administrador	Crea una cuenta
PUT	`/api/users/{id}`	Administrador	Edita nombre, rol y correo
PATCH	`/api/users/{id}/deactivate`	Administrador	Desactiva la cuenta
PATCH	`/api/users/{id}/activate`	Administrador	Reactiva la cuenta
GET	`/api/products`, `/api/products/{id}`	Sesión	Lista y consulta de productos con su estado de stock
POST	`/api/products`	Sesión	Crea un producto (código de barras único)
PUT	`/api/products/{id}`	Sesión	Edita un producto
DELETE	`/api/products/{id}`	Administrador	Baja lógica del producto
PATCH	`/api/products/{id}/activate`	Administrador	Reactiva el producto
GET	`/api/suppliers`, `/api/suppliers/{id}`	Sesión	Lista y consulta de proveedores
POST	`/api/suppliers`	Sesión	Crea un proveedor (correo y teléfono obligatorios)
PUT	`/api/suppliers/{id}`	Sesión	Edita un proveedor
DELETE	`/api/suppliers/{id}`	Administrador	Elimina un proveedor sin productos
POST	`/api/sales`	Sesión	Registra una venta; devuelve la venta, las alertas generadas y el estado del recibo

Método	Ruta	Acceso	Descripción
GET	`/api/sales/recent`	Sesión	Las 10 ventas más recientes
GET	`/api/stock-alerts/pending`	Sesión	Alertas pendientes
PATCH	`/api/stock-alerts/{id}/attend`	Administrador	Marca la alerta como atendida
POST	`/api/stock-alerts/{id}/supplier-order/preview`	Administrador	Vista previa del correo de pedido
POST	`/api/stock-alerts/{id}/supplier-order`	Administrador	Envía el pedido al proveedor
GET	`/api/notifications?read=`	Sesión	Notificaciones visibles para el rol
PATCH	`/api/notifications/read-all`, `/api/notifications/{id}/read`	Sesión	Marca como leídas
POST	`/api/documents/dpi`	Sesión	Genera la fotocopia de DPI (multipart: `front`, `back`, `format` PDF o DOCX, ajustes, giros y esquinas)
POST	`/api/documents/dpi/preview`	Sesión	Vista previa PNG de una cara procesada
POST	`/api/documents/dpi/detect-corners`	Sesión	Detecta las 4 esquinas y devuelve una vista previa JPEG
GET	`/api/documents/id-photo/specs`	Sesión	Medidas y capacidad de cédula y pasaporte
POST	`/api/documents/id-photo`	Sesión	Genera la hoja de fotos (tipo, copias, formato, marco de recorte)
POST	`/api/documents/id-photo/preview`	Sesión	Vista previa PNG de una copia
GET	`/api/reports/sales/summary?from&to`	Sesión	Indicadores de ventas y comparación con el periodo anterior
GET	`/api/reports/inventory/summary`	Sesión	Indicadores del inventario
GET	`/api/reports/sales?from&to&format`	Sesión	Descarga del reporte de ventas (PDF o XLSX)
GET	`/api/reports/inventory?format`	Sesión	Descarga del reporte de inventario (PDF o XLSX)
GET	`/actuator/health`	Público	Estado del servicio (`{"status":"UP"}`), usado por Railway y el CI

24.5. Modelado de datos

24.5.1. Modelo entidad-relación vigente

El esquema vigente corresponde a las migraciones V1 a V11 y consta de nueve tablas. Se verificó aplicando los scripts reales sobre una base PostgreSQL vacía y comparando el resultado con las entidades JPA.

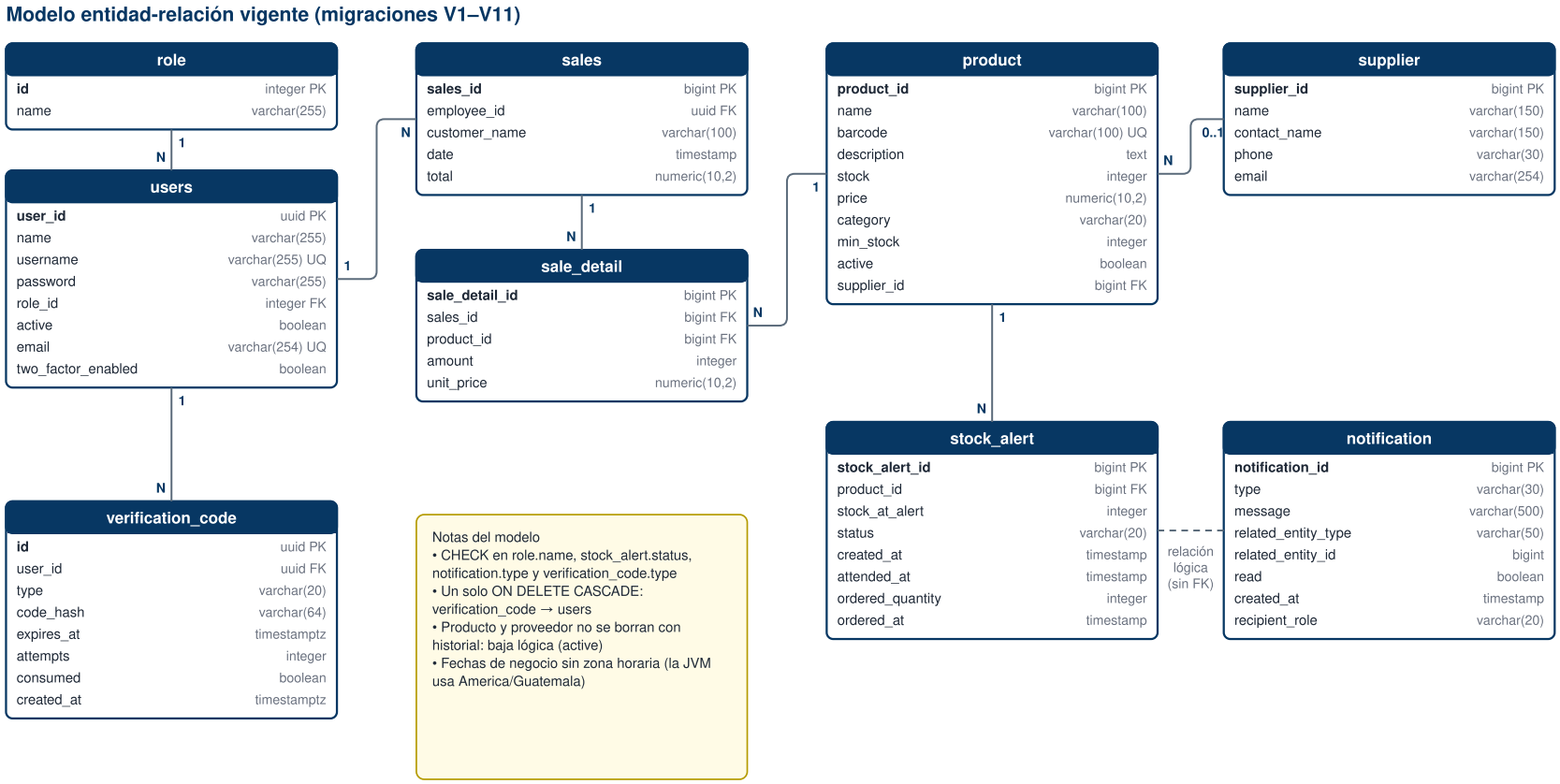


Figura 24.6 — Modelo entidad-relación vigente. Las claves foráneas y restricciones CHECK están definidas en las migraciones Flyway

24.5.2. Diccionario de datos

Tabla `role` — catálogo de roles de acceso.

Columna	Tipo	Restricciones y significado
`id`	integer	PK, identidad
`name`	varchar(255)	NOT NULL; CHECK en `ROLE_EMPLOYEE` o `ROLE_ADMIN`; sembrada de forma idempotente en V2

Tabla `users` — cuentas del sistema (la palabra `user` es reservada en PostgreSQL).

Columna	Tipo	Restricciones y significado
`user_id`	uuid	PK; lo genera la aplicación
`name`	varchar(255)	NOT NULL; nombre completo
`username`	varchar(255)	NOT NULL, único (`users_username_key`)
`password`	varchar(255)	NOT NULL; hash BCrypt, nunca se serializa (`@JsonIgnore`)
`role_id`	integer	NOT NULL; FK a `role(id)`
`active`	boolean	NOT NULL, por defecto TRUE (V4); una cuenta inactiva conserva su historial pero no puede entrar
`email`	varchar(254)	Opcional (V7); índice único sobre `LOWER(email)` cuando no es nulo
`two_factor_enabled`	boolean	NOT NULL, por defecto FALSE (V7); requiere `email`

Tabla `verification_code` — códigos y tokens de un solo uso (V8).

Columna	Tipo	Restricciones y significado
`id`	uuid	PK
`user_id`	uuid	NOT NULL; FK a `users` con `ON DELETE CASCADE`
`type`	varchar(20)	NOT NULL; CHECK en `LOGIN_2FA`, `ENABLE_2FA` o `PASSWORD_RESET`
`code_hash`	varchar(64)	NOT NULL; HMAC-SHA256 en hexadecimal; el código en claro nunca se guarda
`expires_at`, `created_at`	timestamptz	NOT NULL; vigencia y creación
`attempts`	integer	NOT NULL, por defecto 0; intentos fallidos
`consumed`	boolean	NOT NULL, por defecto FALSE; usado, agotado o reemplazado

Índices: `(user_id, type, created_at DESC)` y `(code_hash)`.

Tabla `supplier` — proveedores (V5 y V10).

Columna	Tipo	Restricciones y significado
`supplier_id`	bigint	PK, identidad
`name`	varchar(150)	NOT NULL
`contact_name`	varchar(150)	Persona de contacto
`phone`	varchar(30)	NOT NULL (V10); valores migrados «Pendiente»
`email`	varchar(254)	NOT NULL (V10); los proveedores previos recibieron `sin-correo-<id>@proveedor.invalid`, que el sistema rechaza al pedir producto

Tabla `product` — celulares y accesorios (V1 y V6).

Columna	Tipo	Restricciones y significado
`product_id`	bigint	PK, identidad
`name`	varchar(100)	NOT NULL
`barcode`	varchar(100)	NOT NULL, único (`product_barcode_key`)
`description`	text	Opcional (máximo 2000 caracteres por validación)
`stock`	integer	NOT NULL; unidades disponibles
`price`	numeric(10,2)	NOT NULL, por defecto 0; precio de venta
`category`	varchar(20)	`CELULAR` o `ACCESORIO`; hoy lo controla solo la aplicación (sin CHECK)

Columna	Tipo	Restricciones y significado
`min_stock`	integer	NOT NULL, por defecto 2; umbral de alerta por producto
`active`	boolean	NOT NULL, por defecto TRUE; baja lógica
`supplier_id`	bigint	FK opcional a `supplier`

Tabla `sales` y tabla `sale_detail` — ventas y sus líneas (V1).

Columna	Tipo	Restricciones y significado
`sales.sales_id`	bigint	PK, identidad
`sales.employee_id`	uuid	NOT NULL; FK a `users`; quién registró la venta
`sales.customer_name`	varchar(100)	Opcional; texto libre
`sales.date`	timestamp	NOT NULL; hora local del negocio (sin zona)
`sales.total`	numeric(10,2)	NOT NULL; suma de las líneas
`sale_detail.sale_detail_id`	bigint	PK, identidad
`sale_detail.sales_id`	bigint	NOT NULL; FK a `sales`
`sale_detail.product_id`	bigint	NOT NULL; FK a `product`
`sale_detail.amount`	integer	NOT NULL; cantidad vendida
`sale_detail.unit_price`	numeric(10,2)	NOT NULL; precio histórico al momento de la venta

Tabla `stock_alert` — alertas de reposición (V1 y V11).

Columna	Tipo	Restricciones y significado
`stock_alert_id`	bigint	PK, identidad
`product_id`	bigint	NOT NULL; FK a `product`
`stock_at_alert`	integer	NOT NULL; stock en el momento de la alerta
`status`	varchar(20)	NOT NULL; CHECK en `PENDIENTE` o `ATENDIDA`
`created_at`, `attended_at`	timestamp	Creación (NOT NULL) y atención
`ordered_quantity`, `ordered_at`	integer, timestamp	Último pedido enviado al proveedor desde la alerta (V11)

Tabla `notification` — eventos de la campana (V3 y V9).

Columna	Tipo	Restricciones y significado
`notification_id`	bigint	PK, identidad
`type`	varchar(30)	NOT NULL; CHECK en `STOCK_BAJO`, `DOCUMENTO_GENERADO`, `AGOTADO` o `PRODUCTO_ESTADO`
`message`	varchar(500)	NOT NULL
`related_entity_type`, `related_entity_id`	varchar(50), bigint	Referencia lógica (sin FK) a la entidad que originó el evento
`read`	boolean	NOT NULL, por defecto FALSE; marca global (limitación documentada en la sección 28)
`created_at`	timestamp	NOT NULL
`recipient_role`	varchar(20)	Nulo = visible para todos; CHECK en los dos roles

Índice: `(read, created_at DESC)`.

24.5.3. Restricciones, índices e integridad

Tipo	Vigente	Observación
Unicidad	`users.username`, `product.barcode`, `LOWER(users.email)` (parcial)	Un correo identifica a una sola cuenta sin distinguir mayúsculas
Claves foráneas	7 relaciones (`users→role`, `sales→users`, `sale_detail→sales` y `→product`, `stock_alert→product`, `product→supplier`, `verification_code→users`, entre otras)	Solo `verification_code` usa `ON DELETE CASCADE`; el resto restringe el borrado para proteger el historial

Tipo	Vigente	Observación
CHECK	Dominios cerrados de `role.name`, `stock_alert.status`, `notification.type` y `recipient_role`, `verification_code.type`	No hay CHECK numéricos (stock, precio, cantidad); la aplicación los valida
Índices secundarios	`notification(read, created_at DESC)`, `verification_code(user_id, type, created_at DESC)` y `(code_hash)`	No hay índices sobre `sales.date`, claves foráneas de `sale_detail` ni `stock_alert.status`
Baja lógica	`product.active`, `users.active`	Evita borrar entidades con historial
Unicidad de alerta pendiente	Solo en el servicio (`SaleServiceImpl`)	Una condición de carrera podría crear dos; la migración V12 la lleva a la base de datos

24.5.4. Concurrencia y transacciones

La venta es la única operación con riesgo de carrera real. `ProductRepository.findByIdForUpdate` ejecuta `SELECT … FOR UPDATE` (bloqueo pesimista de escritura) sobre cada producto, siempre en orden ascendente de identificador, de modo que dos ventas simultáneas se serializan y no se produce un interbloqueo. La prueba empírica del 24/10/2026 lanzó 10 ventas simultáneas de una unidad sobre un producto con stock 3: hubo exactamente 3 ventas exitosas, 7 respuestas 409 y el stock final fue 0 (antes de este cambio podía quedar negativo). El resto de operaciones (productos, proveedores, usuarios) son transacciones cortas de una sola fila.

24.5.5. Evolución del esquema aplicada (V1 a V11)

Versión	Fecha	Archivo	Cambio y efecto
V1	14/09	`V1__init.sql`	Esquema inicial generado desde Hibernate: `role`, `users`, `product`, `sales`, `sale_detail`, `stock_alert`; extensión `pgcrypto`
V2	14/09	`V2__seed_roles_and_admin.sql`	Siembra idempotente de los dos roles y del usuario `admin`
V3	14/09	`V3__create_notification.sql`	Tabla `notification` (RF-06) e índice por lectura y fecha
V4	14/09	`V4__add_user_active.sql`	`users.active` para desactivar cuentas sin perder historial
V5	15/09	`V5__create_supplier.sql`	Tabla `supplier`
V6	15/09	`V6__add_product_attributes.sql`	`price`, `category`, `min_stock`, `active` y `supplier_id` en `product`
V7	24/10	`V7__user_email_two_factor.sql`	`users.email` y `two_factor_enabled`; índice único parcial por correo
V8	24/10	`V8__create_verification_code.sql`	Tabla de códigos de un solo uso con HMAC
V9	24/10	`V9__notification_new_types.sql`	CHECK ampliado: `AGOTADO` y `PRODUCTO_ESTADO`
V10	25/10	`V10__supplier_email_required.sql`	Correo y teléfono obligatorios en `supplier`, con valores provisionales para los existentes
V11	25/10	`V11__stock_alert_supplier_order.sql`	`ordered_quantity` y `ordered_at` en `stock_alert`

Las bases creadas antes de adoptar Flyway se marcan como si V1 ya estuviera aplicada (`baseline-on-migrate=true`, `baseline-version=1`); en una base vacía Flyway corre desde V1. La política y la hoja de ruta de nuevas migraciones están en la sección 28.

24.6. Trazabilidad

24.6.1. Matriz de requisitos, historias, componentes y pruebas

La matriz relaciona cada requisito con las historias que lo cumplen, los componentes que lo implementan y la evidencia de prueba. Los números entre paréntesis son la cantidad de pruebas automatizadas de cada clase.

Req.	Historias	Backend	Frontend	Evidencia
RF-01	HU-03 a 05, 38 a 42	`AuthController`, `AuthService`, `JwtUtil`, `JwtAuthenticationFilter`, `SecurityConfig`, `UserController`, `UserManagementServiceImpl`	`sign-in.jsx`, `AuthContext`, `RoleRoute`, `menuByRole.js`, `Users.jsx`	`JwtUtilTest` (5); bitácora de validaciones: 21/21
RF-02	HU-10 a 15	`ProductController`, `ProductServiceImpl`, `StockStatusCalculator`	`Products.jsx`, `CreateProduct.jsx`, `EditProduct.jsx`, `useProducts`	`StockStatusCalculatorTest` (4); código de barras duplicado → 409

Req.	Historias	Backend	Frontend	Evidencia
RF-03	HU-20 a 22	`SaleController`, `SaleServiceImpl`, `ProductRepository.findByIdForUpdate`	`Sales.jsx`, `saleService`	Prueba de concurrencia: 10 ventas, 3 exitosas
RF-04	HU-23, 25, 26, 49	`StockAlertServiceImpl`, `SupplierOrderService`, `StockAlertController`	`Alerts.jsx`, `SupplierOrderDialog`	`StockAlertServiceImplTest` (3), `SupplierOrderServiceTest` (6)
RF-05	HU-27 a 34	`DocumentController`, `DpiDocumentServiceImpl`, paquete `imaging`	`DpiDocuments.jsx`, `CornerAdjustOverlay`, `photoCrop`	`CardDetectionServiceImplTest` (5), `ImageCropServiceImplTest` (12), `ExifOrientationNormalizerImplTest` (5), `DocumentControllerIdPhotoTest` (11), `photoCrop.test` (16)
RF-06	HU-07, 34, 54	`NotificationController`, `NotificationServiceImpl`	`notification-bell.jsx`, `useNotifications`	Bitácora de validaciones (notificaciones por rol)
RF-07 y RF-13	HU-35 a 37, 51	`ReportController`, `SalesAnalyticsServiceImpl`, `PdfReportWriter`, `XlsxReportWriter`	`Reports.jsx`, `SalesReport.jsx`, `InventoryReport.jsx`, `ExportButtons`	`ReportWritersTest` (8), `SalesAnalyticsServiceImplTest` (10), `dateRanges.test` (10); bitácora de reportes: 24/24
RF-08	HU-43 a 47	`AuthService`, `VerificationCodeService`, `PasswordRecoveryServiceImpl`, `AccountServiceImpl`, `LoginAttemptService`	`TwoFactorForm`, `forgot-password`, `reset-password`, `SecuritySettings`	`VerificationCodeServiceTest` (12), `PasswordRecoveryServiceImplTest` (6), `LoginAttemptServiceTest` (5), `MailServiceImplTest` (11); bitácoras 48/48 y 30/30
RF-09	HU-52, 53	`IdPhotoDocumentServiceImpl`, `DocumentLayoutRegistry`, compositores de hoja	`IdPhotoForm`, `useIdPhotoSpecs`, `useIdPhotoPreview`	`IdPhotoDocumentServiceImplTest` (12), `PhotoSheetSpecTest` (11), `PdfPhotoSheetComposerTest` (7), `WordPhotoSheetComposerTest` (11)
RF-10	HU-49	`SupplierOrderService`, `MailServiceImpl`	`SupplierOrderDialog`, `supplier.js`	`SupplierOrderServiceTest` (6), `supplier.test` (3)
RF-11	HU-48	`SaleReceiptService`, `ReceiptEmailLimiter`	`Sales.jsx`	`SaleReceiptServiceTest` (4), `ReceiptEmailLimiterTest` (3)
RF-12	HU-50	— (procesamiento en el navegador)	`ScanButton`, `BarcodeScannerDialog`, `barcode.js`	`barcode.test` (8)
RNF-01	—	`Dockerfile`, `ci-cd.yml`	Despliegue en Netlify	Prueba de humo del pipeline
RNF-02	—	`ImageCropServiceImpl` (límite de 2560 px)	`imageDownscale.js`	Sin el límite se midieron ≈ 13 s con 1 vCPU; medir p95 en producción (25.6)
RNF-03	—	`CookieService`, `SecurityConfig`, BCrypt, HMAC	HTTPS de Netlify, `_headers`	`JwtUtilTest`, `VerificationCodeServiceTest`
RNF-04	—	Flyway V1 a V11	—	`InventoryBackApplicationTests` y el CI aplican todas las migraciones
RNF-05	—	—	—	Historial de ramas y Pull Requests (24.6.2)
RNF-06	—	Workflows de GitHub Actions	idem	Prueba de humo posterior al despliegue
RNF-07	—	`VerificationCodeService`	—	`VerificationCodeServiceTest` (12)
RNF-08	—	Servicios de documentos sin repositorio	—	Revisión de código: no hay persistencia de imágenes
RNF-09	—	`SaleServiceImpl`	—	Prueba de concurrencia
RNF-10	—	`GlobalExceptionHandler`	`api.js` (interceptor 401)	Bitácora de validaciones: 21/21

24.6.2. Trazabilidad del código: ramas, Pull Requests y entornos

El flujo adoptado es GitFlow (figura 25.2 de la sección 25): `main` contiene el código estable, `develop` integra, `feature/*` nace de `develop` y `hotfix/*` nace de `main`. Cada fusión se registra como Pull Request en GitHub. La tabla resume los Pull Requests fusionados en los dos repositorios.

PR	Fusión	Rama origen → destino	Contenido
Back #1 y Front #1	21/10	`chore/cloud-deploy`, `chore/netlify-deploy` → `develop`	Despliegue en la nube: Dockerfile, healthcheck, CI/CD hacia Railway y Netlify

PR	Fusión	Rama origen → destino	Contenido
Back #2 y Front #3	25/10	`integration/2fa` → `main`	2FA por correo, recuperación de contraseña, validaciones de negocio, reportes rediseñados, pruebas Vitest y bitácoras
Back #3 y Front #5	25/10	`feature/ajustes-ui-reportes-notificaciones` → `main`	Ajustes de interfaz, notificaciones y ventas
Front #4	25/10	`feat/barcode-scanner` → `main`	Escáner de códigos de barras para móvil
Front #6	25/10	`feature/modo-oscuro` → `main`	Modo oscuro
Back #4	25/10	`feature/correo-brevo-api` → `main`	Envío por la API HTTP de Brevo
Back #6 y Front #8	25/10	`feature/recibo-por-correo` → `main`	Recibo de venta por correo
Back #8, #10 y Front #10, #12	25/10	`feature/pedido-a-proveedor` → `main`	Pedido de reposición al proveedor desde la alerta
Back #11 y #12	25/10	`fix/repair-email-description` → `main`	Corrección de la descripción de los correos
Back #13 y Front #13	26/10	`feature/cedula-passport-integration` → `main`	Fotos tamaño cédula y pasaporte

> Observación de proceso: entre el 24 y el 26 de octubre los Pull Requests se abrieron directamente hacia `main` para acelerar la entrega de la Fase 3, en lugar de pasar primero por `develop`, y el repositorio lo opera una sola persona, por lo que no queda evidencia automática de la revisión de un segundo integrante. Para cumplir plenamente RNF-05 se recomienda activar la protección de las ramas `main` y `develop` en GitHub (revisión obligatoria, estado de CI en verde y prohibición de push directo) y retomar el camino `feature/*` → `develop` → `main`, como lo describen la figura 25.2 y el BPMN-08.

Los entornos se corresponden con las ramas así: las ejecuciones sobre Pull Requests y `develop` solo corren el CI; un push a `main` corre el CI y despliega al entorno `production` de GitHub, con su prueba de humo. No existe un entorno de pruebas (staging) separado; se propone en la sección 29.

24.6.3. Trazabilidad con el enunciado del proyecto

Pilar o requisito del enunciado	Evidencia en el sistema o en este documento	Sección
BPM: producto tecnológico a partir de una necesidad del cliente	Mapa de procesos, BPMN AS-IS y TO-BE, priorización y requisitos	1 a 17 y 27
Infraestructura y despliegue cloud con URL pública	Netlify, Railway y PostgreSQL en producción; healthcheck	25.2
Automatización (CI/CD)	Dos workflows de GitHub Actions con pruebas, construcción, despliegue y prueba de humo	25.4
Seguridad y gestión de datos: identidades, cifrado y versionado de bases	JWT en cookie HttpOnly, BCrypt, HMAC, HTTPS, roles, Flyway	24.3.5 y 28
Calidad y colaboración: GitFlow y revisión	Ramas y Pull Requests	24.6.2
Autenticación: login con sesiones seguras, recuperación y 2FA por correo	HU-03, 43 a 46; BPMN-05 y BPMN-06	23.5 y 27
Gestión de roles (mínimo dos perfiles)	Administrador y Encargado; no hay registro público por ser un sistema B2E	24.3.5
CRUD completo de al menos una entidad	Productos, proveedores y usuarios	24.4
Interactividad: notificaciones	Campana por rol y correos transaccionales	24.3.7
Reportes con exportación a Excel y PDF	Reportes de ventas e inventario	24.3.7 y BPMN-07
Estrategia de respaldo (completo, incremental y registro de transacciones)	Estrategia, RPO y RTO y procedimiento de restauración	25.7
Gestión de migraciones de base de datos	Flyway V1 a V11 y plan V12 a V18	24.5.5 y 28
Documentación para escalabilidad futura	Procesos, migraciones y tecnologías propuestas	26 a 29

Pilar o requisito del enunciado	Evidencia en el sistema o en este documento	Sección
Metodología ágil (Scrum con Jira, sprints de 3 semanas)	Planificada en la sección 16; los tableros de Jira son evidencia externa	16

24.7. Calidad y pruebas

Inventario de pruebas automatizadas

Área	Pruebas	Qué verifican
Seguridad y autenticación (backend)	28	JWT, bloqueo por fallos, códigos OTP (vigencia, intentos, espera), recuperación de contraseña
Correo y pedidos (backend)	31	Plantillas y envío, cliente de Brevo, recibos y su límite por hora, pedido a proveedor, validación del proveedor
Inventario (backend)	7	Estado de stock (disponible, bajo, agotado) y alertas
Imágenes y documentos (backend)	95	Detección de bordes, geometría, recorte, EXIF, mejora, especificaciones de hoja, PDF y Word, y el endpoint de fotos
Reportes (backend)	18	Analítica de ventas con comparación de periodos y exportadores PDF y Excel
Contexto de Spring (backend)	1	Arranque contra PostgreSQL real: aplica V1 a V11 y valida las entidades
Frontend (Vitest)	71	Validadores, recorte de fotos, rangos de fechas, formato, códigos de barras, proveedor y utilidades
Total	**251**	180 en el backend y 71 en el frontend (además existe `ManualRealPhotoCheck`, una inspección manual con fotos reales que nunca corre en el CI)

Pruebas empíricas registradas

Bitácora	Fecha	Resultado	Alcance
Validaciones de negocio	24/10/2026	21 de 21	Productos, ventas (precio del servidor, stock, concurrencia), usuarios y notificaciones; PostgreSQL 15 real
Correo, 2FA y recuperación	24/10/2026	48 de 48	Flujo HTTP → servicio → SMTP local → contenido del correo; 5 correos esperados
Interfaz	24/10/2026	30 de 30	Login, 2FA, sesión, recuperación, perfil y usuarios en Chromium real
Reportes	24/10/2026	24 de 24	Cifras contra SQL directo (total Q 291,376.00, 69 transacciones, 248 unidades) y exportaciones

Puertas de calidad y limitaciones conocidas

- **CI:** cada Pull Request y cada push a `develop` o `main` ejecutan las pruebas contra PostgreSQL 17 y construyen la imagen Docker o el bundle de Vite; sin CI en verde no hay despliegue.
- **Cobertura:** no hay una herramienta de cobertura configurada (por ejemplo JaCoCo); se propone en la sección 29.
- **Word:** los `.docx` se validaron por su estructura XML, no con Microsoft Word.
- **Correo real:** las bitácoras usaron un SMTP local descartable; el envío real por Brevo se incorporó después (PR #4 del backend) y se verifica en cada ambiente con la lista de la sección 25.4.
- **Rendimiento:** no hay pruebas de carga; las medidas de memoria y tiempo provienen de pruebas puntuales con 1 vCPU (sección 26.1).
- **Móviles:** la interfaz se verificó en un ancho de 390 px, no en dispositivos reales.

25. Guía de Despliegue y Runbook

25.1. Propósito, alcance y convenciones

Esta guía permite a una persona con conocimientos básicos de Git, Docker y bases de datos desplegar el sistema desde cero, entregar cambios, revertirlos, vigilar su salud y resolver incidentes sin conocer el código. Cubre las dos piezas desplegadas (API y SPA) y la base de datos. Los nombres de pantallas de Railway, Netlify, GitHub y Brevo pueden cambiar con el tiempo: si una ruta de menú ya no coincide, busque la opción equivalente en la consola del proveedor; los comandos y las variables de este documento sí provienen de los archivos del repositorio.

Convención	Significado
Operador	Persona responsable de la operación técnica (administrador técnico del sistema)
`$VARIABLE`	Valor que se obtiene del gestor de secretos o de la consola del proveedor; nunca se escribe en el repositorio
SEV1, SEV2 y SEV3	Severidades de incidente definidas en 25.8.1
Pipeline	Flujo de GitHub Actions `.github/workflows/ci-cd.yml` de cada repositorio

25.2. Topología de producción

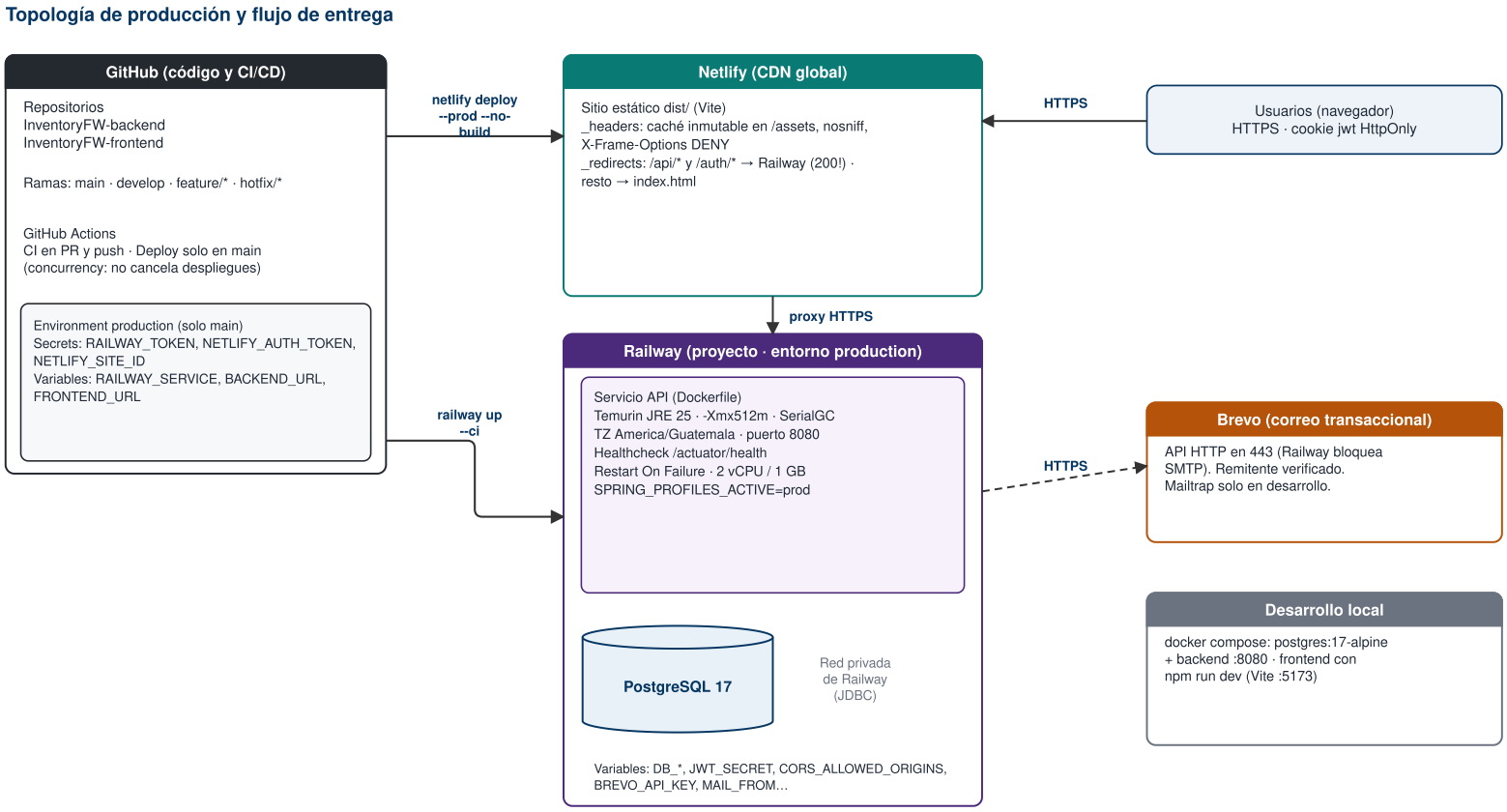


Figura 25.1 — Topología de producción: GitHub Actions entrega a Netlify y Railway; el navegador solo conoce el dominio de Netlify

Componentes y proveedores

Componente	Proveedor	Qué hospeda	Configuración clave
Código y CI/CD	GitHub	Repositorios `InventoryFW-backend` e `InventoryFW-frontend` y sus workflows	Environment `production` restringido a la rama `main`; concurrencia por rama
SPA y CDN	Netlify	`dist/` generado por Vite; proxy de `/api/*` y de los endpoints de `/auth/` hacia Railway	Despliegue por CLI sin conectar el repositorio; `_headers` y `_redirects` van dentro del build
API	Railway	Contenedor construido desde el `Dockerfile` (Temurin 25, `-Xmx512m`, `SerialGC`, zona horaria America/Guatemala)	Dominio público al puerto 8080; healthcheck `/actuator/health`; reinicio «On Failure»; límite de 2 vCPU y 1 GB de RAM; modo Serverless apagado
Base de datos	Railway	PostgreSQL 17 en la red privada del proyecto	La API se conecta con `DB_URL` armada con las variables del servicio de PostgreSQL
Correo	Brevo	Envío transaccional por API HTTP (443)	Clave `BREVO_API_KEY`; remitente verificado en Brevo

El dominio actual del API es `https://inventory-backend-production-ad86.up.railway.app` (su estado se consulta en `/actuator/health`). La URL pública del sistema es la del sitio de Netlify y está guardada en la variable `FRONTEND_URL` del environment `production` del repositorio del frontend.

> **Importante:** el modo Serverless de Railway debe permanecer apagado. Con 1 vCPU el arranque tarda cerca de 30 segundos, más que el límite de 26 segundos del proxy de Netlify, y la primera petición del día fallaría.

25.3. Requisitos previos, cuentas y secretos

Cuentas y herramientas

Elemento	Detalle
Cuentas	GitHub (con acceso a ambos repositorios), Railway (proyecto con PostgreSQL y un servicio vacío para la API), Netlify (un sitio vacío) y Brevo (remitente verificado y clave de API)
Herramientas locales	Git, Docker y Docker Compose, JDK 25, Node 24 (archivo `.nvmrc`), Railway CLI 5.70.0 y Netlify CLI 27.13.0 (las versiones que fijan los workflows)
Entorno local completo	`docker compose up --build` en el backend levanta PostgreSQL 17 (puerto 5433) y la API (8080); el frontend se corre aparte con `npm run dev` (puerto 5173)

Secretos y variables por sitio

Dónde	Nombre	Para qué
GitHub, environment `production` del backend	Secreto `RAILWAY_TOKEN`; variables `RAILWAY_SERVICE`, `BACKEND_URL`	Token de proyecto de Railway limitado al entorno `production`; nombre del servicio; URL para la prueba de humo
GitHub, environment `production` del frontend	Secretos `NETLIFY_AUTH_TOKEN`, `NETLIFY_SITE_ID`; variable `FRONTEND_URL`	Publicar el sitio y probar el proxy
Railway, servicio de la API	`SPRING_PROFILES_ACTIVE=prod`, `PORT=8080`, `DB_URL`, `DB_USER`, `DB_PASSWORD`, `JWT_SECRET`, `CORS_ALLOWED_ORIGINS`, `FRONTEND_URL`, `BREVO_API_KEY`, `MAIL_FROM`, `MAIL_FROM_NAME`	Configuración de producción (descrita en 24.3.9); `DB_*` se arman con referencias al servicio de PostgreSQL de Railway
Railway, servicio de la API (recomendado)	`SECURITY_FAIL_ON_DEFAULT_ADMIN_PASSWORD=true`	Impide arrancar mientras `admin` use la contraseña sembrada
Frontend, archivo `.env.production`	`VITE_API_URL=/`	Valor público incrustado en el bundle; nunca contiene secretos

> **Advertencia:** los secretos nunca se escriben en el repositorio, en tickets ni en capturas de pantalla. Guárdelos en un gestor de contraseñas con acceso compartido solo a quien opera el sistema.

25.4. Despliegue

25.4.1. Flujo de entrega

El sistema se entrega de forma continua: cada Pull Request y cada push a `develop` o `main` ejecutan el CI; un push a `main` ejecuta además el despliegue al entorno `production`. Los workflows tienen concurrencia por rama: en un PR se cancela la ejecución anterior si llega un push nuevo, pero un despliegue a `main` nunca se cancela a la mitad.

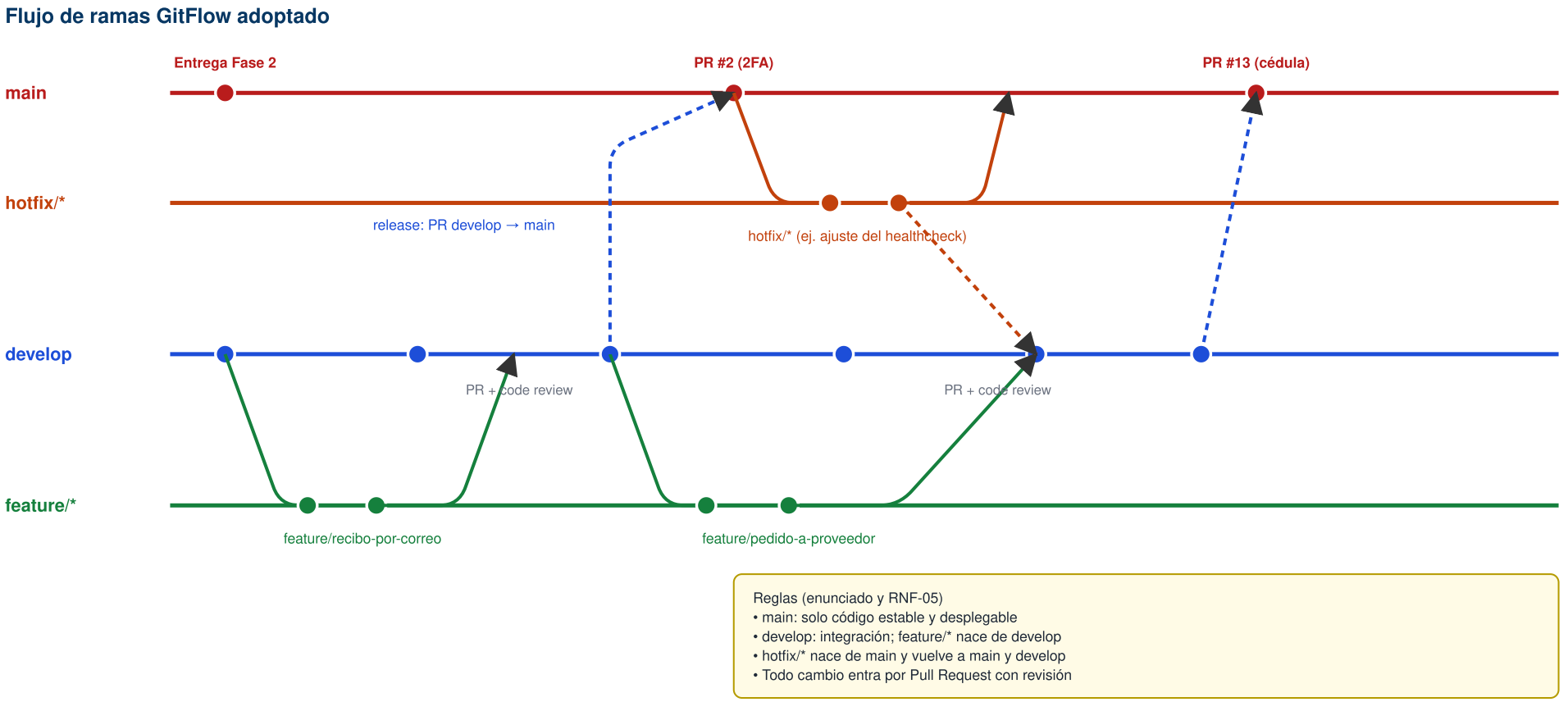
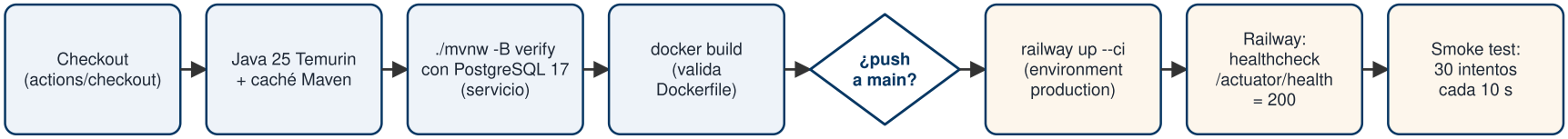


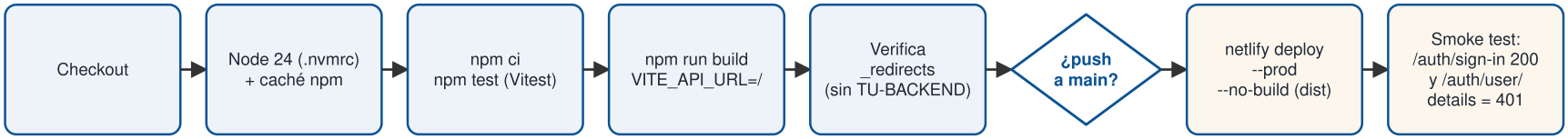
Figura 25.2 — Flujo de ramas GitFlow adoptado

Pipelines de GitHub Actions (CI en PR y push · Deploy solo en main)

Backend · InventoryFW-backend



Frontend · InventoryFW-frontend



Cada ejecución usa la concurrencia por rama: en PR se cancela la anterior; en main nunca se cancela un despliegue en curso. El artefacto dist se conserva 7 días.

Figura 25.3 — Pipelines de GitHub Actions del backend y del frontend

25.4.2. Procedimiento estándar de entrega

1. Crear la rama `feature/<nombre>` desde `develop` y hacer commits pequeños con mensajes descriptivos.
2. Abrir un Pull Request hacia `develop`. El CI ejecuta las pruebas (backend: `./mvnw -B verify` contra PostgreSQL 17 real, que aplica todas las migraciones; frontend: `npm test` y `npm run build`) y construye la imagen Docker o el bundle.
3. Un segundo integrante revisa y aprueba el PR (revisión de código obligatoria).
4. Fusionar a `develop`. Cuando el conjunto de cambios esté listo para producción, abrir un PR de `develop` hacia `main` y fusionarlo.
5. El push a `main` dispara el despliegue. **Backend:** `railway up --ci --service "$RAILWAY_SERVICE"` sube el código (respeta `.gitignore`), Railway construye el `Dockerfile` y solo desvía el tráfico a la versión nueva cuando `/actuator/health` responde 200; si no, la versión anterior sigue sirviendo. **Frontend:** se publica el `dist/` construido en el CI con `netlify deploy --prod --no-build`.
6. La prueba de humo automática confirma el resultado. Backend: hasta 30 intentos cada 10 segundos de `GET $BACKEND_URL/actuator/health` buscando `"status":"UP"`. Frontend: `GET /auth/sign-in` debe responder 200 y `GET /auth/user/details` sin sesión debe responder exactamente 401 (si el proxy fallara, el respaldo de la SPA devolvería 200).
7. Completar la lista de verificación de 25.4.5.

> **Nota:** si un cambio toca a la vez la API y la interfaz (por ejemplo, un endpoint nuevo y su pantalla), despliegue primero el backend con un cambio compatible hacia atrás y después el frontend. Las migraciones se ejecutan al arrancar la API nueva (Flyway); PostgreSQL permite DDL transaccional, de modo que una migración que falle se revierte completa, la API no arranca, el healthcheck no pasa y la versión anterior sigue atendiendo.

25.4.3. Primer despliegue en un entorno nuevo

Backend

1. En Railway, crear un proyecto, agregar PostgreSQL y un servicio vacío (sin repositorio conectado: los despliegues llegan desde GitHub Actions) en la misma región.
2. En el servicio, definir las variables de 25.3. `DB_URL` usa la referencia `jdbc:postgresql://${{Postgres.PGHOST}}:${{Postgres.PGPORT}}/${{Postgres.PGDATABASE}}`, y `DB_USER` y `DB_PASSWORD` las de `PGUSER` y `PGPASSWORD`. Generar `JWT_SECRET` con `openssl rand -base64 48`.
3. En Settings: dominio público con puerto de destino 8080, healthcheck `/actuator/health`, reinicio «On Failure», límites de 2 vCPU y 1 GB y Serverless apagado.
4. En GitHub, crear el environment `production` limitado a `main`; guardar `RAILWAY_TOKEN` (token de proyecto del entorno `production`) y las variables `RAILWAY_SERVICE` y `BACKEND_URL`.
5. Hacer push a `main`. El primer arranque ejecuta V1 a V11: crea las tablas, los dos roles y el usuario `admin`.
6. **De inmediato:** entrar como `admin` (la contraseña de la migración V2 es pública en el historial del repositorio), crear otro Administrador con contraseña propia y correo, desactivar `admin` y activar `SECURITY_FAIL_ON_DEFAULT_ADMIN_PASSWORD=true`.

Frontend

1. En Netlify, crear un sitio vacío (sin conectar el repositorio) y obtener su identificador y un token personal.
2. Guardar `NETLIFY_AUTH_TOKEN` y `NETLIFY_SITE_ID` como secretos y `FRONTEND_URL` como variable del environment `production` del repositorio del frontend.
3. Editar `public/_redirects` para que todas las reglas apunten al dominio del API en Railway. El CI falla mientras exista el marcador `TU-BACKEND` o si la regla `/api/*` no tiene la forma esperada.
4. En Railway, fijar `CORS_ALLOWED_ORIGINS` con la URL exacta del sitio de Netlify (con `https://` y sin barra final) y `FRONTEND_URL` con esa misma URL.
5. Hacer push a `main` y ejecutar la lista de 25.4.5.

Correo

En Brevo, verificar el remitente (Senders & IP) y generar una clave de API; guardarla en `BREVO_API_KEY` junto con `MAIL_FROM`. Si `BREVO_API_KEY` tiene valor, las variables SMTP se ignoran. Railway bloquea el SMTP saliente, por lo que en producción debe usarse la API HTTP.

25.4.4. Despliegue manual de emergencia

Solo con autorización del responsable y registrando el motivo en el post-mortem. Deben existir los mismos secretos en el entorno local.

```
# Backend (desde la carpeta InventoryFW-backend)
export RAILWAY_TOKEN=...            # token de proyecto del entorno production
npx --yes @railway/cli@5.70.0 up --ci --service "$RAILWAY_SERVICE"

# Frontend (desde la carpeta InventoryFW-frontend)
npm ci && npm test && npm run build
npx --yes netlify-cli@27.13.0 deploy --prod --no-build --dir dist \
    --site "$NETLIFY_SITE_ID" --auth "$NETLIFY_AUTH_TOKEN" --message "despliegue manual"
```

25.4.5. Verificación posterior al despliegue

N.º	Comprobación	Resultado esperado
1	`curl -fsS "$BACKEND_URL/actuator/health"`	`{"status":"UP"}`
2	`curl -s -o /dev/null -w '%{http_code}' "$FRONTEND_URL/auth/sign-in"`	200 (la página de inicio de sesión de la SPA)
3	`curl -s -o /dev/null -w '%{http_code}' "$FRONTEND_URL/auth/user/details"`	401 (el proxy llega al API y no hay sesión)
4	Iniciar sesión con un usuario de prueba; si tiene 2FA, comprobar que llega el código	Entra al inicio del rol correspondiente
5	Registrar una venta de un producto de prueba	Stock descendido; alerta creada si queda en el mínimo; recibo `ENVIADO` si se indicó correo
6	Generar una fotocopia de DPI con fotos de prueba	Documento en menos de 10 segundos (RNF-02)
7	Descargar un reporte de ventas en PDF y en Excel	Archivos con nombre y formato correctos
8	Revisar el registro del servicio durante los primeros 10 minutos	Sin líneas `ERROR` nuevas; la zona horaria del registro es America/Guatemala

25.5. Rollback

25.5.1. Cuándo revertir

Revertir es la primera opción cuando un despliegue reciente coincide con el inicio de una falla grave y la corrección no es evidente y rápida. Se decide con el árbol de la figura 25.4.

Señal tras un despliegue	Decisión
Healthcheck en DOWN o errores 5xx sostenidos durante 5 minutos	Rollback inmediato de la aplicación (SEV1)
La prueba de humo automática falla	Rollback inmediato; el pipeline ya dejó la versión anterior sirviendo si falló el healthcheck
Funcionalidad crítica rota (iniciar sesión, vender) sin servicio caído	Rollback de la aplicación (SEV2)
Defecto menor con una solución clara de menos de una hora	Corrección hacia adelante con `hotfix/*`
Datos dañados por una migración o un defecto	Corrección hacia adelante o restauración desde respaldo (25.5.4 y 25.7)

Árbol de decisión para revertir un despliegue

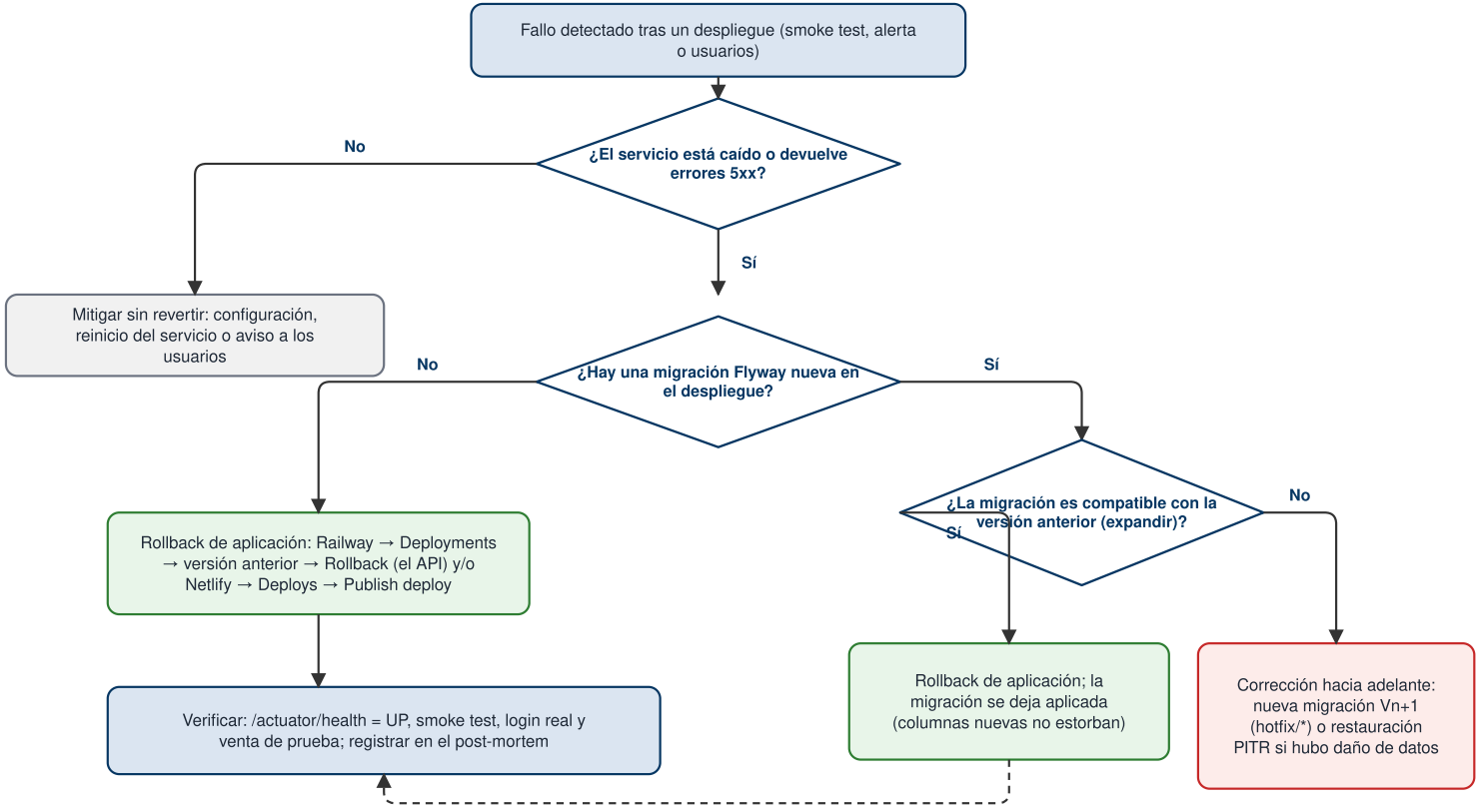


Figura 25.4 — Árbol de decisión para revertir un despliegue

25.5.2. Rollback del backend (Railway)

1. En Railway, abrir el servicio de la API y la pestaña de despliegues. Identificar el último despliegue exitoso anterior al problema (anote su fecha y confirme el commit correspondiente en GitHub).
2. Ejecutar la acción de reversión (Rollback o redespliegue) sobre ese despliegue. Railway vuelve a levantar la imagen anterior y solo le pasa tráfico cuando el healthcheck responde 200.
3. Comprobar `/actuator/health` y la lista 25.4.5, puntos 1 a 6.
4. Evaluar la base de datos: si el despliegue fallido incluía una migración, aplique 25.5.4 antes de dar por resuelto el incidente.
5. Alinear el repositorio con producción para que el siguiente push no vuelva a desplegar el defecto: revertir el commit en una rama `hotfix/revert-<tema>` con `git revert <sha>` y fusionarla a `main` y `develop` mediante Pull Request.

25.5.3. Rollback del frontend (Netlify)

En Netlify, abrir el sitio, entrar a la lista de despliegues y elegir el último despliegue de producción correcto (cada uno lleva en su mensaje el SHA del commit, porque el workflow usa `--message "$GITHUB_SHA"`). Usar la acción «Publish deploy»: la publicación es atómica e inmediata, sin reconstruir. Luego comprobar los puntos 2 y 3 de la lista 25.4.5 y alinear el repositorio con `git revert`, igual que en el backend.

25.5.4. Rollback de la base de datos

Las migraciones de Flyway son de avance únicamente: la edición Community no ofrece deshacer. Por eso el diseño sigue el patrón expandir y contraer (sección 28): primero se agregan columnas y tablas compatibles con la versión anterior del código y solo después se retiran las columnas que dejaron de usarse. Con esa regla, la mayoría de los rollbacks de aplicación no necesitan tocar la base.

1. **Si la migración nueva es compatible con la versión anterior** (solo agrega tablas o columnas opcionales): haga el rollback de la aplicación y deje la migración aplicada.
2. **Si el defecto está en la migración o en los datos**: escriba una migración nueva `V<n+1>` que lo corrija (`hotfix/*`), pruebe en una copia de la base y despliegue hacia adelante.
3. **Si hubo pérdida o daño de datos**: restaure desde el respaldo o por PITR en una instancia nueva (25.7), verifique y apunte el API a ella.
4. **Nunca** edite una migración ya aplicada (Flyway valida su checksum y el arranque fallará). `flyway repair` solo se usa con autorización y dejando constancia.

25.5.5. Procedimiento de hotfix con GitFlow

1. Crear `hotfix/<nombre>` desde `main`.
2. Corregir y probar localmente, incluyendo la prueba que reproduce el error.
3. Abrir un Pull Request hacia `main` con revisión; al fusionar se despliega.
4. Fusionar también la rama hacia `develop`.
5. Registrar el incidente y las acciones (25.8.6).

Se recomienda etiquetar la versión (`git tag`) tras cada despliegue a producción para facilitar identificar el punto de retorno.

Etapa	Meta	Responsable
Detectar la falla	≤ 5 minutos (healthcheck externo)	Monitoreo
Decidir revertir	≤ 10 minutos	Operador
Ejecutar el rollback	≤ 15 minutos	Operador
Verificar y comunicar	≤ 15 minutos	Operador
Tiempo total hasta restaurar el servicio (RTO de aplicación)	**≤ 45 minutos**	Operador

25.6. Monitoreo y observabilidad

25.6.1. Qué se monitorea hoy

Señal	Fuente	Cómo consultarla
Salud del API	Endpoint `/actuator/health` (público, sin detalles); Railway lo consulta en cada despliegue y el pipeline en la prueba de humo	`curl -fsS "$BACKEND_URL/actuator/health"`
Consumo de CPU, memoria y red	Métricas del servicio y de PostgreSQL en Railway	Pestaña de métricas del servicio
Registros de la aplicación	Salida estándar del contenedor en Railway (Spring Security en WARN o INFO, sin SQL en producción)	Pestaña de registros; filtrar por texto
Estado de los despliegues	Railway, Netlify y GitHub Actions	Pestañas de despliegues y de ejecuciones
Actividad de la SPA	Registros de despliegue y analítica de Netlify	Panel del sitio

El indicador de correo está deshabilitado en el healthcheck a propósito (`management.health.mail.enabled=false`): un fallo del proveedor de correo no debe tumbar un despliegue sano. Por eso un `UP` no garantiza que el correo funcione; esa comprobación está en la lista 25.4.5.

25.6.2. Objetivos de nivel de servicio propuestos

Indicador (SLI)	Objetivo (SLO)	Cómo medirlo
Disponibilidad del API	≥ 99.5 % mensual	Consulta externa de `/actuator/health` cada minuto
Disponibilidad de la SPA	≥ 99.5 % mensual	Consulta externa de `/auth/sign-in`
Latencia p95 de lecturas (productos, alertas)	< 1 s	Métricas de Micrometer (sección 29)
Generación de DPI y fotos, p95	< 10 s (RNF-02)	Métrica propia por tipo de documento
Errores 5xx	< 1 % de las peticiones	Métricas HTTP o registros
Entrega del código 2FA	< 60 s en el 95 % de los casos	Seguimiento de Brevo
Despliegues exitosos	≥ 95 % de los push a `main`	Historial de GitHub Actions

25.6.3. Alertas recomendadas

Señal	Umbral	Severidad	Acción inicial
Healthcheck DOWN	2 consultas seguidas (≈ 2 min)	SEV1	Registros y rollback (25.5)
Tasa de 5xx	> 5 % durante 5 min	SEV2	Registros; revisar el despliegue reciente
Memoria del contenedor	> 900 MiB durante 10 min	SEV2	Riesgo de reinicio por OutOfMemory; revisar generación de documentos
CPU	> 85 % durante 15 min	SEV3	Revisar picos de procesamiento de imágenes
Fallos de despliegue	Cualquiera en `main`	SEV3	Registros del build y del arranque
Errores de Brevo	Líneas «Brevo respondió …» repetidas	SEV3	Clave, remitente y cuota (25.8.3)
Picos de 429 en el inicio de sesión	> 20 por hora	SEV3	Posible ataque de fuerza bruta; revisar registros
Espacio del volumen de PostgreSQL	> 80 %	SEV3	Limpiar o ampliar el volumen

Implementación sugerida sin costo: un monitor externo (UptimeRobot o Better Stack) con las dos consultas de disponibilidad y notificaciones a correo o chat, más las notificaciones de despliegue de Railway y GitHub.

25.6.4. Qué buscar en los registros

Texto en el registro	Significado
`Error no controlado`	Excepción inesperada que produjo un 500; incluye la traza
`Violación de integridad de datos`	Un conflicto de la base (unicidad o clave foránea) que devolvió 409
`Brevo respondió <código>: <cuerpo>`	El proveedor de correo rechazó el envío; el cuerpo explica el motivo (clave inválida, remitente no verificado, cuota)

Texto en el registro	Significado
`Recuperación de contraseña no enviada a un usuario existente`	Espera activa o correo caído; el usuario recibió la respuesta genérica
`Recibo de la venta N no enviado`	Límite por hora alcanzado o error de correo; la venta sí quedó registrada
`El usuario 'admin' conserva la contraseña sembrada`	Advertencia de seguridad vigente: cambiarla
`Migration checksum mismatch` o `Validate failed`	Flyway detectó una migración editada o pendiente; el arranque falla
`Schema-validation: missing column`	Una entidad espera una columna que no existe; migración sin aplicar
`HikariPool-1 - Connection is not available`	Se agotó el grupo de conexiones o la base no responde

25.6.5. Rutinas de revisión

Frecuencia	Revisión
Diaria	Estado del health, últimos errores en el registro, correo de resumen de alertas externas
Semanal	Respaldos generados y su tamaño; consumo de memoria y CPU; alertas de stock sin atender; despliegues de la semana
Mensual	Restaurar un respaldo en una instancia de prueba y comparar conteos; revisar cuentas y roles; actualizar dependencias; revisar la cuota de Brevo
Semestral	Rotar secretos (25.9.3); revisar este Runbook con un ejercicio de rollback

25.7. Respaldo y recuperación

25.7.1. Objetivos y alcance

El único dato persistente del sistema es la base de datos PostgreSQL: las fotos de los clientes no se guardan, el código vive en GitHub y la configuración en variables de entorno. Se adopta la estrategia del enunciado (sección 5.1) con estos objetivos: **RPO ≤ 30 minutos** y **RTO ≤ 2 horas** para un desastre de datos.

Activo	Dónde vive	Método de protección	Frecuencia
Base de datos	PostgreSQL de Railway	Respaldo completo, incrementales y registro de transacciones (ver abajo)	Semanal, diaria y cada 15 a 30 min
Código	GitHub (dos repositorios)	Historial completo; clon local del mantenedor	Continua
Configuración y secretos	Variables de Railway, Netlify y GitHub	Copia de la lista de variables (sin valores) en este Runbook y valores en el gestor de contraseñas	En cada cambio
Plantillas de correo y recursos	Dentro del repositorio	Igual que el código	Continua

25.7.2. Estrategia de respaldo

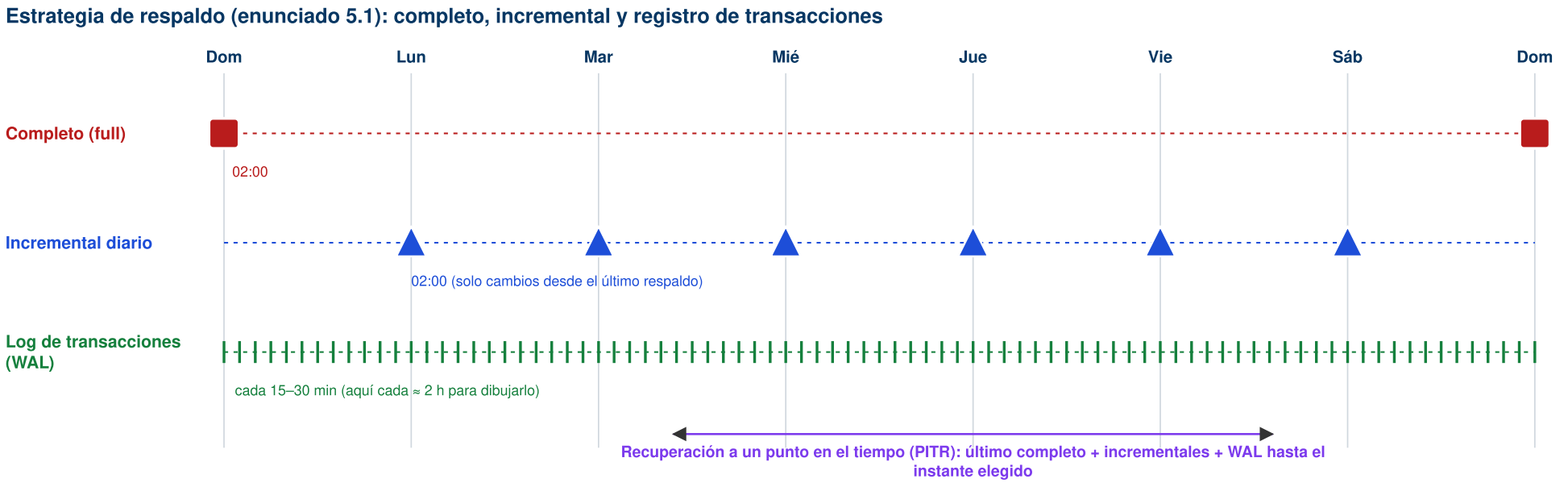


Figura 25.5 — Estrategia de respaldo: completo semanal, incremental diario y registro de transacciones para recuperación a un punto en el tiempo

25.7.3. Estado actual y plan de implementación

Estado actual. Railway ofrece respaldos de los volúmenes y de las bases según el plan contratado; antes de darlos por válidos hay que confirmar en la consola de PostgreSQL qué frecuencia y qué retención incluye el plan, y probar una restauración. El PostgreSQL administrado de Railway no expone el archivado del registro de transacciones (WAL), por lo que la recuperación a un punto exacto en el tiempo no está disponible allí.

Plan propuesto (por fases).

1. **Fase A, de inmediato:** respaldo lógico nocturno con `pg_dump` ejecutado por GitHub Actions, cifrado y enviado a un bucket compatible con S3 con retención de 14 días y completos semanales de 4 semanas. Cubre el RPO de un día.
2. **Fase B, cuando el negocio lo requiera:** mover la base a un servicio administrado con PITR nativo (Amazon RDS, Neon u otro equivalente) para alcanzar el RPO de 30 minutos, o autoadministrar PostgreSQL con WAL-G. La decisión se toma con los criterios de la sección 26.8.
3. **Fase C:** restauración de prueba mensual automatizada y registro de su duración.

```
# PROPUESTO - .github/workflows/db-backup.yml (no existe todavía en el repositorio)
name: Respaldo nocturno de la base de datos
on:
  schedule:
    - cron: '0 8 * * *'        # 02:00 en Guatemala (UTC-6)
  workflow_dispatch:
jobs:
  dump:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Instalar cliente de PostgreSQL 17
        run: |
          sudo apt-get update && sudo apt-get install -y postgresql-client-17
      - name: pg_dump en formato custom, cifrado
        env:
          BACKUP_DATABASE_URL: ${{ secrets.BACKUP_DATABASE_URL }}   # usuario de solo lectura
          BACKUP_PASSPHRASE: ${{ secrets.BACKUP_PASSPHRASE }}
        run: |
          FILE="inventory_$(date -u +%Y%m%d_%H%M).dump.gpg"
          pg_dump --format=custom --no-owner --no-privileges "$BACKUP_DATABASE_URL" \
            | gpg --batch --symmetric --cipher-algo AES256 --passphrase "$BACKUP_PASSPHRASE" -o "$FILE"
          echo "FILE=$FILE" >> "$GITHUB_ENV"
      - name: Subir a un bucket compatible con S3
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.BACKUP_S3_KEY }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.BACKUP_S3_SECRET }}
        run: aws s3 cp "$FILE" "s3://${{ vars.BACKUP_BUCKET }}/daily/$FILE" --endpoint-url "${{
vars.BACKUP_S3_ENDPOINT }}"
```

25.7.4. Procedimiento de restauración

Paso	Acción
1	Declarar el incidente (SEV1) y congelar los despliegues; informar al responsable del negocio
2	Elegir el respaldo: el más reciente anterior al daño; anotar su fecha y hora
3	Crear una instancia de PostgreSQL 17 vacía (un servicio nuevo en Railway o `docker run postgres:17-alpine` para ensayar)
4	Descifrar y restaurar con `pg_restore` (comandos abajo)
5	Verificar `flyway_schema_history`, los conteos y la fecha de la última venta; comparar con lo esperado
6	Apuntar el API a la base restaurada cambiando `DB_URL`, `DB_USER` y `DB_PASSWORD`, y redesplegar
7	Ejecutar la lista 25.4.5, informar la pérdida de datos (ventas entre el respaldo y el incidente) y registrar el post-mortem

```
# 1) Crear una instancia vacía de PostgreSQL 17 (servicio nuevo en Railway o docker run local)
# 2) Descifrar y restaurar
gpg --batch --decrypt --passphrase "$BACKUP_PASSPHRASE" inventory_YYYYMMDD_HHMM.dump.gpg > inventory.dump
pg_restore --clean --if-exists --no-owner --no-privileges --dbname "$RESTORE_DATABASE_URL" inventory.dump

# 3) Verificar que el historial de Flyway y los datos clave están completos
psql "$RESTORE_DATABASE_URL" -c "SELECT version, description, success FROM flyway_schema_history ORDER BY
installed_rank;"
psql "$RESTORE_DATABASE_URL" -c "SELECT (SELECT count(*) FROM users) AS usuarios, (SELECT count(*) FROM
product) AS productos, (SELECT count(*) FROM sales) AS ventas, (SELECT max(date) FROM sales) AS
ultima_venta;"
```

```
# 4) Apuntar el API a la base restaurada (variable DB_URL del servicio) y redesplegar
# 5) Comprobar /actuator/health, iniciar sesión y registrar una venta de prueba
```

Importante: una restauración que no se ha probado no es un respaldo. Documente en cada prueba mensual la fecha, la duración y los conteos obtenidos; si la restauración falla, es un incidente SEV2 aunque no haya pérdida real.

25.8. Resolución de incidentes

25.8.1. Severidades y tiempos de respuesta

Nivel	Definición	Ejemplo	Respuesta	Comunicación
SEV1	Servicio caído o datos en riesgo; el negocio no puede vender	Healthcheck en DOWN; corrupción de datos	Inmediata, máximo 15 min	Aviso al responsable al inicio y cada hora
SEV2	Función crítica degradada o rota para parte de los usuarios	No llegan los códigos 2FA; la generación de DPI falla	Máximo 1 hora	Aviso al responsable al inicio y al resolver
SEV3	Defecto menor o riesgo sin impacto inmediato	Reportes lentos; advertencia de seguridad	Siguiente día hábil	Registro en el tablero

25.8.2. Proceso de gestión

El proceso completo está diagramado en el BPMN-09 (sección 27). En resumen: detectar y registrar; clasificar la severidad; diagnosticar con el health, los registros y los despliegues recientes; mitigar (rollback si coincide con un despliegue, esperar si la causa es del proveedor, corregir si es de configuración o de datos); verificar con la prueba de humo; comunicar la resolución; y cerrar con un post-mortem sin culpables dentro de los 5 días hábiles.

25.8.3. Catálogo de incidentes conocidos

Síntoma	Causa probable	Diagnóstico	Solución
Todos los usuarios son enviados al inicio de sesión	Sesión vencida (3600 s por defecto) o `JWT_SECRET` cambiado	Fecha de vencimiento de la cookie `jwt`; ¿se rotó el secreto?	Iniciar sesión de nuevo; ajustar `JWT_EXPIRATION` si se desea una sesión más larga
`403 Invalid CORS request` o fallos de preflight	`CORS_ALLOWED_ORIGINS` no coincide con el origen exacto del navegador	Comparar la variable con la URL de Netlify (https, sin barra final)	Corregir la variable y reiniciar el servicio
`429 Demasiados intentos fallidos`	Bloqueo por 5 fallos en 15 min (estado en memoria)	Registros de inicio de sesión; ¿es el usuario legítimo?	Esperar 15 min o reiniciar el servicio (borra el contador) tras descartar un ataque
502 o 504 al generar DPI o fotos	El proxy de Netlify corta a los 26 s; fotos pesadas o CPU saturada	Tamaño de las fotos, CPU y memoria en Railway	Reintentar con fotos más livianas (la SPA reduce a 2560 px); ampliar vCPU
El servicio se reinicia solo	Heap de 512 MB agotado con varias fotocopias simultáneas (`ExitOnOutOfMemoryError`)	Memoria y reinicios en Railway	Limitar la concurrencia; subir `-Xmx` y la RAM a más de 1 GB
El despliegue falla en el healthcheck	Migración fallida, variable faltante o base no lista	Registros de build y de arranque	Corregir y volver a desplegar; la versión anterior sigue sirviendo
`JWT_SECRET debe tener al menos 32 caracteres`	Secreto corto o ausente	Registro de arranque	Generar uno con `openssl rand -base64 48`
`IllegalStateException … conserva la contraseña sembrada`	`SECURITY_FAIL_ON_DEFAULT_ADMIN_PASSWORD=true` con `admin` sin cambiar	Registro de arranque	Poner la variable en `false` un momento, cambiar la contraseña y volver a `true`
`Migration checksum mismatch`	Se editó una migración ya aplicada	Comparar el archivo con el historial de Git	Restaurar el archivo original; nunca editar (25.5.4)

Síntoma	Causa probable	Diagnóstico	Solución
No llegan los correos (2FA, recibos, pedidos)	`BREVO_API_KEY` inválida, remitente sin verificar o cuota diaria agotada	Registro «Brevo respondió …»	Corregir la clave o el remitente; si hay usuarios bloqueados por 2FA, un Administrador puede editar la cuenta sin correo, lo que desactiva su 2FA
Recibo con estado `NO_ENVIADO`	Tope por hora o error de correo	Registro «Recibo de la venta N no enviado»	La venta ya existe; reenviar más tarde
«Ventas de hoy» omite las ventas de la tarde	La JVM no usa la zona America/Guatemala	Opción `-Duser.timezone` en el `ENTRYPOINT`	Restaurar el `Dockerfile` original
Pantalla en blanco o 404 al recargar una ruta	Falta la regla `/* /index.html 200` en `_redirects`	Archivo `_redirects` publicado	Restaurar la regla y republicar
El CI falla en «Verificar el proxy a Railway»	Marcador `TU-BACKEND` o regla `/api/*` mal formada	Salida del paso del CI	Poner el dominio real del API
`HikariPool … Connection is not available`	Conexiones agotadas o base caída	`pg_stat_activity` y estado de PostgreSQL	Reiniciar el servicio; revisar el volumen y el pool
Un producto «no se puede vender»	Está dado de baja (inactivo)	Estado del producto	Un Administrador lo reactiva

25.8.4. Consultas SQL de diagnóstico

Son de solo lectura y se probaron contra el esquema V11. Ejecútelas con un usuario de lectura.

```
SELECT product_id, name, stock FROM product WHERE stock < 0;
SELECT product_id, count(*) FROM stock_alert WHERE status = 'PENDIENTE' GROUP BY product_id HAVING count(*)
> 1;
SELECT p.product_id, p.name, p.stock, p.min_stock FROM product p WHERE p.active AND p.stock <= p.min_stock
AND NOT EXISTS (SELECT 1 FROM stock_alert a WHERE a.product_id = p.product_id AND a.status = 'PENDIENTE');
SELECT count(*) AS ventas, coalesce(sum(total), 0) AS total FROM sales WHERE date >= date_trunc('day',
now() AT TIME ZONE 'America/Guatemala');
SELECT r.name AS rol, u.active, u.two_factor_enabled, count(*) FROM users u JOIN role r ON r.id = u.role_id
GROUP BY 1, 2, 3 ORDER BY 1;
SELECT type, count(*) FILTER (WHERE NOT consumed AND expires_at > now()) AS vigentes, count(*) FILTER
(WHERE consumed OR expires_at <= now()) AS usados_o_vencidos FROM verification_code GROUP BY type;
SELECT state, count(*) FROM pg_stat_activity WHERE datname = current_database() GROUP BY state;
SELECT pg_size_pretty(pg_database_size(current_database())) AS tamano_base;
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS tamano FROM
pg_catalog.pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC LIMIT 5;
```

25.8.5. Escalamiento

Nivel	Quién	Cuándo se involucra
1	Operador (administrador técnico)	Todo incidente, de forma inmediata
2	Responsable del negocio (dueño de Multiservicios Wilson)	SEV1 y SEV2: decisiones sobre continuidad del servicio y comunicación a clientes
3	Soporte del proveedor (Railway, Netlify, Brevo, GitHub)	Cuando el diagnóstico apunta a la plataforma; usar su página de estado y su canal de soporte

25.8.6. Plantilla de post-mortem

Campo	Contenido
Resumen	Qué pasó, en dos frases
Impacto	Duración, usuarios o funciones afectadas, datos perdidos
Línea de tiempo	Detección, diagnóstico, mitigación y resolución, con horas
Causa raíz	Por qué ocurrió (técnica y de proceso), sin culpables
Qué funcionó y qué no	Alertas, Runbook, herramientas
Acciones	Responsable y fecha límite de cada mejora; actualizar este Runbook si corresponde

25.9. Operación rutinaria

25.9.1. Alta de personal y cambio de contraseñas

Solo un Administrador crea cuentas (no hay registro público): pantalla Usuarios, nombre, usuario, contraseña (de 8 a 72 caracteres con letra y número), rol y correo; con correo la cuenta nace con 2FA activo. Para dar de baja a una persona se desactiva su cuenta (conserva el historial de ventas). Cada usuario cambia su contraseña y su correo en Perfil, pestaña de seguridad; quien olvide su contraseña usa «Olvidé mi contraseña».

25.9.2. Cambio de la contraseña de `admin` sembrada

Después del primer despliegue: crear un Administrador nuevo; iniciar sesión con él; desactivar `admin`; confirmar que no queda ninguna cuenta con la contraseña sembrada; activar `SECURITY_FAIL_ON_DEFAULT_ADMIN_PASSWORD=true`.

25.9.3. Rotación de secretos

Secreto	Procedimiento	Efecto
`JWT_SECRET`	Generar uno nuevo, actualizar la variable en Railway y reiniciar	Cierra todas las sesiones y anula los códigos y enlaces de recuperación pendientes (se firman con esa clave); hacerlo en horario sin ventas
`BREVO_API_KEY`	Crear una clave nueva en Brevo, actualizar la variable y revocar la anterior	Sin efecto para los usuarios si se hace en ese orden
`RAILWAY_TOKEN`, `NETLIFY_AUTH_TOKEN`	Generar nuevos, actualizar los secretos de GitHub y revocar los anteriores	Ninguno; confirmar con un despliegue de prueba
`DB_PASSWORD`	Rotar en PostgreSQL y en la variable del servicio dentro de la misma ventana	Breve reinicio del API

25.9.4. Otras operaciones

- **Cambiar el dominio del API:** actualizar `public/_redirects` y desplegar el frontend; después fijar `CORS_ALLOWED_ORIGINS` y `FRONTEND_URL` si cambió el del sitio.
- **Ampliar recursos:** subir los límites de vCPU y RAM del servicio en Railway; si se sube la RAM, subir también `-Xmx` en el `ENTRYPOINT` del `Dockerfile` (hoy fijo en 512 m a propósito, para que el consumo no dependa del límite configurado).
- **Cambiar el stock mínimo por defecto:** `inventory.stock-minimo` en `application.properties` (solo afecta a los productos nuevos sin valor propio).
- **Ejecutar las pruebas en una máquina con JDK 25:** `JAVA_HOME=<ruta del JDK 25> ./mvnw verify` con las variables `DB_URL`, `DB_PASSWORD` y `JWT_SECRET` apuntando a un PostgreSQL de prueba.

25.10. Lista de endurecimiento de seguridad

Control	Estado	Cómo verificarlo
HTTPS en la SPA y en el API	Vigente	Ambos dominios responden solo por HTTPS; la cookie `jwt` lleva `Secure`
`JWT_SECRET` único, de más de 32 bytes y fuera del repositorio	Vigente	La aplicación no arranca con uno corto; revisar el historial de Git
CORS limitado a la URL exacta del sitio	Vigente	`CORS_ALLOWED_ORIGINS` sin comodines
Environment `production` restringido a `main` y token de Railway de alcance mínimo	Vigente	Configuración del repositorio y del token
Contraseña sembrada de `admin` reemplazada y `SECURITY_FAIL_ON_DEFAULT_ADMIN_PASSWORD=true`	Verificar	Usuarios y variables de Railway
2FA activo en todas las cuentas de Administrador	Verificar	Pantalla de Usuarios y consulta de cuentas (25.8.4)
Actuator limitado a `/actuator/health`	Vigente	`/actuator/env` y similares responden 401
Protección de ramas `main` y `develop` (revisión y CI obligatorios)	Pendiente	Configuración de GitHub
Escaneo de dependencias, secretos y código (Dependabot, CodeQL, Trivy)	Pendiente	Sección 29
Cabeceras HSTS y Content-Security-Policy en la SPA	Pendiente	Archivo `_headers` y analizador de cabeceras
Respaldos cifrados y restauración probada	Pendiente	25.7
Revocación de sesiones al cerrar sesión	Pendiente	Migración V13 (sección 28)

26. Documentación para Escalabilidad Futura

26.1. Línea base actual y límites medidos

Escalar bien empieza por saber dónde se está. La tabla resume lo que el sistema soporta hoy y de dónde sale cada dato; las cifras de carga son supuestos de planificación que deben confirmarse con pruebas de carga.

Dimensión	Valor actual	Fuente
Instancias del API	1 contenedor en Railway (límite de 2 vCPU y 1 GB; se cobra lo realmente usado)	README del backend
Memoria de la JVM	Heap fijo de 512 MB con `SerialGC`; ≈ 230 MiB en reposo y ≈ 720 MiB de pico con dos fotocopias simultáneas de fotos de 12 MP (1 vCPU)	`Dockerfile` y README
Tiempo de un documento	Sin reducir las fotos se midieron ≈ 13 s por documento con 1 vCPU; el sistema reduce a 2560 px (en el navegador y en el servidor)	Comentarios de `ImageCropServiceImpl`
Concurrencia de documentos	Dos fotocopias de fotos de 12 MP o tres de fotos de 2560 px a la vez; con un heap de 384 MB se queda sin memoria	`Dockerfile`
Límite del proxy	26 s por petición, incluida la subida	README del frontend
Conexiones a la base	Grupo HikariCP por defecto (10); con `SPRING_PROFILES_ACTIVE=prod` la primera conexión se reintenta hasta 60 s	`application-prod.properties`
Estado en memoria	Dos contadores: bloqueo de inicio de sesión y recibos por hora	`LoginAttemptService`, `ReceiptEmailLimiter`
Sesión	JWT sin estado: no exige afinidad de sesión	`SecurityConfig`
Migraciones	Flyway al arrancar, con bloqueo en la base: dos instancias no migran a la vez	Flyway
Carga esperada (supuesto)	1 a 3 usuarios, 50 a 150 ventas y hasta 40 documentos por día	Operación de una sola sucursal
Volumen de datos (supuesto)	≈ 55 mil ventas por año con 150 al día: tamaño trivial para PostgreSQL	Cálculo

La conclusión es que el sistema no tiene hoy un problema de volumen sino de **estado compartido** (para poder tener más de una instancia), **concurrencia de procesamiento de imágenes** (CPU y memoria) y **observabilidad** (para saber cuándo escalar).

26.2. Cuellos de botella y deuda técnica identificados

ID	Hallazgo y evidencia	Impacto al escalar	Mitigación	Referencia
DT-01	Los contadores de bloqueo de inicio de sesión y de recibos por hora viven en memoria de un solo nodo	Con 2 o más instancias los límites se multiplican y el bloqueo se evade con peticiones repartidas	Mover a Redis o a tablas	V13; Redis (29)
DT-02	El procesamiento de imágenes es síncrono dentro de la petición y consume CPU y memoria	Pocas fotocopias simultáneas agotan el heap; el proxy corta a los 26 s	Cola con trabajadores; semáforo de concurrencia	Cola (29)
DT-03	El cierre de sesión borra la cookie pero no invalida el JWT	Un token copiado sigue válido hasta vencer	Versión de token por usuario	V13
DT-04	La marca «leída» de una notificación es global	Una lectura de un rol oculta la notificación a otro	Tabla de lecturas por usuario	V16
DT-05	Los reportes cargan todas las ventas del periodo con detalle (`findForReport`)	Memoria y tiempo crecen con el volumen; el rango máximo de 366 días lo acota	Agregaciones en SQL, caché y réplica de lectura	Índices en V12
DT-06	No hay índices sobre `sales.date`, claves foráneas de `sale_detail` ni `stock_alert.status`	Reportes y dashboard hacen lecturas completas	Índices	V12
DT-07	La unicidad de la alerta pendiente se valida solo en el servicio	Una carrera podría duplicar alertas	Índice parcial único	V12
DT-08	El recibo por correo se envía dentro de la petición de venta (hasta 10 s de espera)	Latencia de la venta cuando el proveedor de correo es lento	Bandeja de salida con reintentos y un trabajador	V13 (`email_outbox`)
DT-09	Las fechas de negocio no llevan zona horaria y dependen de `-Duser.timezone`	Riesgo al mover la aplicación a otra región o JVM	`timestamptz`	Migración opcional

ID	Hallazgo y evidencia	Impacto al escalar	Mitigación	Referencia
DT-10	Solo existe el healthcheck como observabilidad	No se detectan degradaciones ni se mide la capacidad	Métricas, trazas y alertas	Micrometer (29)
DT-11	No hay un entorno de pruebas previo a producción	Los defectos se descubren en producción	Entorno `staging`	Terraform (29)
DT-12	El cliente de la venta es un texto libre	No se puede analizar la recompra ni enviar comprobantes repetidos	Entidad `customer`	V15
DT-13	El stock vive en `product` (una sola sucursal)	Una segunda sucursal exige rediseño	Tabla de stock por sucursal	V17
DT-14	Los listados de productos, usuarios, proveedores y notificaciones no se paginan	Respuestas grandes con muchos registros	Paginación con `Pageable`	Código
DT-15	Dependencias con versiones fijas (por ejemplo JJWT 0.11.5 y POI 5.3.0)	Vulnerabilidades sin parchar con el tiempo	Actualización automatizada y escaneo	Dependabot (29)

26.3. Estrategia de escalado por componente

Componente	Cómo escala hoy	Siguiente paso	Criterio
SPA y CDN	Archivos estáticos en la CDN de Netlify: escala sola	Sin cambios; entornos separados	—
API	Vertical (más vCPU y RAM); sin estado salvo los dos contadores	Dos o más réplicas detrás de un balanceador, con DT-01 resuelto	CPU sostenida por encima del 70 %
Procesamiento de imágenes	Vertical	Trabajadores dedicados con cola; el API responde 202 y la SPA consulta el estado	p95 por encima de 8 s o errores 502
Base de datos	Vertical; sin índices de apoyo	Índices (V12), réplica de lectura para reportes y PITR	Conexiones, p95 de reportes y tamaño
Correo	Síncrono con tope de 300 mensajes diarios del plan gratuito de Brevo	Bandeja de salida con reintentos; plan de pago o Amazon SES	Más del 70 % de la cuota
Reportes	Cálculo en cada consulta	Agregaciones SQL, caché con vigencia y réplica	p95 por encima de 3 s
Respaldos y objetos	Ninguno propio	Bucket compatible con S3 (copias cifradas y reportes con URL firmada)	Desde la Fase A del plan 25.7

26.4. Arquitectura objetivo y equivalencias entre proveedores

La arquitectura objetivo conserva el monolito modular (sección 24.2) y agrega solo lo que una métrica justifique: estado compartido, cola de trabajos, réplica de lectura, observabilidad e infraestructura como código. No incluye microservicios ni orquestadores como Kubernetes: la complejidad operativa superaría el beneficio para el tamaño del negocio.

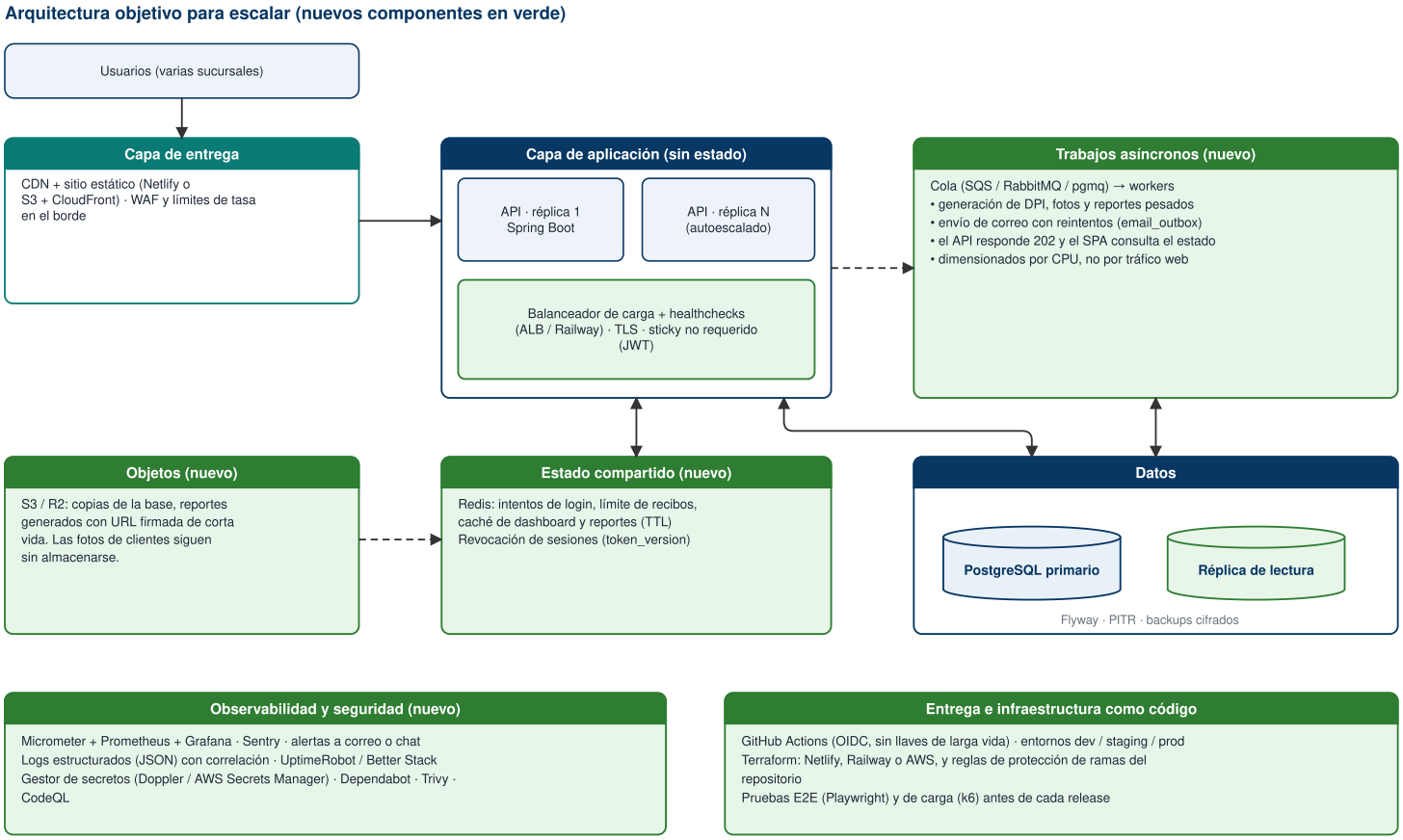


Figura 26.1 — Arquitectura objetivo para escalar. Los componentes en verde son nuevos respecto de la producción actual

Equivalencias para migrar a un proveedor de nube de propósito general

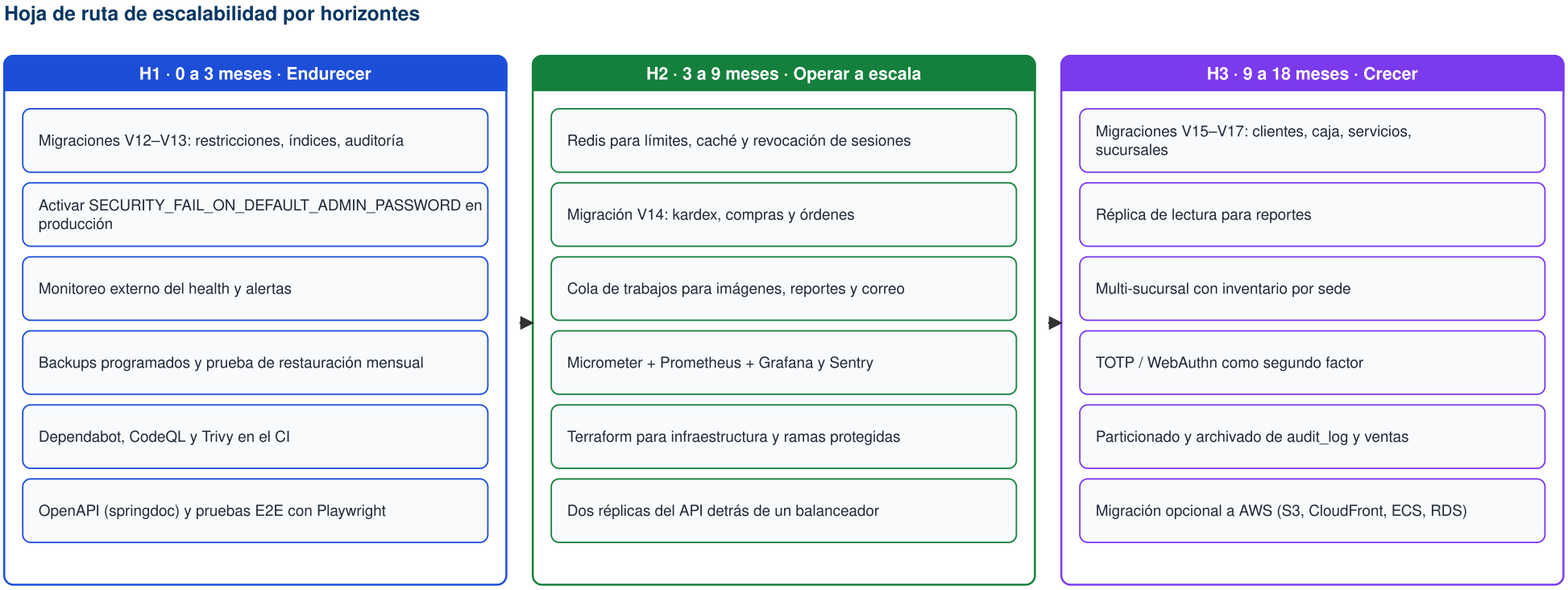
El enunciado menciona AWS (S3, EC2 y RDS), Azure y GCP. La decisión de usar plataformas como servicio (Railway y Netlify) se tomó por costo y simplicidad operativa; la tabla muestra a qué servicio corresponde cada pieza si el crecimiento o una exigencia institucional pidieran migrar.

Función	Hoy	AWS	Azure	GCP
SPA y CDN	Netlify	S3 + CloudFront	Static Web Apps o Blob Storage + Front Door	Cloud Storage + Cloud CDN, o Firebase Hosting
API en contenedor	Railway	ECS Fargate o App Runner, con ALB (o EC2 con Docker)	Container Apps o App Service	Cloud Run
PostgreSQL	Railway	RDS for PostgreSQL (Multi-AZ, PITR)	Database for PostgreSQL Flexible Server	Cloud SQL for PostgreSQL
Cola de trabajos	—	SQS	Service Bus	Pub/Sub o Cloud Tasks
Caché y estado compartido	—	ElastiCache for Redis	Cache for Redis	Memorystore
Secretos	Variables de entorno	Secrets Manager o SSM Parameter Store	Key Vault	Secret Manager
Correo	Brevo	SES (o seguir con Brevo)	Communication Services (o Brevo)	Brevo o SendGrid
Observabilidad	Registros de Railway	CloudWatch y Managed Grafana	Azure Monitor	Cloud Monitoring
Respaldos	—	RDS snapshots + AWS Backup + S3	Azure Backup	Backups de Cloud SQL + Cloud Storage
IaC	—	Terraform o CDK	Terraform o Bicep	Terraform

Ruta de migración a AWS (orden sugerido).

1. Escribir la infraestructura con Terraform en un entorno `staging` y comprobar con el mismo pipeline.
2. Crear RDS PostgreSQL 17, restaurar un `pg_dump` reciente y comparar conteos y `flyway_schema_history`.
3. Publicar la imagen del `Dockerfile` en ECR y crear el servicio en ECS Fargate con los mismos healthcheck, variables y zona horaria.
4. Crear el bucket S3 y la distribución de CloudFront para la SPA; replicar las reglas de `_redirects` como comportamientos de CloudFront hacia el balanceador.
5. Ajustar `CORS_ALLOWED_ORIGINS`, `FRONTEND_URL` y el workflow de despliegue (credenciales por OIDC).
6. Ensayar el corte: congelar escrituras, tomar el último dump, restaurar, cambiar DNS. Ventana estimada de 15 a 30 minutos con un volumen de datos pequeño.
7. Mantener el entorno anterior 7 días como plan de retorno y luego darlo de baja.

26.5. Hoja de ruta por horizontes



Cada horizonte se activa por métricas, no por fecha: ver los criterios de activación de la sección 26.8.

Figura 26.2 — Hoja de ruta de escalabilidad: endurecer, operar a escala y crecer

Horizonte	Migraciones y datos	Tecnología y operación
H1 · 0 a 3 meses	V12 (integridad e índices) y V13 (auditoría y seguridad)	Monitoreo externo, respaldo nocturno con `pg_dump`, Dependabot, CodeQL y Trivy, OpenAPI y pruebas E2E
H2 · 3 a 9 meses	V14 (kardex y compras)	Redis, cola de trabajos, Micrometer con Prometheus y Grafana, Sentry, Terraform, entorno `staging` y dos réplicas del API
H3 · 9 a 18 meses	V15 a V18 (clientes, caja, servicios, sucursales y contracción)	Réplica de lectura, multi-sucursal, TOTP o WebAuthn, particionado de bitácoras y migración opcional a AWS

26.6. Evolución funcional: multi-sucursal y nuevos canales

- **Multi-sucursal:** el stock pasa a la tabla `inventory_stock` por sucursal, las ventas y las cajas llevan `branch_id`, los usuarios se asignan a una sucursal y los reportes se filtran por sucursal. Aparece un proceso nuevo, el traslado de mercancía entre sucursales (entrada y salida enlazadas en el kardex). Se recomienda un esquema compartido con `branch_id` y, si el sistema se ofreciera a otros negocios como servicio, evaluar la seguridad por filas de PostgreSQL (Row Level Security) para aislar cada negocio.
- **PWA instalable:** convertir la SPA en una aplicación instalable con caché del catálogo para que el escáner y la venta sigan funcionando con conexión intermitente.
- **Facturación y cobros:** a evaluar con el responsable del negocio, según la obligación tributaria que aplique (por ejemplo, la factura electrónica en línea de la SAT en Guatemala) y la pasarela de pago elegida; ambas se modelan con las tablas de clientes, métodos de pago y caja de la migración V15.
- **API para integraciones:** documentación OpenAPI y claves de API de alcance limitado para integrar proveedores o tiendas en línea.

26.7. Capacidad y dimensionamiento orientativo

Escenario	Supuesto de carga	Recursos sugeridos
Actual (1 sucursal)	1 a 3 usuarios, 50 a 150 ventas y hasta 40 documentos por día	1 instancia de 1 a 2 vCPU y 1 GB; PostgreSQL pequeño con respaldos
3 sucursales	10 usuarios, 500 ventas y 150 documentos por día	2 instancias del API (2 vCPU y 1 GB), 1 trabajador de imágenes (2 vCPU y 2 GB), Redis y réplica de lectura
10 sucursales o servicio para varios negocios	50 usuarios, 3000 ventas y 800 documentos por día	Autoescalado del API de 3 a 6 instancias, 2 a 4 trabajadores de imágenes, PostgreSQL Multi-AZ y particionado de bitácoras

> **Nota:** son estimaciones de planificación. Antes de activar un horizonte se debe medir con una prueba de carga (por ejemplo con k6) reproduciendo el flujo de venta, la generación de documentos y la descarga de reportes.

26.8. Criterios de activación

Cada paso de escalado se activa por una métrica sostenida, no por una fecha.

Métrica	Umbral	Acción
CPU del API	> 70 % durante 7 días	Agregar una réplica (requiere DT-01 resuelto)
p95 de generación de documentos	> 8 s, o errores 502	Mover el procesamiento a trabajadores con cola
Memoria del API	> 85 % o más de un reinicio por falta de memoria a la semana	Subir RAM y `-Xmx`, o limitar la concurrencia de documentos
Conexiones a la base de datos	> 70 % del grupo	Ajustar el pool o agregar PgBouncer
p95 de reportes	> 3 s, o más de 1 millón de ventas	Réplica de lectura, agregaciones o vistas materializadas
Filas de `audit_log` o `sales`	> 10 millones	Particionar por mes y archivar
Cuota de correo	> 70 % del plan	Plan de pago o Amazon SES, con bandeja de salida
Segunda sucursal confirmada	Decisión del negocio	Ejecutar V15 a V17
Disponibilidad medida	< 99.5 % en el mes	Pasar a dos réplicas y alertas (H2)
Exigencia de auditoría o incidente de seguridad	Cualquiera	Bitácora de auditoría y segundo factor adicional (H1)

27. Procesos Identificados y Diagramados (BPMN)

27.1. Inventario de procesos

En las Fases 1 y 2 se identificaron y priorizaron los procesos del negocio y se modelaron en BPMN las situaciones actual (AS-IS) y propuesta (TO-BE). Esta sección actualiza esa base con lo que el sistema ejecuta hoy, formaliza los procesos técnicos y de seguridad que aparecieron durante la construcción y deja modelados los procesos que el negocio podría automatizar a continuación. Todos los diagramas BPMN de esta sección se dibujaron a partir del código fuente: cada tarea corresponde a un componente real del sistema, salvo los tres procesos marcados como «propuesto».

ID	Proceso	Tipo	Estado	Diagrama	Requisitos y migraciones
P1	Venta, descuento de stock y alerta automática	Misional	Implementado	BPMN-01	RF-02, RF-03, RF-04, RF-11
P1	Reposición: pedido al proveedor y cierre de la alerta	Misional	Implementado	BPMN-02	RF-04, RF-10
P2	Fotocopia de DPI en PDF o Word	Misional	Implementado	BPMN-03	RF-05
P3 y P4	Fotos tamaño cédula y pasaporte	Misional	Implementado	BPMN-04	RF-09
P5	Inicio de sesión con 2FA	Apoyo y seguridad	Implementado	BPMN-05	RF-01, RF-08
P6	Recuperación de contraseña	Apoyo y seguridad	Implementado	BPMN-06	RF-08
P7	Reportes con exportación	Apoyo	Implementado	BPMN-07	RF-07, RF-13
P8	Entrega continua con GitFlow	Técnico	Implementado	BPMN-08	RNF-05, RNF-06
P9	Gestión de incidentes	Técnico	Implementado	BPMN-09	RNF-06
P10	Compras a proveedores con orden y recepción	Apoyo (Fase 1)	Propuesto	BPMN-F1	Migración V14
P11	Apertura y cierre de caja diario	Apoyo (Fase 1)	Propuesto	BPMN-F2	Migración V15
P12	Registro de servicios (fotocopias y emplasticados)	Misional (excluido en Fase 1)	Propuesto	BPMN-F3	Migración V15

Los procesos estratégicos del mapa de la Fase 1 (gestión general del negocio y relación con los proveedores de telefonía) y la atención al cliente siguen siendo humanos: no tienen reglas que automatizar. Las fotocopias generales y el emplasticado son físicos; el BPMN-F3 solo propone registrarlos para que sus ingresos aparezcan en los reportes.

27.2. Notación y convenciones

Los diagramas usan BPMN 2.0 con carriles (lanes) por actor. Una tarea se colorea según quién la ejecuta; los eventos de fin con borde rojo indican un rechazo o una condición de error; las líneas punteadas son mensajes hacia otro sistema (correo o HTTP). Cada diagrama cabe en una página apaisada y va seguido de su diccionario de actividades.

Leyenda de la notación BPMN usada en este documento



Figura 27.1 — Leyenda de la notación BPMN utilizada

27.3. BPMN-01 · Venta, descuento de stock y alerta automática

Objetivo: registrar una venta de celulares o accesorios descontando el stock de forma atómica y avisar automáticamente cuando un producto llega a su mínimo. **Disparador:** un cliente solicita un producto. **Actores:** Cliente, Encargado o Administrador, Sistema y servicio de correo. **Resultado:** venta registrada, stock actualizado, alerta y notificación creadas si corresponde, y recibo enviado si se indicó un correo.

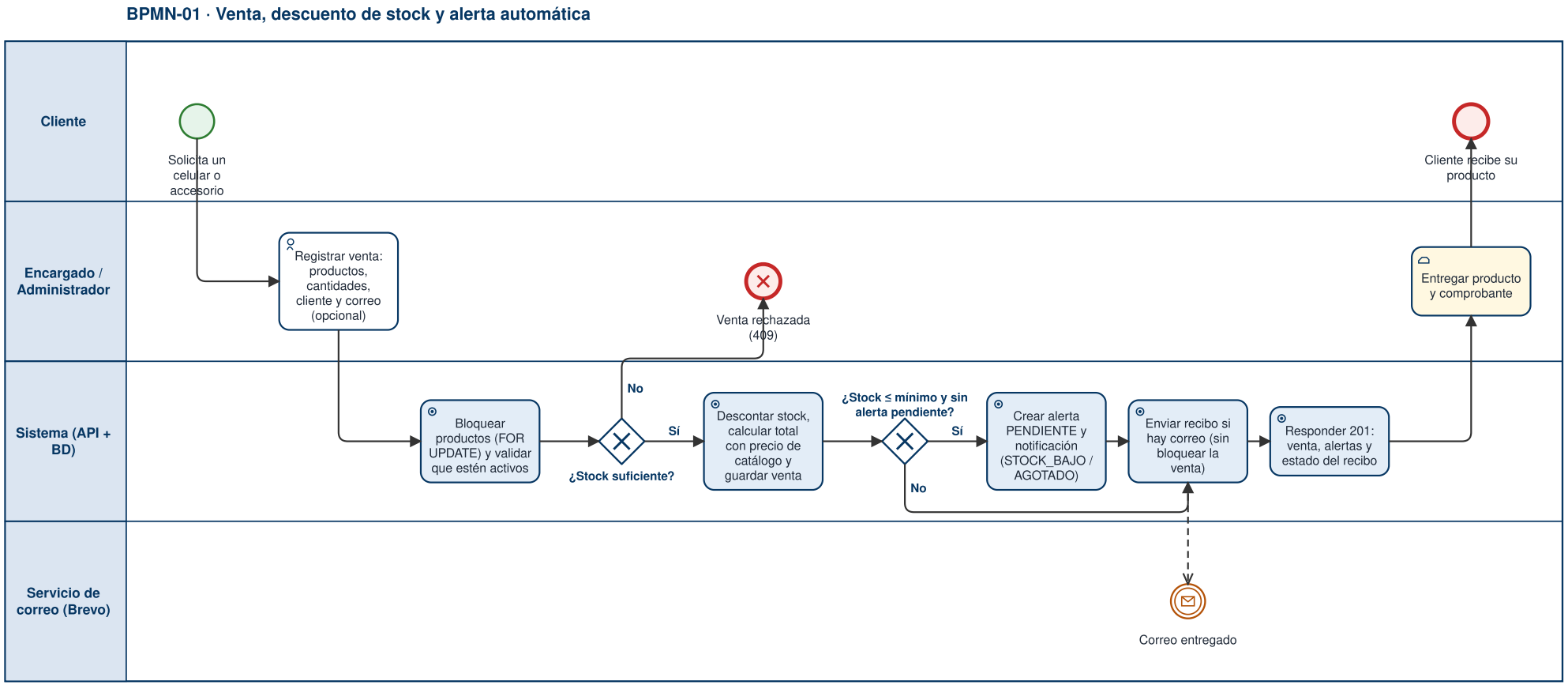


Figura 27.2 — BPMN-01: venta con descuento de stock, alerta automática y recibo por correo

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Registrar la venta	Encargado o Administrador	Productos (búsqueda o escáner), cantidades, cliente y correo opcional	`POST /api/sales`	`Sales.jsx`, HU-20
2	Bloquear y validar productos	Sistema	Líneas de la venta	Filas bloqueadas con `FOR UPDATE` en orden de identificador	RN-03, RN-05
3	¿Stock suficiente?	Sistema	Stock y cantidad pedida	Continuar o 409 y venta completa rechazada	RN-03, HU-21
4	Descontar y guardar	Sistema	Líneas válidas	Venta y detalle con precio de catálogo; stock actualizado	RN-04, HU-22
5	¿Stock en el mínimo y sin alerta pendiente?	Sistema	Stock restante y mínimo del producto	Alerta PENDIENTE y notificación STOCK_BAJO o AGOTADO	RN-01, RN-02, RN-09, HU-23
6	Enviar el recibo	Sistema y Brevo	Correo del cliente	Estado ENVIADO, NO_ENVIADO u OMITIDO	RN-08, HU-48
7	Responder 201	Sistema	Venta confirmada	Venta, alertas generadas y estado del recibo	`SaleResponseDto`
8	Entregar el producto y el comprobante	Encargado	Venta registrada	Cliente atendido	—

- **Excepciones:** producto inactivo, inexistente o con stock insuficiente producen 409 y la transacción se revierte sin descontar nada. Un fallo del correo no afecta a la venta.
- **Reglas de decisión:** una alerta se crea solo si el stock quedó en el mínimo propio del producto o por debajo y no hay otra PENDIENTE para él.

27.4. BPMN-02 · Reposición: pedido al proveedor y cierre de la alerta

Objetivo: reabastecer un producto desde su alerta sin salir del sistema y dejar constancia del pedido. **Disparador:** alerta PENDIENTE creada por el BPMN-01. **Actores:** Encargado, Administrador, Sistema y Proveedor. **Resultado:** pedido enviado por correo con cantidad y fecha registradas, y alerta atendida cuando llega la mercancía.

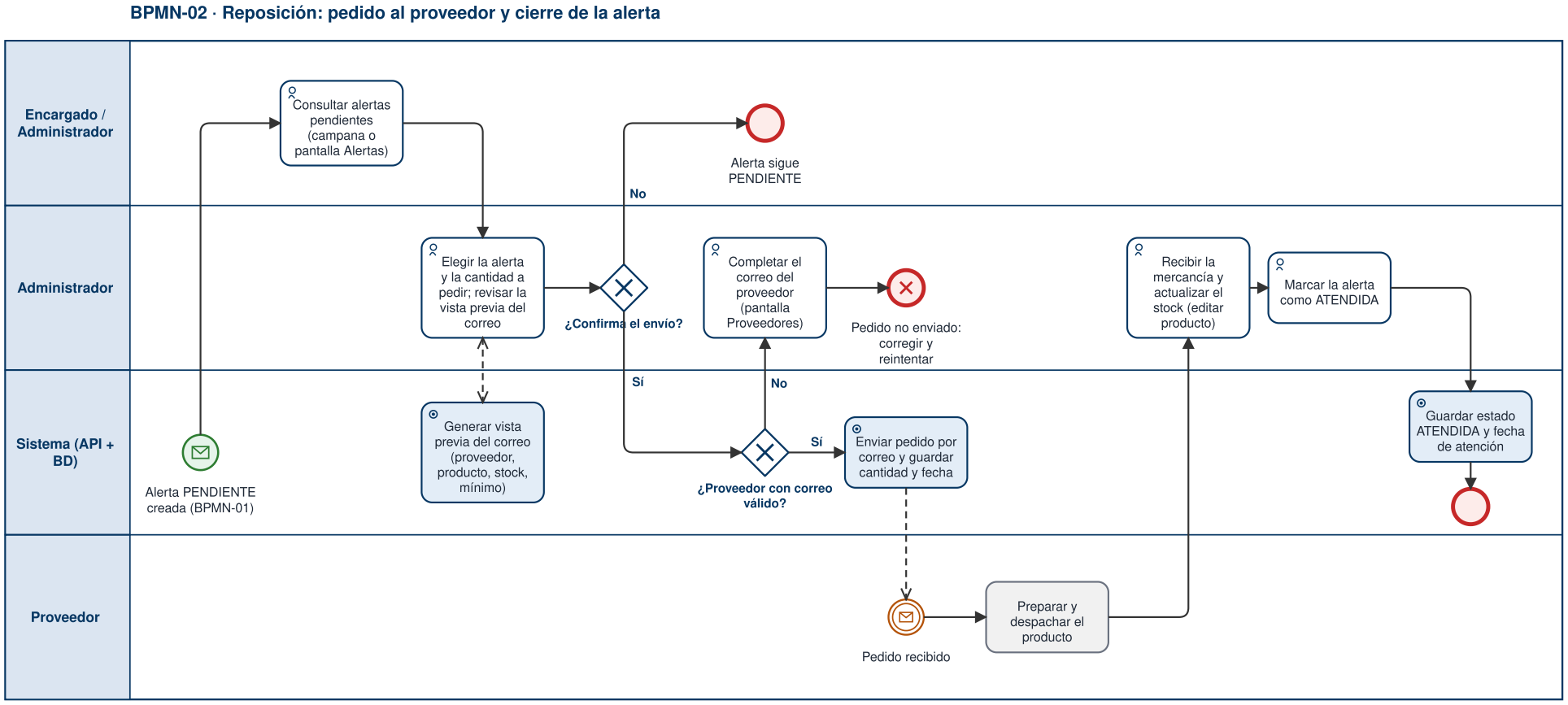


Figura 27.3 — BPMN-02: pedido de reposición al proveedor desde la alerta de stock

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Consultar alertas pendientes	Encargado o Administrador	Alerta PENDIENTE	Lista con producto, stock, proveedor y fecha	HU-25, campana
2	Elegir alerta y cantidad	Administrador	Alerta	Cantidad ≥ 1	HU-49
3	Generar la vista previa	Sistema	Alerta y cantidad	Correo renderizado (proveedor, producto, stock, mínimo)	`SupplierOrderService.preview`
4	¿Confirma el envío?	Administrador	Vista previa	Enviar o cancelar	—
5	¿Proveedor con correo válido?	Sistema	Proveedor del producto	Continuar o 409	RN-07
6	Completar el correo del proveedor	Administrador	Mensaje 409	Proveedor actualizado	HU-18
7	Enviar el pedido	Sistema y Brevo	Pedido	`ordered_quantity` y `ordered_at` guardados	RF-10
8	Preparar y despachar	Proveedor	Correo del pedido	Producto en camino	RACI de la Fase 1
9	Recibir y actualizar el stock	Administrador	Mercancía	Stock ajustado al editar el producto	HU-13
10	Marcar la alerta como ATENDIDA	Administrador	Alerta	Estado ATENDIDA y fecha	RN-06, HU-26

> **Nota:** hoy el stock recibido se actualiza editando el producto a mano y no hay vínculo entre el pedido y la recepción. El proceso propuesto BPMN-F1 lo automatiza con órdenes de compra y movimientos de inventario.

27.5. BPMN-03 · Fotocopia de DPI

Objetivo: convertir dos fotos del DPI en una fotocopia lista para imprimir en menos de 10 segundos. **Disparador:** un cliente pide la fotocopia de su DPI. **Actores:** Cliente, Encargado o Administrador, navegador y API de imágenes. **Resultado:** documento PDF o Word descargable y notificación registrada.

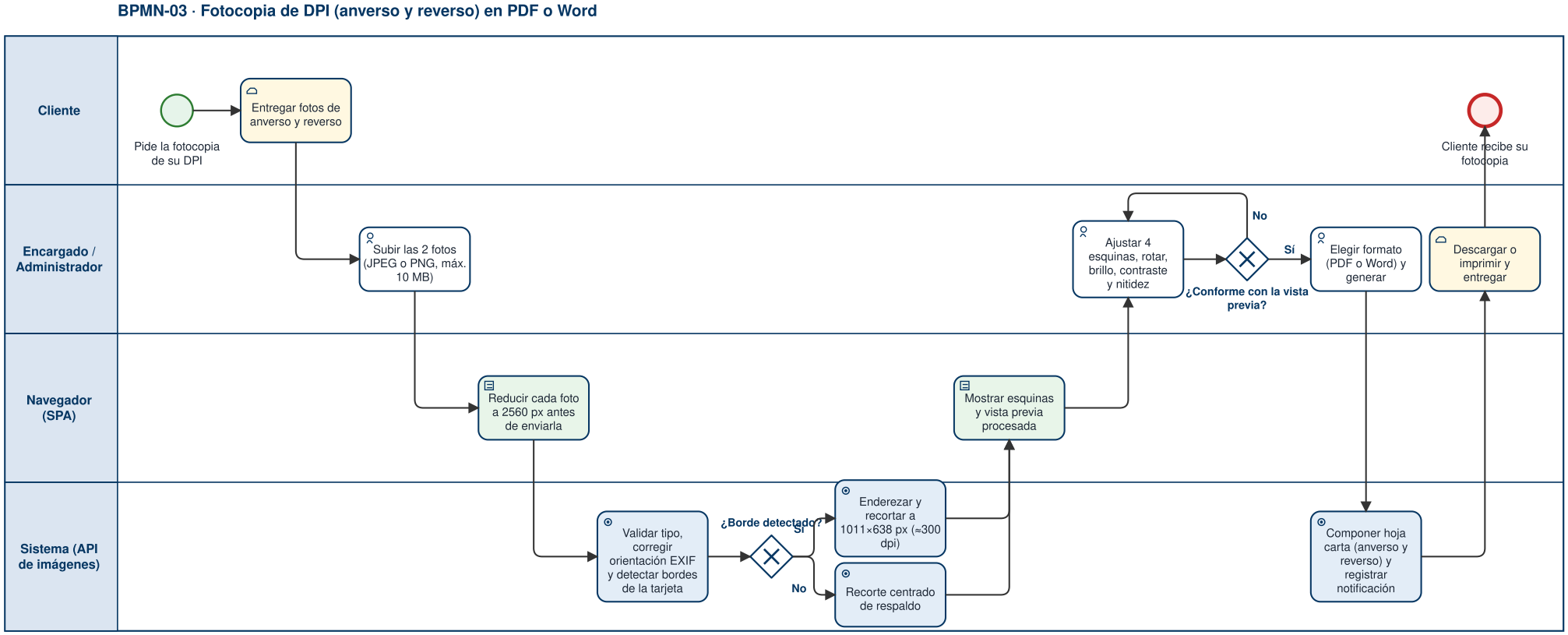


Figura 27.4 — BPMN-03: fotocopia de DPI con detección automática de bordes y ajuste manual

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Entregar las fotos	Cliente	DPI	Fotos de anverso y reverso	—
2	Subir las 2 fotos	Encargado o Administrador	Archivos JPEG o PNG	Fotos cargadas en la SPA	RN-13, HU-27
3	Reducir a 2560 px	Navegador	Foto original	Foto liviana	`imageDownscale.js`
4	Validar, normalizar y detectar bordes	Sistema	Foto	Esquinas detectadas o ausencia de detección	HU-28, `CardDetectionServiceImpl`
5	¿Borde detectado?	Sistema	Contorno candidato	Recorte con perspectiva o recorte centrado	Respaldo automático
6	Enderezar y recortar	Sistema	Esquinas	Imagen de 1011×638 px (≈ 300 dpi)	`ImageCropServiceImpl`
7	Mostrar esquinas y vista previa	Navegador	Imagen procesada	Esquinas arrastrables y PNG de la cara	HU-29
8	Ajustar esquinas, giro y mejora	Encargado o Administrador	Vista previa	Parámetros de recorte, giro, brillo, contraste y nitidez	HU-29 a 31
9	¿Conforme con la vista previa?	Encargado o Administrador	Vista previa	Repetir el ajuste o continuar	—
10	Elegir formato y generar	Encargado o Administrador	PDF o Word	`POST /api/documents/dpi`	HU-32
11	Componer la hoja y notificar	Sistema	Dos caras procesadas	Hoja carta con anverso y reverso; notificación DOCUMENTO_GENERADO	HU-34
12	Descargar o imprimir y entregar	Encargado o Administrador	Documento	Documento entregado	HU-33

27.6. BPMN-04 · Fotos tamaño cédula y pasaporte

Objetivo: producir una hoja de papel fotográfico 4×6 in con varias copias de una foto. **Disparador:** un cliente pide fotos de cédula o de pasaporte. **Actores:** Cliente, Encargado o Administrador y API de imágenes. **Resultado:** PDF o Word con 9 copias de 3.1×3.9 cm o 2 copias de 5×5 cm.

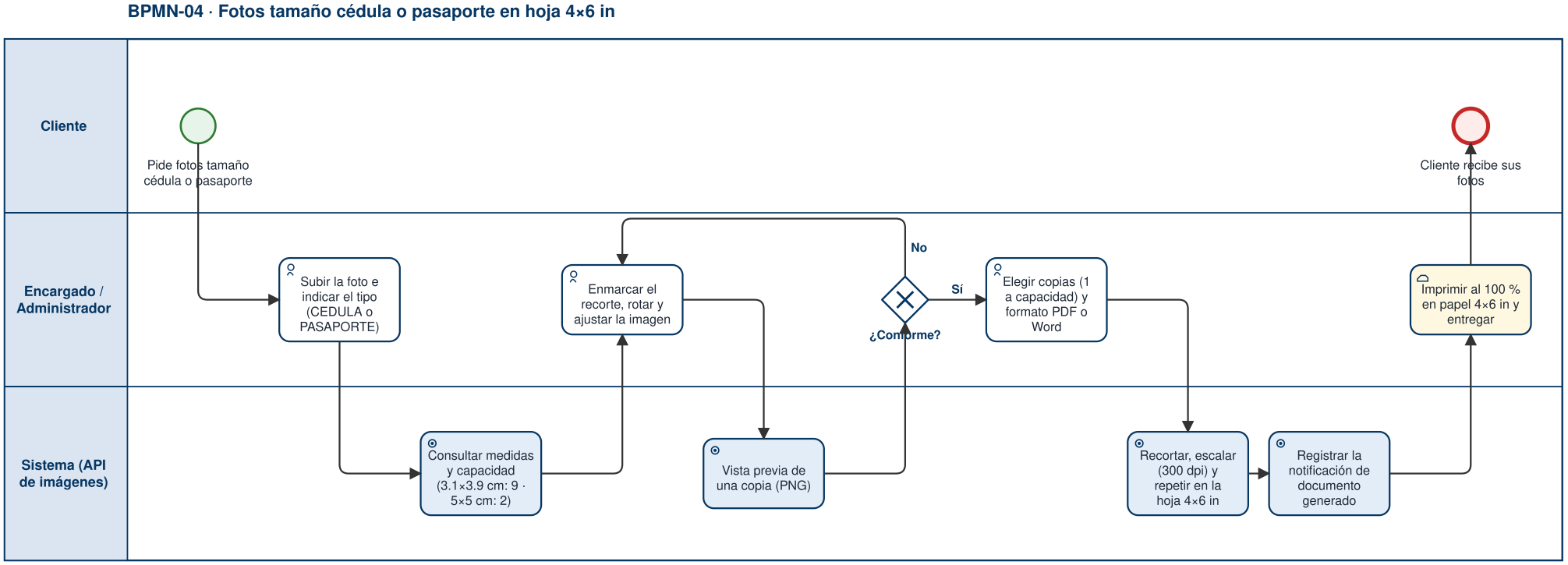


Figura 27.5 — BPMN-04: hoja de fotos tamaño cédula o pasaporte

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Subir la foto e indicar el tipo	Encargado o Administrador	Foto y tipo CEDULA o PASAPORTE	Foto cargada	HU-52, 53
2	Consultar medidas y capacidad	Sistema	`GET /api/documents/id-photo/specs`	Medidas, copias y cuadrícula de la hoja	`DocumentLayoutRegistry`
3	Enmarcar el recorte y ajustar	Encargado o Administrador	Foto	Marco (fracciones de la foto), giro, brillo, contraste y nitidez	`PhotoCropFrame`
4	Vista previa de una copia	Sistema	Parámetros	PNG de una copia a 300 dpi	`POST /id-photo/preview`
5	¿Conforme?	Encargado o Administrador	Vista previa	Ajustar de nuevo o continuar	—
6	Elegir copias y formato	Encargado o Administrador	Copias (1 a capacidad) y PDF o Word	Solicitud de generación	RN-14
7	Componer la hoja 4×6 in	Sistema	Foto recortada	Hoja con las copias y borde de guía de corte	`PdfPhotoSheetComposer`, `WordPhotoSheetComposer`
8	Registrar la notificación	Sistema	Documento generado	Notificación DOCUMENTO_GENERADO	RF-06
9	Imprimir al 100 % y entregar	Encargado o Administrador	Archivo	Fotos entregadas	Papel 4×6 in sin ajustar a la página

27.7. BPMN-05 · Inicio de sesión con verificación en dos pasos

Objetivo: autenticar a un usuario de forma segura, con segundo factor opcional por correo. **Disparador:** un usuario quiere entrar al sistema. **Actores:** Usuario, navegador, API y servicio de correo. **Resultado:** cookie de sesión `jwt` y acceso al inicio según el rol.

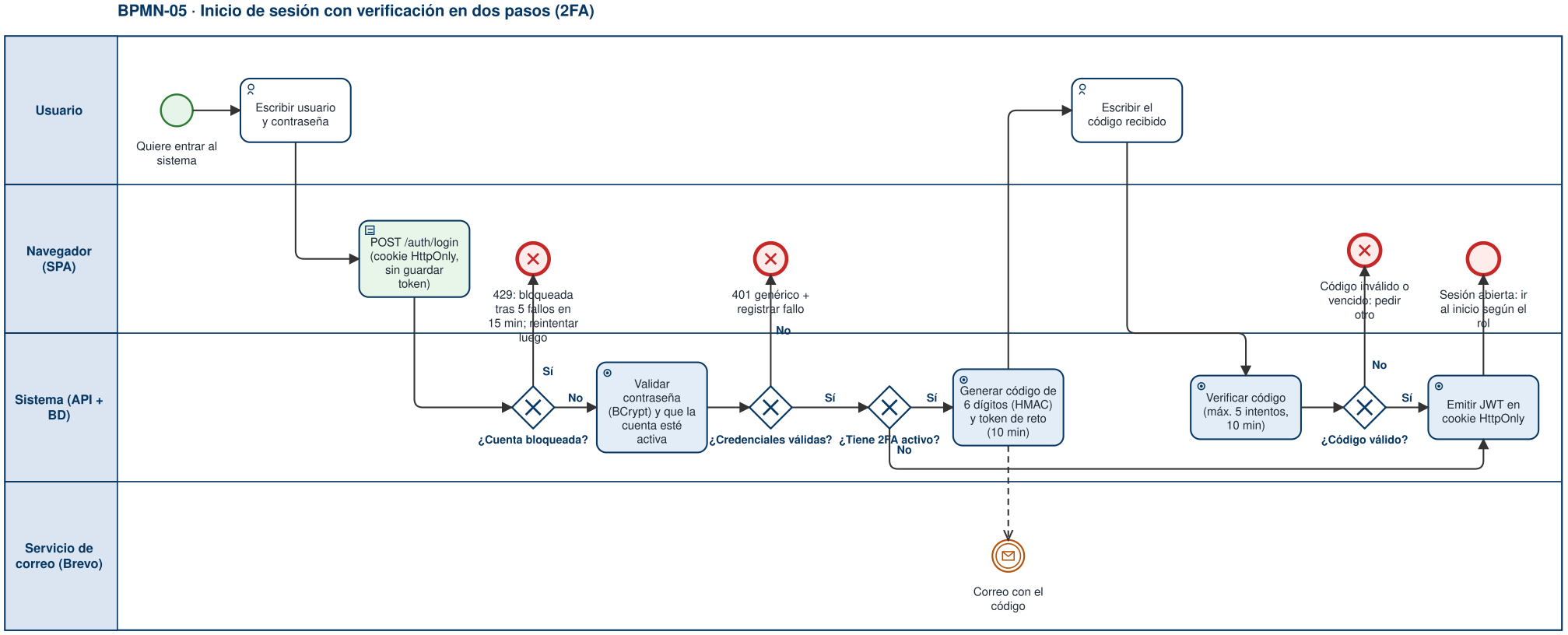


Figura 27.6 — BPMN-05: inicio de sesión con bloqueo por fallos y segundo factor por correo

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Escribir usuario y contraseña	Usuario	Credenciales	Formulario enviado	HU-03
2	Enviar `POST /auth/login`	Navegador	Credenciales	Solicitud con cookie de primera parte	`authService.js`
3	¿Cuenta bloqueada?	Sistema	Intentos del usuario	429 con minutos restantes o continuar	HU-47, `LoginAttemptService`
4	Validar contraseña y cuenta activa	Sistema	Credenciales	Autenticación con BCrypt	`AuthService`
5	¿Credenciales válidas?	Sistema	Resultado	401 genérico y registro del fallo, o continuar	Sin revelar si el usuario existe
6	¿Tiene 2FA activo?	Sistema	Usuario	Emitir JWT o iniciar el reto	HU-43
7	Generar código y token de reto	Sistema	Usuario	Código de 6 dígitos (HMAC), token de 10 min	RNF-07
8	Enviar el código por correo	Servicio de correo	Código	Correo entregado	`MailServiceImpl`
9	Escribir el código	Usuario	Correo recibido	`POST /auth/2fa/verify`	`TwoFactorForm`
10	Verificar el código	Sistema	Código y token de reto	Válido, inválido o agotado (5 intentos)	`VerificationCodeService`
11	Emitir el JWT	Sistema	Verificación correcta	Cookie `jwt` HttpOnly, Secure y SameSite	`CookieService`

27.8. BPMN-06 · Recuperación de contraseña por correo

Objetivo: que un usuario recupere el acceso sin intervención del administrador y sin que el sistema revele qué correos existen. **Disparador:** el usuario olvidó su contraseña. **Actores:** Usuario, navegador, API y servicio de correo. **Resultado:** contraseña actualizada y token consumido.

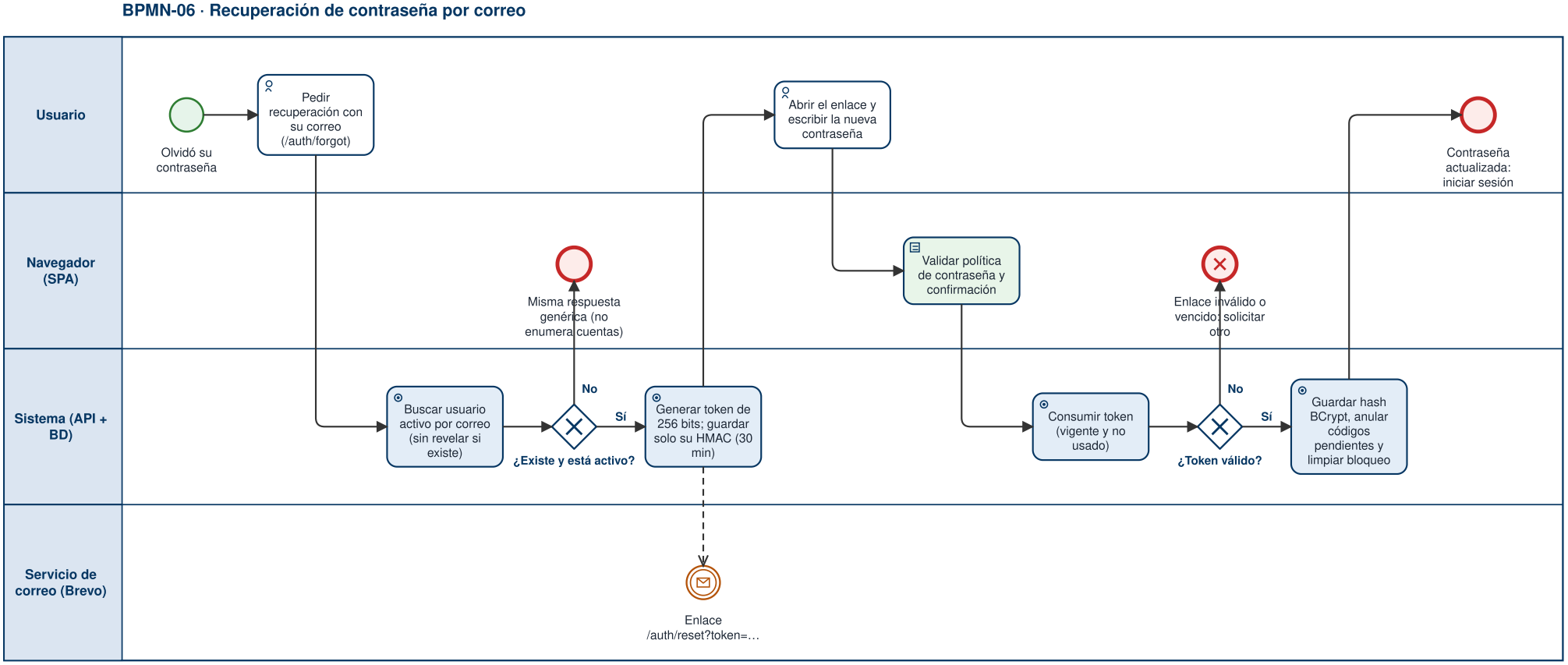


Figura 27.7 — BPMN-06: recuperación de contraseña con enlace de un solo uso

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Pedir la recuperación	Usuario	Correo registrado	`POST /auth/forgot-password`	HU-45, `/auth/forgot`
2	Buscar un usuario activo con ese correo	Sistema	Correo (sin distinguir mayúsculas)	Usuario o ninguno	`PasswordRecoveryServiceImpl`
3	¿Existe y está activo?	Sistema	Resultado	Siempre la misma respuesta genérica	Anti-enumeración de cuentas
4	Generar el token	Sistema	Usuario	Token de 256 bits; en la base solo su HMAC, vigencia de 30 min	RNF-07
5	Enviar el enlace	Servicio de correo	Token	Correo con `/auth/reset?token=…`	`MailServiceImpl`
6	Abrir el enlace y escribir la nueva contraseña	Usuario	Correo	Formulario con confirmación	`reset-password.jsx`
7	Validar la política de contraseña	Navegador	Contraseña	De 8 a 72 caracteres con letra y número	`PasswordRules`
8	Consumir el token	Sistema	Token	Válido (vigente y no usado) o inválido	`VerificationCodeService.consumeResetToken`
9	Actualizar la contraseña	Sistema	Usuario	Hash BCrypt nuevo; códigos 2FA y de recuperación anulados; bloqueo reiniciado	HU-45

27.9. BPMN-07 · Reportes de ventas e inventario con exportación

Objetivo: que el administrador analice el negocio con indicadores y gráficos y comparta los resultados en PDF o Excel. **Disparador:** necesidad de analizar un periodo o el estado del inventario. **Actores:** Encargado o Administrador, navegador y API. **Resultado:** análisis en pantalla y archivo descargable.

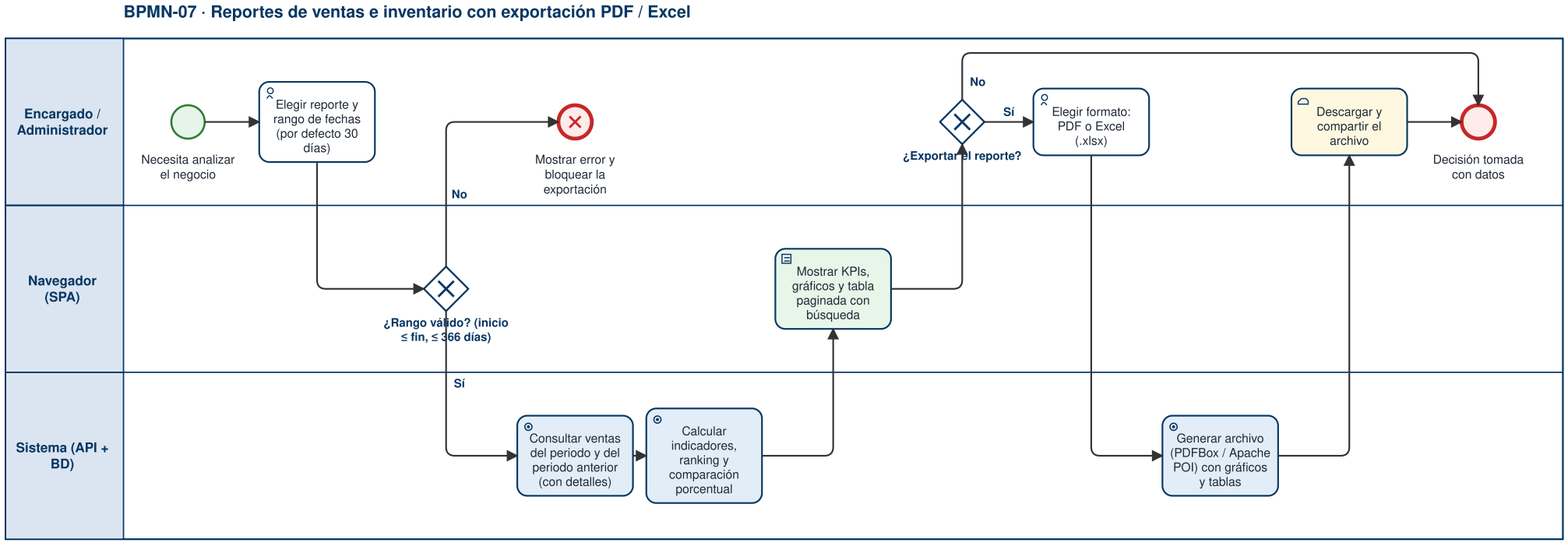


Figura 27.8 — BPMN-07: reportes con comparación de periodos y exportación

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Elegir reporte y rango	Encargado o Administrador	Tipo de reporte y fechas (por defecto 30 días)	Rango seleccionado	HU-35, `DateRangeFilter`
2	¿Rango válido?	Navegador	Fechas	Error y exportación bloqueada, o continuar	RN-12
3	Consultar periodo y periodo anterior	Sistema	Rango	Ventas con detalles en una sola consulta	`findForReport`
4	Calcular indicadores	Sistema	Ventas	Totales, ranking de empleados y productos, variación porcentual	`SalesAnalyticsServiceImpl`
5	Mostrar KPIs, gráficos y tabla	Navegador	Resumen	Panel con tabla paginada y búsqueda	HU-51, ApexCharts
6	¿Exportar? y elegir formato	Encargado o Administrador	Decisión	PDF o Excel	HU-37
7	Generar el archivo	Sistema	Reporte	PDF (PDFBox) o XLSX (POI) con gráficos y tablas	`ReportWriterFactory`
8	Descargar y compartir	Encargado o Administrador	Archivo	`reporte_ventas_AAAAMMDD_AAAAMMDD.pdf` o `.xlsx`	`ReportController`

27.10. BPMN-08 · Entrega continua con GitFlow

Objetivo: llevar un cambio de código a producción de forma repetible, revisada y verificada. **Disparador:** una funcionalidad nueva o una corrección. **Actores:** Desarrollador, Revisor, GitHub Actions y la plataforma de producción. **Resultado:** versión desplegada y comprobada, o versión anterior conservada.

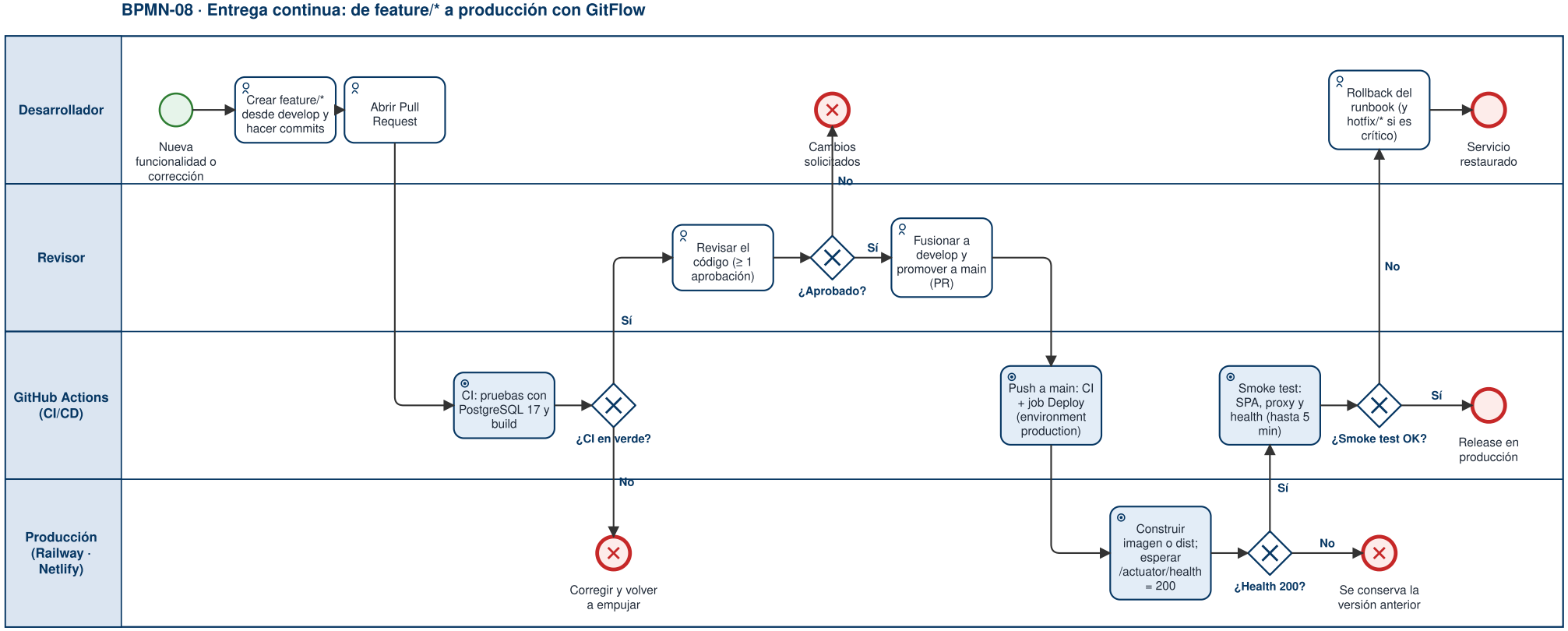


Figura 27.9 — BPMN-08: de la rama feature/ a producción con revisión, CI, despliegue y prueba de humo*

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Crear `feature/*` y hacer commits	Desarrollador	`develop`	Rama con cambios	GitFlow
2	Abrir el Pull Request	Desarrollador	Rama	PR hacia `develop`	RNF-05
3	CI: pruebas y construcción	GitHub Actions	PR	Pruebas con PostgreSQL 17, imagen Docker o bundle	25.4
4	¿CI en verde?	GitHub Actions	Resultado	Continuar o corregir y volver a empujar	Sin CI verde no hay fusión
5	Revisar el código	Revisor	PR	Al menos una aprobación	Code review obligatorio
6	Fusionar y promover a `main`	Revisor	PR aprobado	`develop` y luego PR hacia `main`	Observación de 24.6.2
7	Despliegue desde `main`	GitHub Actions	Push a `main`	CI más el job Deploy con environment `production`	`railway up`, `netlify deploy`
8	Construir y esperar el healthcheck	Plataforma	Código o `dist`	`/actuator/health` = 200 o versión anterior conservada	Railway
9	Prueba de humo	GitHub Actions	URL pública	SPA, proxy y health verificados	RNF-06
10	Rollback y hotfix	Desarrollador	Prueba fallida	Servicio restaurado	25.5

27.11. BPMN-09 · Gestión de incidentes en producción

Objetivo: restaurar el servicio con el menor impacto y aprender de cada incidente. **Disparador:** una alerta del healthcheck, la queja de un usuario o un error en los registros. **Actores:** Usuario o monitoreo, Operador y proveedores cloud. **Resultado:** incidente cerrado con post-mortem y acciones de mejora.

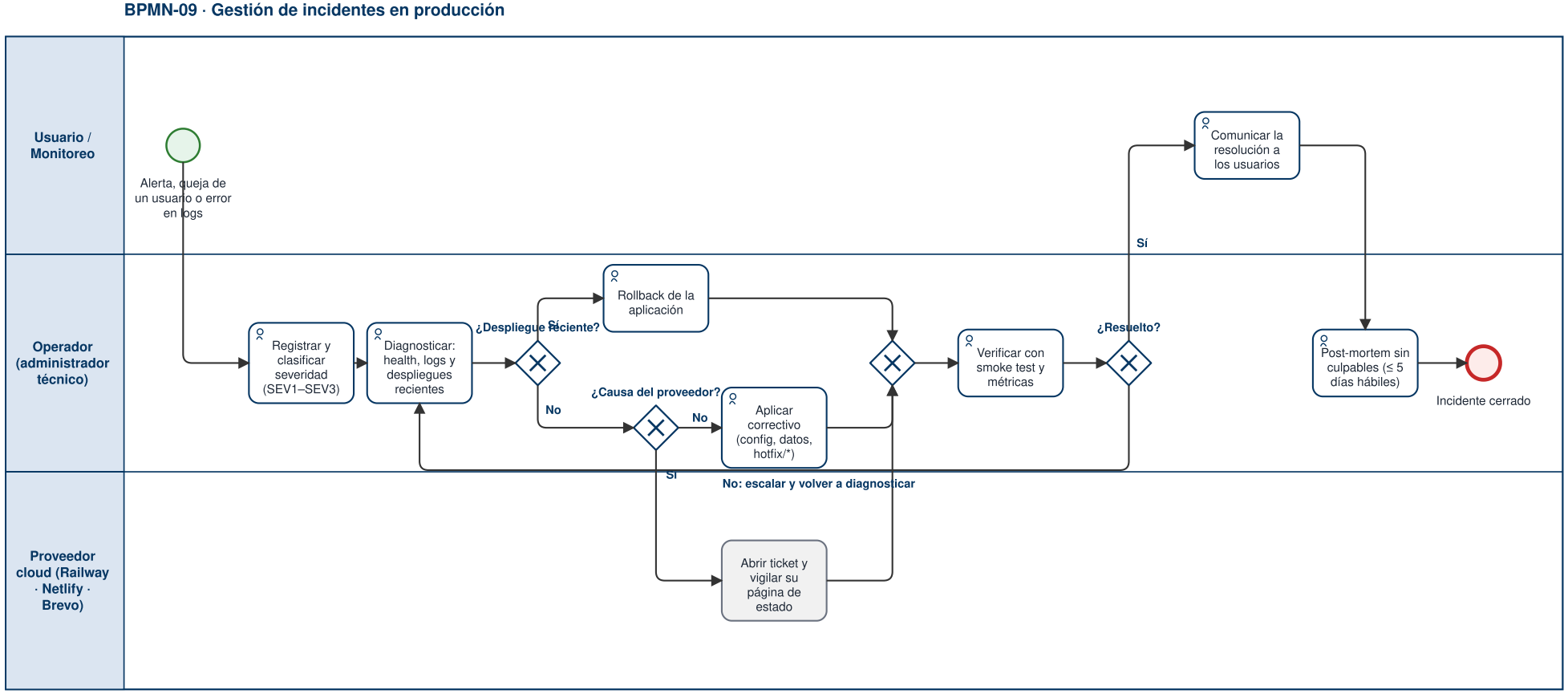


Figura 27.10 — BPMN-09: gestión de incidentes con rollback, escalamiento y post-mortem

N.º	Actividad	Responsable	Entrada	Salida	Regla o componente
1	Registrar y clasificar	Operador	Alerta o reporte	Incidente con severidad SEV1 a SEV3	25.8.1
2	Diagnosticar	Operador	Incidente	Hipótesis de causa (health, registros, despliegues, SQL)	25.8.3 y 25.8.4
3	¿Despliegue reciente?	Operador	Línea de tiempo	Rollback o seguir diagnosticando	25.5
4	¿Causa del proveedor?	Operador	Estado de la plataforma	Abrir ticket y vigilar, o corregir	Escalamiento nivel 3
5	Aplicar el correctivo	Operador	Causa identificada	Configuración, datos o `hotfix/*` desplegado	25.5.5
6	Verificar	Operador	Cambio aplicado	Prueba de humo y métricas normales	25.4.5
7	¿Resuelto?	Operador	Verificación	Comunicar o escalar y volver a diagnosticar	—
8	Comunicar la resolución	Operador	Incidente resuelto	Aviso a los usuarios y al responsable	25.8.5
9	Post-mortem	Operador y responsable	Línea de tiempo	Causa raíz y acciones con responsable y fecha	25.8.6

27.12. Procesos propuestos

Los tres procesos siguientes no existen todavía en el sistema. Se modelan para documentar la evolución natural del negocio y justificar las migraciones de la sección 28: cada uno indica los datos que necesita y la migración que los crea.

27.13. BPMN-F1 · Compras a proveedores con orden y recepción (propuesto)

Objetivo: cerrar el ciclo de reposición: pedir con una orden formal, recibir contra la orden y que el inventario y la alerta se actualicen solos. **Disparador:** alerta de stock o decisión de compra. **Datos que necesita:** `purchase_order`, `purchase_order_item`, `stock_movement` y `product.cost` (V14).

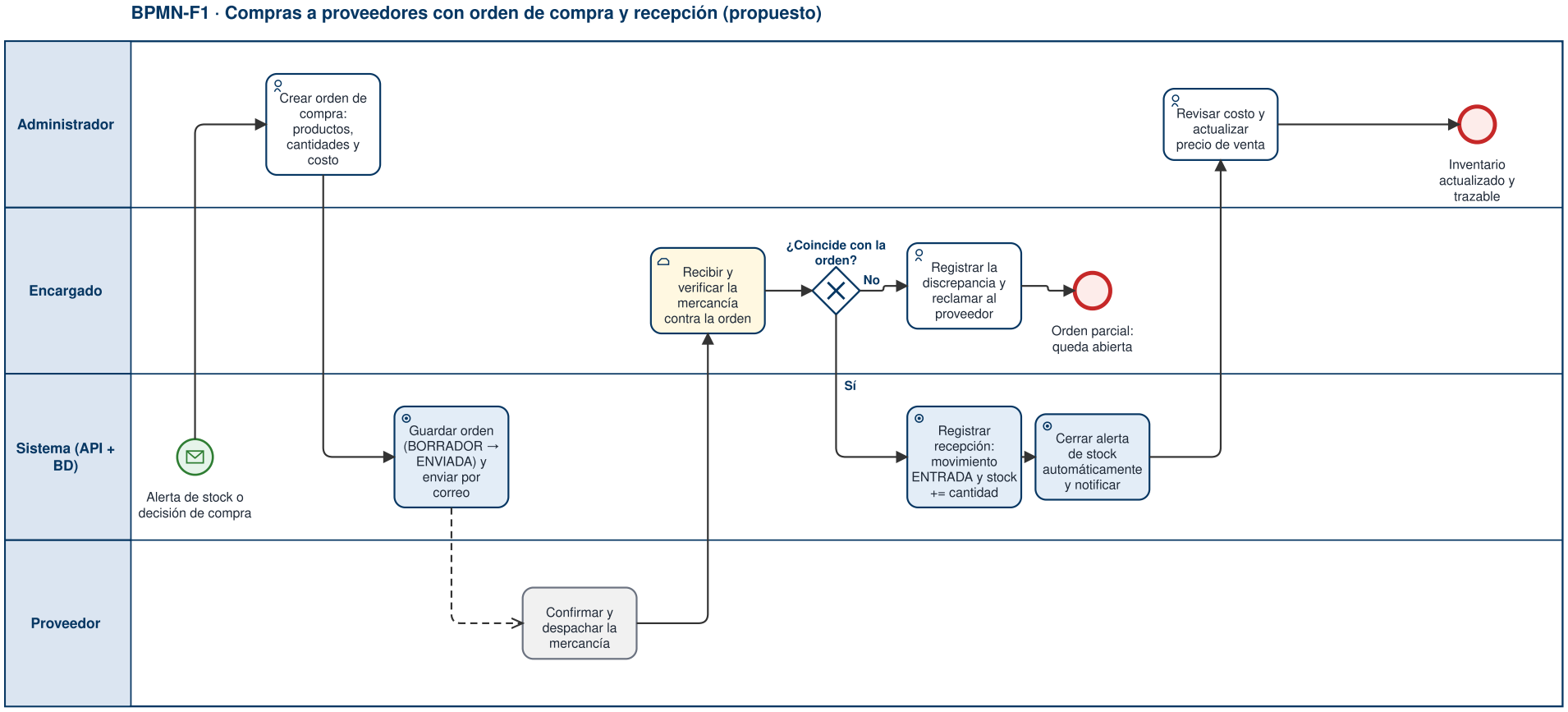


Figura 27.11 — BPMN-F1 (propuesto): compras con orden de compra, recepción y kardex

N.º	Actividad	Responsable	Entrada	Salida	Datos
1	Crear la orden de compra	Administrador	Productos, cantidades y costo	Orden BORRADOR	`purchase_order`, `purchase_order_item`
2	Guardar y enviar	Sistema	Orden	Estado ENVIADA y correo al proveedor	`sent_at`
3	Confirmar y despachar	Proveedor	Correo	Mercancía en camino	—
4	Recibir y verificar	Encargado	Mercancía y orden	Cantidades recibidas	`quantity_received`
5	¿Coincide con la orden?	Encargado	Verificación	Recepción completa o discrepancia	—
6	Registrar la recepción	Sistema	Cantidades	Movimiento ENTRADA y stock actualizado	`stock_movement`
7	Registrar la discrepancia	Encargado	Diferencia	Orden PARCIAL y reclamo al proveedor	`status`
8	Cerrar la alerta y notificar	Sistema	Stock repuesto	Alerta ATENDIDA automática	`stock_alert.purchase_order_id`
9	Revisar costo y precio	Administrador	Costo nuevo	Precio de venta actualizado	`product.cost`

27.14. BPMN-F2 · Apertura y cierre de caja diario (propuesto)

Objetivo: conciliar cada día lo vendido con el dinero recibido. **Disparador:** inicio de la jornada. **Datos que necesita:** `cash_session`, `sales.cash_session_id` y `sales.payment_method` (V15).

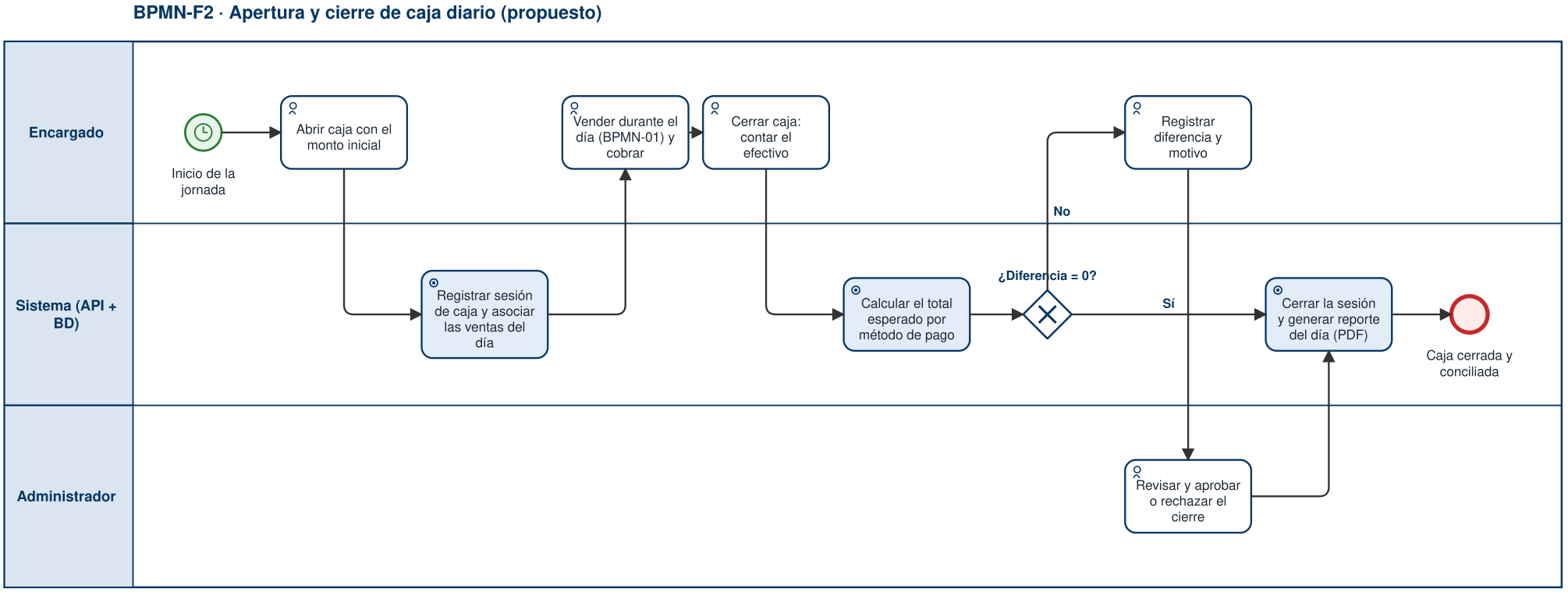


Figura 27.12 — BPMN-F2 (propuesto): apertura y cierre de caja con conciliación

N.º	Actividad	Responsable	Entrada	Salida	Datos
1	Abrir la caja	Encargado	Monto inicial	Sesión ABIERTA (una sola a la vez)	`cash_session`
2	Asociar las ventas del día	Sistema	Ventas y método de pago	`cash_session_id` en cada venta	`sales`
3	Cerrar la caja y contar	Encargado	Efectivo contado	Monto contado	`counted_amount`
4	Calcular el total esperado	Sistema	Ventas en efectivo del día	Monto esperado y diferencia	`expected_amount`, `difference`
5	¿Diferencia igual a cero?	Sistema	Diferencia	Cierre directo o revisión	—
6	Registrar la diferencia y el motivo	Encargado	Diferencia	Sesión OBSERVADA	`notes`
7	Revisar y aprobar	Administrador	Cierre observado	Aprobación o rechazo	—
8	Cerrar y reportar	Sistema	Cierre aprobado	Sesión CERRADA y reporte del día en PDF	RF-07

27.15. BPMN-F3 · Registro de servicios (propuesto)

Objetivo: que fotocopias, emplasticados y los documentos generados por el sistema aparezcan como ingresos y en los reportes. **Disparador:** un cliente solicita un servicio. **Datos que necesita:** `service_item`, `sale_detail.service_id`, `generated_document` y `customer` (V15).

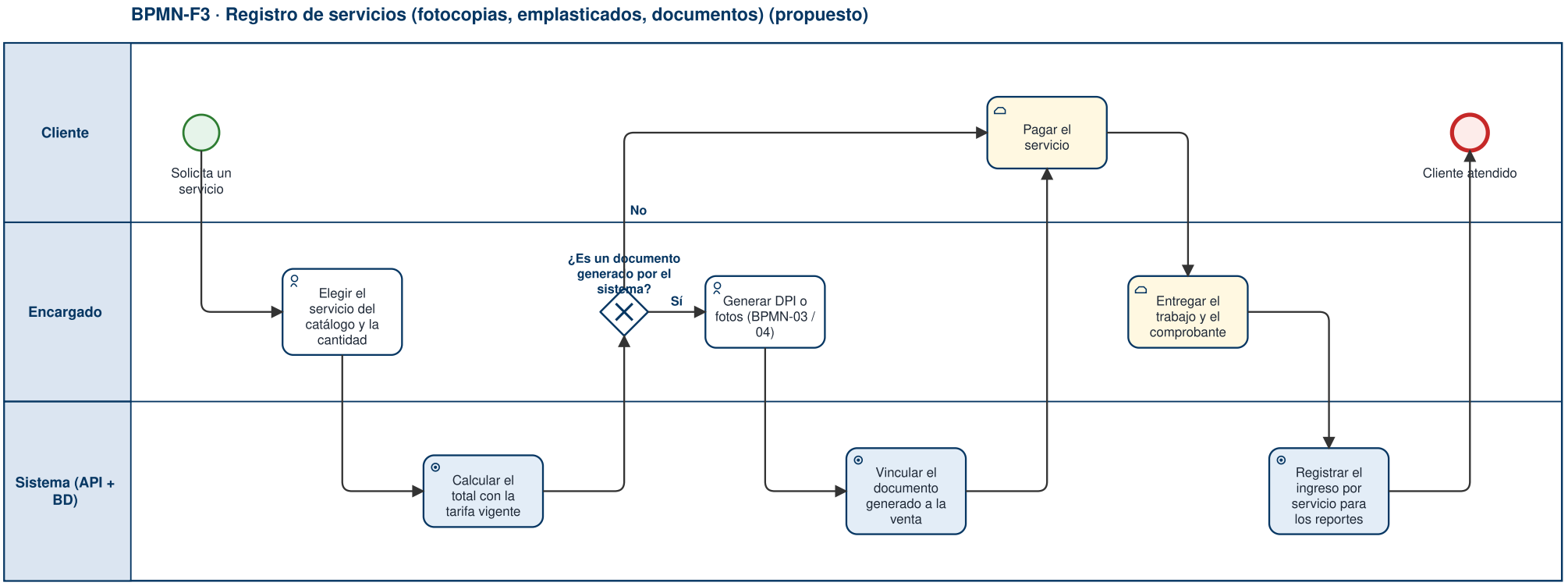


Figura 27.13 — BPMN-F3 (propuesto): venta de servicios y vínculo con los documentos generados

N.º	Actividad	Responsable	Entrada	Salida	Datos
1	Elegir el servicio y la cantidad	Encargado	Catálogo de servicios	Línea de servicio en la venta	`service_item`
2	Calcular el total	Sistema	Tarifa vigente	Total de la venta	`unit_price`
3	¿Es un documento generado por el sistema?	Encargado	Tipo de servicio	Generar DPI o fotos, u otro servicio	—
4	Generar el documento	Encargado	BPMN-03 o BPMN-04	Documento	—
5	Vincular el documento a la venta	Sistema	Documento y venta	Registro con tipo, formato, copias y duración	`generated_document`
6	Pagar el servicio	Cliente	Total	Pago	`payment_method`
7	Entregar el trabajo y el comprobante	Encargado	Trabajo terminado	Cliente atendido	—
8	Registrar el ingreso	Sistema	Venta cerrada	Ingreso por servicio en los reportes	RF-07

Nota: `generated_document` también mide la duración de cada generación, lo que permite comprobar RNF-02 con datos reales en lugar de pruebas puntuales.

28. Migraciones para la Evolución del Modelo de Datos

28.1. Política y convenciones

El esquema evoluciona únicamente con migraciones Flyway versionadas junto al código (`src/main/resources/db/migration`). Hibernate valida pero no genera DDL. Estas reglas rigen las once migraciones ya aplicadas y deben cumplirse en las siguientes.

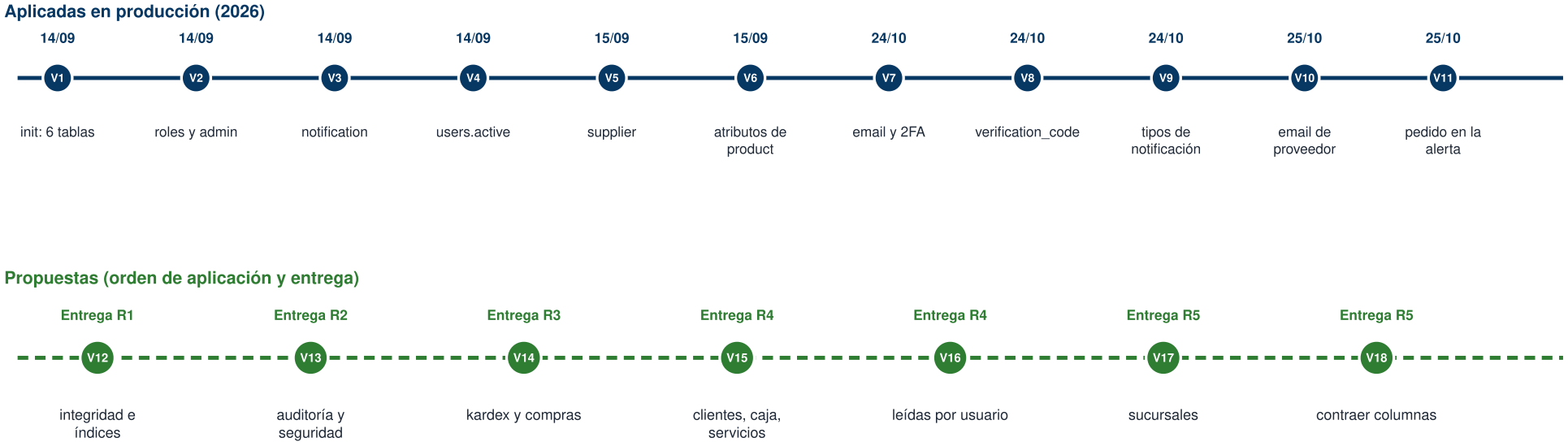
N.º	Regla
1	**Nunca se edita una migración ya aplicada**: Flyway valida su checksum y la aplicación no arrancaría. Toda corrección es una migración nueva
2	Nombre `V<n>__<descripcion_en_minusculas>.sql`, con un encabezado de comentario que explique el propósito; una responsabilidad por migración
3	**Expandir y contraer:** primero se agregan tablas y columnas compatibles con la versión anterior del código (opcionales o con valor por defecto); la columna vieja se retira en una migración posterior, cuando ningún código la usa
4	Las migraciones de datos (rellenos) van en la misma migración que crea la estructura y deben poder ejecutarse sobre datos reales sin supuestos
5	Restricciones e índices con nombre explícito: `uq_` para únicos, `idx_` para índices, `*_check` para CHECK
6	Restricciones CHECK sobre tablas con datos: `NOT VALID` y luego `VALIDATE CONSTRAINT`, para no bloquear escrituras durante la validación
7	Con tablas grandes, índices con `CREATE INDEX CONCURRENTLY` (requiere desactivar la transacción de esa migración en su archivo de configuración de Flyway)
8	No existe «deshacer»: el retorno es una migración que corrige hacia adelante o la restauración de un respaldo (25.5.4)
9	Todo cambio de esquema se prueba antes en una copia de la base y en el CI, que aplica todas las migraciones contra un PostgreSQL 17 real
10	Antes de desplegar una migración con rellenos o cambios destructivos se toma un respaldo y se anota su identificador en el Pull Request

Ciclo de vida de una migración: diseñarla con la regla 3, escribirla y probarla en local (`docker compose up`), abrir el Pull Request, dejar que el CI la aplique sobre PostgreSQL 17, desplegar (Flyway la ejecuta al arrancar la API nueva) y verificar `flyway_schema_history`, los conteos y la prueba de humo.

28.2. Historial aplicado y línea de tiempo

Las once migraciones aplicadas (V1 a V11) se detallan en la sección 24.5.5. La figura muestra su ubicación en el tiempo y el plan de las siguientes siete.

Línea de tiempo del esquema: aplicado (V1–V11) y propuesto (V12–V18)



Las migraciones aplicadas no se editan jamás (Flyway valida su checksum). Toda corrección se hace con una migración nueva.

Figura 28.1 — Migraciones aplicadas (V1 a V11) y propuestas (V12 a V18), agrupadas en cinco entregas

28.3. Hallazgos del modelo actual que motivan las migraciones

ID	Hallazgo	Evidencia	Riesgo	Migración
H-01	Las reglas numéricas (stock, precio, cantidad) solo las valida la aplicación	V1 y V6 sin CHECK	Datos inválidos si se escribe por SQL o por otro cliente	V12
H-02	Una alerta pendiente por producto solo se garantiza en el servicio	`SaleServiceImpl` consulta antes de insertar	Alertas duplicadas por una carrera	V12
H-03	No hay índices sobre `sales.date` ni sobre claves foráneas de `sale_detail`	Esquema V1	Reportes y dashboard con lecturas completas	V12
H-04	No existe una bitácora de auditoría	Sin tabla	No se puede reconstruir quién hizo qué	V13
H-05	El bloqueo de inicio de sesión vive en memoria y el cierre de sesión no revoca el JWT	`LoginAttemptService`, `AuthService.logout`	Seguridad y escalabilidad horizontal	V13
H-06	La contraseña de `admin` sembrada por V2 es pública	README y migración V2	Acceso no autorizado	V13
H-07	El correo se envía dentro de la petición, sin cola ni reintentos	`MailServiceImpl`	Latencia y correos perdidos	V13
H-08	El stock es un contador mutable sin historial de movimientos	`product.stock`	No se explica una diferencia de inventario	V14
H-09	El pedido a proveedor no se enlaza con la recepción	`stock_alert.ordered_*`	Inventario desactualizado hasta editar a mano	V14
H-10	Cliente como texto libre; sin método de pago, caja ni servicios	`sales.customer_name`	Reportes incompletos; sin conciliación diaria	V15
H-11	La marca «leída» de una notificación es global	`notification.read`	Un rol oculta la notificación al otro	V16
H-12	El stock pertenece a una sola sucursal	`product.stock`, `min_stock`	Imposible una segunda sucursal sin rediseño	V17
H-13	Columnas que quedarán sin uso	`ordered_*`, `customer_name`, `read`, `stock`	Deuda técnica y confusión	V18

28.4. Plan de migraciones propuestas

Versión	Entrega	Nombre	Cambios clave	Riesgo	Compatibilidad con el código anterior
V12	R1	Integridad e índices	Índice único de alerta pendiente, 6 restricciones CHECK y 6 índices	Bajo	Total: solo restringe y acelera
V13	R2	Auditoría y estado de seguridad	`audit_log`, `login_attempt`, `email_outbox`; columnas de `users`	Medio	Total: columnas con valor por defecto
V14	R3	Kardex y compras	`purchase_order`, `purchase_order_item`, `stock_movement`, `product.cost`; relleno del histórico	Medio	Total: tablas y columnas nuevas
V15	R4	Clientes, caja, servicios y documentos	`customer`, `cash_session`, `service_item`, `generated_document`; cambios en `sales` y `sale_detail`	Alto	Parcial: `product_id` pasa a ser opcional; desplegar primero el código nuevo
V16	R4	Notificaciones leídas por usuario	`notification_read` y relleno desde la marca global	Bajo	Total: la columna `read` se conserva
V17	R5	Sucursales	`branch`, `inventory_stock`, `branch_id` en cuatro tablas	Alto	Requiere código nuevo: el stock pasa a `inventory_stock`
V18	R5	Contraer columnas heredadas	Retira `ordered_*`, `customer_name`, `read`, `stock` y `min_stock`	Alto (destructiva)	Solo después de un ciclo completo sin usar esas columnas

Las cinco entregas son: **R1** endurecer (V12), **R2** seguridad y auditoría (V13), **R3** compras e inventario trazable (V14), **R4** clientes, caja y servicios (V15 y V16) y **R5** sucursales y limpieza (V17 y V18).

Importante: los scripts de las páginas siguientes se ejecutaron en secuencia sobre una base con el esquema V1 a V11 y datos de ejemplo (alertas pendientes duplicadas, ventas con y sin cliente y notificaciones leídas y sin leer). Todas aplicaron sin errores y sus rellenos dieron el resultado esperado: la alerta duplicada se cerró, cada línea de venta pasada produjo su movimiento de kardex, el cliente por nombre se convirtió en registro y las notificaciones leídas se trasladaron por rol. La prueba corrió en PostgreSQL 18.6; las instrucciones usadas son válidas en PostgreSQL 17, la versión de producción.

28.5. V12 · Integridad e índices

Propósito. Llevar a la base de datos las reglas que hoy solo protege la aplicación (hallazgos H-01 a H-03). Antes de crear el índice único se cierran las alertas pendientes duplicadas conservando la más reciente, y las restricciones se agregan con `NOT VALID` y se validan aparte para no bloquear escrituras.

```
-- Reglas que hoy solo vigilan los servicios pasan a ser restricciones de la base de datos.

-- 1) Una alerta PENDIENTE por producto (HU-23). Antes, las duplicadas se cierran.
UPDATE stock_alert a
   SET status = 'ATENDIDA', attended_at = COALESCE(a.attended_at, now())
 WHERE a.status = 'PENDIENTE'
   AND a.stock_alert_id <> (SELECT max(b.stock_alert_id) FROM stock_alert b
                            WHERE b.product_id = a.product_id AND b.status = 'PENDIENTE');
CREATE UNIQUE INDEX uq_stock_alert_pending_per_product
    ON stock_alert (product_id) WHERE status = 'PENDIENTE';

-- 2) Dominio de los datos (NOT VALID + VALIDATE evita bloqueos largos).
ALTER TABLE product     ADD CONSTRAINT product_stock_nonneg      CHECK (stock >= 0)       NOT VALID;
ALTER TABLE product     ADD CONSTRAINT product_price_nonneg      CHECK (price >= 0)       NOT VALID;
ALTER TABLE product     ADD CONSTRAINT product_min_stock_nonneg  CHECK (min_stock >= 0)   NOT VALID;
ALTER TABLE sale_detail ADD CONSTRAINT sale_detail_amount_pos    CHECK (amount > 0)       NOT VALID;
ALTER TABLE sale_detail ADD CONSTRAINT sale_detail_price_nonneg  CHECK (unit_price >= 0)  NOT VALID;
ALTER TABLE sales       ADD CONSTRAINT sales_total_nonneg        CHECK (total >= 0)       NOT VALID;
ALTER TABLE product     VALIDATE CONSTRAINT product_stock_nonneg;
ALTER TABLE product     VALIDATE CONSTRAINT product_price_nonneg;
ALTER TABLE product     VALIDATE CONSTRAINT product_min_stock_nonneg;
ALTER TABLE sale_detail VALIDATE CONSTRAINT sale_detail_amount_pos;
ALTER TABLE sale_detail VALIDATE CONSTRAINT sale_detail_price_nonneg;
ALTER TABLE sales       VALIDATE CONSTRAINT sales_total_nonneg;

-- 3) Índices para reportes, dashboard y claves foráneas sin índice.
CREATE INDEX idx_sales_date            ON sales (date DESC);
CREATE INDEX idx_sales_employee_date   ON sales (employee_id, date DESC);
CREATE INDEX idx_sale_detail_sales     ON sale_detail (sales_id);
CREATE INDEX idx_sale_detail_product   ON sale_detail (product_id);
CREATE INDEX idx_stock_alert_status    ON stock_alert (status, created_at DESC);
CREATE INDEX idx_product_supplier      ON product (supplier_id);
```

- **Impacto en el código:** ninguno obligatorio. Conviene que `SaleServiceImpl` trate la violación de `uq_stock_alert_pending_per_product` como «ya existe una alerta» en lugar de un 409 genérico.
- **Reversión:** `DROP INDEX` y `ALTER TABLE … DROP CONSTRAINT` de los nombres anteriores. Las alertas cerradas por la limpieza no se reabren.

28.6. V13 · Auditoría y estado de seguridad

Propósito. Crear la bitácora de auditoría, sacar el estado de seguridad de la memoria del proceso, permitir revocar sesiones y forzar el cambio de la contraseña sembrada (hallazgos H-04 a H-07).

```
-- Bitácora de auditoría y estado de seguridad persistente (sirve con varias instancias).

CREATE TABLE audit_log (
    audit_id       BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    occurred_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
    actor_user_id  UUID REFERENCES users (user_id) ON DELETE SET NULL,
    actor_username VARCHAR(255),
    action         VARCHAR(60)  NOT NULL,
    entity_type    VARCHAR(50),
    entity_id      VARCHAR(64),
    detail         JSONB,
    client_ip      INET
);
CREATE INDEX idx_audit_occurred ON audit_log (occurred_at DESC);
CREATE INDEX idx_audit_entity   ON audit_log (entity_type, entity_id);
CREATE INDEX idx_audit_actor    ON audit_log (actor_user_id, occurred_at DESC);
```

```
-- Revocar sesiones (logout real, cambio de contraseña) y forzar cambio de la contraseña sembrada.
ALTER TABLE users ADD COLUMN token_version        INTEGER      NOT NULL DEFAULT 0;
ALTER TABLE users ADD COLUMN must_change_password BOOLEAN      NOT NULL DEFAULT FALSE;
ALTER TABLE users ADD COLUMN password_changed_at  TIMESTAMPTZ;
ALTER TABLE users ADD COLUMN last_login_at        TIMESTAMPTZ;
UPDATE users SET must_change_password = TRUE WHERE username = 'admin';

-- Sustituye al mapa en memoria de LoginAttemptService.
CREATE TABLE login_attempt (
    username_key    VARCHAR(255) PRIMARY KEY,
    failures        INTEGER     NOT NULL DEFAULT 0,
    first_failure_at TIMESTAMPTZ NOT NULL,
    locked_until    TIMESTAMPTZ
);

-- Cola de correos: reintentos, límites por destinatario y visibilidad de fallos.
CREATE TABLE email_outbox (
    email_id     BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    kind         VARCHAR(30)  NOT NULL
        CONSTRAINT email_outbox_kind_check CHECK (kind IN
('LOGIN_2FA','ENABLE_2FA','PASSWORD_RESET','SALE_RECEIPT','SUPPLIER_ORDER')),
    recipient    VARCHAR(254) NOT NULL,
    subject      VARCHAR(200) NOT NULL,
    status       VARCHAR(12)  NOT NULL DEFAULT 'PENDIENTE'
        CONSTRAINT email_outbox_status_check CHECK (status IN ('PENDIENTE','ENVIADO','FALLIDO')),
    attempts     INTEGER      NOT NULL DEFAULT 0,
    last_error   VARCHAR(500),
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    sent_at      TIMESTAMPTZ
);
CREATE INDEX idx_email_outbox_pending ON email_outbox (status, created_at) WHERE status <> 'ENVIADO';
```

- **Impacto en el código:** el JWT incorpora la versión del token y `JwtAuthenticationFilter` la compara con `users.token_version`; el cierre de sesión y el cambio de contraseña la incrementan; `LoginAttemptService` usa `login_attempt`; `MailServiceImpl` escribe en `email_outbox` y un trabajo programado envía y reintenta; el frontend redirige a cambiar la contraseña cuando `must_change_password` es verdadero.
- **Reversión:** las columnas nuevas tienen valor por defecto, de modo que el código anterior las ignora; las tablas nuevas pueden eliminarse si aún no se usan.

28.7. V14 · Kardex y compras

Propósito. Registrar cada movimiento de inventario y cerrar el ciclo de reposición con órdenes de compra y recepción (hallazgos H-08 y H-09). El histórico de ventas se convierte en movimientos de salida sin saldo posterior (`stock_after` queda nulo), porque ese dato no se conserva hoy.

```
-- Kardex de inventario y compras a proveedores con orden y recepción.

ALTER TABLE product ADD COLUMN cost NUMERIC(10, 2) CONSTRAINT product_cost_nonneg CHECK (cost >= 0);

CREATE TABLE purchase_order (
    purchase_order_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    supplier_id       BIGINT      NOT NULL REFERENCES supplier (supplier_id),
    status            VARCHAR(12) NOT NULL DEFAULT 'BORRADOR'
        CONSTRAINT purchase_order_status_check CHECK (status IN
('BORRADOR','ENVIADA','PARCIAL','RECIBIDA','CANCELADA')),
    created_by        UUID        NOT NULL REFERENCES users (user_id),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    sent_at           TIMESTAMPTZ,
    received_at       TIMESTAMPTZ,
    notes             VARCHAR(500)
);
CREATE INDEX idx_purchase_order_supplier ON purchase_order (supplier_id, status);

CREATE TABLE purchase_order_item (
    item_id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    purchase_order_id BIGINT  NOT NULL REFERENCES purchase_order (purchase_order_id) ON DELETE CASCADE,
    product_id        BIGINT  NOT NULL REFERENCES product (product_id),
    quantity_ordered  INTEGER NOT NULL CHECK (quantity_ordered > 0),
    quantity_received INTEGER NOT NULL DEFAULT 0 CHECK (quantity_received >= 0),
    unit_cost         NUMERIC(10, 2) CHECK (unit_cost >= 0),
    CONSTRAINT uq_po_item UNIQUE (purchase_order_id, product_id)
);

CREATE TABLE stock_movement (
```

```
    movement_id       BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    product_id        BIGINT      NOT NULL REFERENCES product (product_id),
    movement_type     VARCHAR(12) NOT NULL
        CONSTRAINT stock_movement_type_check CHECK (movement_type IN
('VENTA','ENTRADA','AJUSTE','ANULACION')),
    quantity          INTEGER     NOT NULL CHECK (quantity <> 0),   -- con signo: negativo = salida
    stock_after       INTEGER,                                       -- nulo en el histórico reconstruido
    sale_id           BIGINT REFERENCES sales (sales_id),
    purchase_order_id BIGINT REFERENCES purchase_order (purchase_order_id),
    actor_user_id     UUID REFERENCES users (user_id),
    reason            VARCHAR(200),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_stock_movement_product ON stock_movement (product_id, created_at DESC);
CREATE INDEX idx_stock_movement_sale    ON stock_movement (sale_id);

-- Histórico: cada línea de venta pasada se convierte en un movimiento de salida.
INSERT INTO stock_movement (product_id, movement_type, quantity, sale_id, actor_user_id, created_at)
SELECT d.product_id, 'VENTA', -d.amount, s.sales_id, s.employee_id, s.date AT TIME ZONE 'America/Guatemala'
  FROM sale_detail d JOIN sales s ON s.sales_id = d.sales_id;

-- La alerta se enlaza con la orden (expandir); las columnas ordered_* se eliminan más adelante (contraer).
ALTER TABLE stock_alert ADD COLUMN purchase_order_id BIGINT REFERENCES purchase_order (purchase_order_id);
```

- **Impacto en el código:** `SaleServiceImpl` inserta un movimiento `VENTA` por línea; un servicio nuevo de órdenes de compra reemplaza el pedido por correo de `SupplierOrderService` (que crea la orden y la envía); la recepción genera movimientos `ENTRADA` y cierra la alerta. Habilita el BPMN-F1.
- **Reversión:** las tablas nuevas pueden eliminarse sin pérdida de datos propios (los movimientos históricos se pueden reconstruir con el mismo `INSERT`).

28.8. V15 · Clientes, caja, servicios y documentos generados

Propósito. Pasar del cliente como texto a una entidad, registrar el método de pago y la caja diaria, incluir los servicios en las ventas y medir cada documento generado (hallazgo H-10). Habilita los procesos BPMN-F2 y BPMN-F3.

```
-- Clientes, métodos de pago, caja diaria, catálogo de servicios y documentos generados.

CREATE TABLE customer (
    customer_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    phone       VARCHAR(30),
    email       VARCHAR(254),
    nit         VARCHAR(20),
    active      BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_customer_email ON customer (LOWER(email)) WHERE email IS NOT NULL;

CREATE TABLE cash_session (
    cash_session_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    status          VARCHAR(10) NOT NULL DEFAULT 'ABIERTA'
        CONSTRAINT cash_session_status_check CHECK (status IN ('ABIERTA','CERRADA','OBSERVADA')),
    opened_by       UUID        NOT NULL REFERENCES users (user_id),
    opened_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    opening_amount  NUMERIC(10, 2) NOT NULL CHECK (opening_amount >= 0),
    closed_by       UUID REFERENCES users (user_id),
    closed_at       TIMESTAMPTZ,
    expected_amount NUMERIC(10, 2),
    counted_amount  NUMERIC(10, 2),
    difference      NUMERIC(10, 2) GENERATED ALWAYS AS (counted_amount - expected_amount) STORED,
    notes           VARCHAR(500)
);
CREATE UNIQUE INDEX uq_cash_session_one_open ON cash_session (status) WHERE status = 'ABIERTA';

ALTER TABLE sales ADD COLUMN customer_id     BIGINT REFERENCES customer (customer_id);
ALTER TABLE sales ADD COLUMN cash_session_id BIGINT REFERENCES cash_session (cash_session_id);
ALTER TABLE sales ADD COLUMN payment_method  VARCHAR(14) NOT NULL DEFAULT 'EFECTIVO'
    CONSTRAINT sales_payment_method_check CHECK (payment_method IN ('EFECTIVO','TARJETA','TRANSFERENCIA'));
CREATE INDEX idx_sales_customer ON sales (customer_id);

-- Los clientes ya registrados por nombre se conservan como registros de cliente.
INSERT INTO customer (name)
SELECT DISTINCT TRIM(customer_name) FROM sales WHERE customer_name IS NOT NULL AND TRIM(customer_name) <>
'';
UPDATE sales s SET customer_id = c.customer_id
```

```
  FROM customer c WHERE s.customer_name IS NOT NULL AND c.name = TRIM(s.customer_name);

CREATE TABLE service_item (
    service_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code       VARCHAR(30)  NOT NULL UNIQUE,
    name       VARCHAR(100) NOT NULL,
    unit_price NUMERIC(10, 2) NOT NULL CHECK (unit_price >= 0),
    active     BOOLEAN NOT NULL DEFAULT TRUE
);
INSERT INTO service_item (code, name, unit_price) VALUES
    ('FOTOCOPIA', 'Fotocopia', 0.50), ('EMPLASTICADO', 'Emplasticado', 5.00),
    ('FOTOCOPIA_DPI', 'Fotocopia de DPI', 5.00), ('FOTO_CEDULA', 'Fotos tamaño cédula', 15.00),
    ('FOTO_PASAPORTE', 'Fotos tamaño pasaporte', 20.00);

-- Una línea de venta es un producto o un servicio, nunca ambos.
ALTER TABLE sale_detail ALTER COLUMN product_id DROP NOT NULL;
ALTER TABLE sale_detail ADD COLUMN service_id BIGINT REFERENCES service_item (service_id);
ALTER TABLE sale_detail ADD CONSTRAINT sale_detail_product_xor_service CHECK ((product_id IS NULL) <>
(service_id IS NULL));

CREATE TABLE generated_document (
    document_id   BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    document_type VARCHAR(10) NOT NULL CONSTRAINT generated_document_type_check CHECK (document_type IN
('DPI','CEDULA','PASAPORTE')),
    output_format VARCHAR(4)  NOT NULL CONSTRAINT generated_document_format_check CHECK (output_format IN
('PDF','DOCX')),
    copies        INTEGER     NOT NULL DEFAULT 1 CHECK (copies >= 1),
    client_name   VARCHAR(100),
    generated_by  UUID        NOT NULL REFERENCES users (user_id),
    duration_ms   INTEGER CHECK (duration_ms >= 0),
    sale_id       BIGINT REFERENCES sales (sales_id),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_generated_document_created ON generated_document (created_at DESC);
```

- **Impacto en el código:** `Sale` y `SaleDetail` pasan a mapear `customer`, `cash_session`, `payment_method` y `service`; `sale_detail.product_id` deja de ser obligatorio (la restricción `sale_detail_product_xor_service` garantiza producto o servicio); `DpiDocumentServiceImpl` e `IdPhotoDocumentServiceImpl` registran un `generated_document` con la duración.
- **Advertencia:** el relleno de clientes une a las personas que comparten exactamente el mismo nombre; después se pueden separar a mano. El valor por defecto `EFECTIVO` se aplica a las ventas históricas.
- **Reversión:** requiere restaurar `product_id NOT NULL` solo si no existen ventas de servicios; por eso se despliega primero el código y recién después se activa el catálogo de servicios.

28.9. V16 · Notificaciones leídas por usuario

Propósito. Que cada usuario marque las notificaciones por separado (hallazgo H-11). El relleno traslada lo ya leído a todos los usuarios activos a quienes iba dirigida cada notificación.

```
-- La marca "leída" deja de ser global: cada usuario lee las notificaciones por separado.
CREATE TABLE notification_read (
    notification_id BIGINT      NOT NULL REFERENCES notification (notification_id) ON DELETE CASCADE,
    user_id         UUID        NOT NULL REFERENCES users (user_id) ON DELETE CASCADE,
    read_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (notification_id, user_id)
);
CREATE INDEX idx_notification_read_user ON notification_read (user_id);

-- Lo ya leído (flag global) se traslada a todos los usuarios activos.
INSERT INTO notification_read (notification_id, user_id)
SELECT n.notification_id, u.user_id
  FROM notification n CROSS JOIN users u
 WHERE n.read AND u.active
   AND (n.recipient_role IS NULL OR n.recipient_role = (SELECT r.name FROM role r WHERE r.id = u.role_id))
ON CONFLICT DO NOTHING;
```

- **Impacto en el código:** `NotificationServiceImpl` consulta `notification_read` en lugar de la columna `read` (que se conserva hasta V18).
- **Reversión:** la tabla puede eliminarse; el código anterior sigue usando `notification.read`.

28.10. V17 · Sucursales

Propósito. Preparar el negocio para más de una sucursal: el stock pasa a una tabla por sucursal y las ventas, cajas y movimientos quedan asociados a ella (hallazgo H-12).

```
-- Multi-sucursal: el stock pasa de la tabla product a inventory_stock (una fila por sucursal y producto).
CREATE TABLE branch (
    branch_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name      VARCHAR(100) NOT NULL UNIQUE,
    address   VARCHAR(200),
    active    BOOLEAN NOT NULL DEFAULT TRUE
);
INSERT INTO branch (name) VALUES ('Sede principal');

CREATE TABLE inventory_stock (
    branch_id  BIGINT  NOT NULL REFERENCES branch (branch_id),
    product_id BIGINT  NOT NULL REFERENCES product (product_id),
    stock      INTEGER NOT NULL CHECK (stock >= 0),
    min_stock  INTEGER NOT NULL DEFAULT 2 CHECK (min_stock >= 0),
    PRIMARY KEY (branch_id, product_id)
);
INSERT INTO inventory_stock (branch_id, product_id, stock, min_stock)
SELECT (SELECT branch_id FROM branch WHERE name = 'Sede principal'), p.product_id, p.stock, p.min_stock
FROM product p;

ALTER TABLE sales          ADD COLUMN branch_id BIGINT REFERENCES branch (branch_id);
ALTER TABLE cash_session   ADD COLUMN branch_id BIGINT REFERENCES branch (branch_id);
ALTER TABLE stock_movement ADD COLUMN branch_id BIGINT REFERENCES branch (branch_id);
ALTER TABLE users          ADD COLUMN branch_id BIGINT REFERENCES branch (branch_id);
UPDATE sales          SET branch_id = (SELECT branch_id FROM branch LIMIT 1);
UPDATE cash_session   SET branch_id = (SELECT branch_id FROM branch LIMIT 1);
UPDATE stock_movement SET branch_id = (SELECT branch_id FROM branch LIMIT 1);
ALTER TABLE sales ALTER COLUMN branch_id SET NOT NULL;
-- Una sola caja abierta por sucursal (sustituye al índice global de V15).
DROP INDEX uq_cash_session_one_open;
CREATE UNIQUE INDEX uq_cash_session_one_open ON cash_session (branch_id) WHERE status = 'ABIERTA';
```

- **Impacto en el código:** el stock se lee y se descuenta de `inventory_stock` (con el mismo bloqueo pesimista, ahora sobre `(branch_id, product_id)`); la interfaz permite elegir la sucursal; los reportes se filtran por sucursal. Es el cambio de código más grande del plan.
- **Reversión:** mientras `product.stock` exista (hasta V18) se puede volver al mapeo anterior; después solo por restauración.

28.11. V18 · Contraer las columnas heredadas

Propósito. Retirar lo que dejó de usarse (hallazgo H-13). Es destructiva y solo se aplica cuando V14 a V17 llevan un ciclo completo en producción sin referencias a esas columnas.

```
-- Fase "contraer" del patrón expandir/contraer: solo después de desplegar la aplicación que ya no los usa.
ALTER TABLE stock_alert DROP COLUMN ordered_quantity, DROP COLUMN ordered_at;
ALTER TABLE sales       DROP COLUMN customer_name;
ALTER TABLE notification DROP COLUMN read;
ALTER TABLE product     DROP COLUMN stock, DROP COLUMN min_stock;
```

Lista previa obligatoria: respaldo reciente con una restauración probada; búsqueda en el código de cualquier uso de `ordered_quantity`, `ordered_at`, `customer_name`, `notification.read`, `product.stock` y `product.min_stock`; despliegue fuera del horario de atención; plan de retorno por restauración (esta migración no tiene reversión directa).

28.12. Estrategia de ejecución y pruebas

Puerta	Qué se comprueba
Sintaxis y orden	El CI aplica todas las migraciones sobre PostgreSQL 17 con `ON_ERROR_STOP`; ya lo hace `InventoryBackApplicationTests` al arrancar el contexto
Datos reales	Un trabajo de CI restaura una copia anonimizada de producción, aplica las migraciones nuevas y ejecuta las consultas de comprobación de 25.8.4
Compatibilidad	Con el esquema nuevo se ejecutan las pruebas del código anterior (garantiza que se pueda hacer rollback de la aplicación)
Respaldo	Obligatorio antes de V14 en adelante; V18 exige además haber probado la restauración

Puerta	Qué se comprueba
Ventana	Fuera del horario de atención del negocio; las migraciones de este plan duran segundos con el volumen actual
Verificación	`flyway_schema_history` sin fallos, conteos de tablas, venta de prueba y prueba de humo (25.4.5)

28.13. Modelo entidad-relación objetivo y trazabilidad de las migraciones

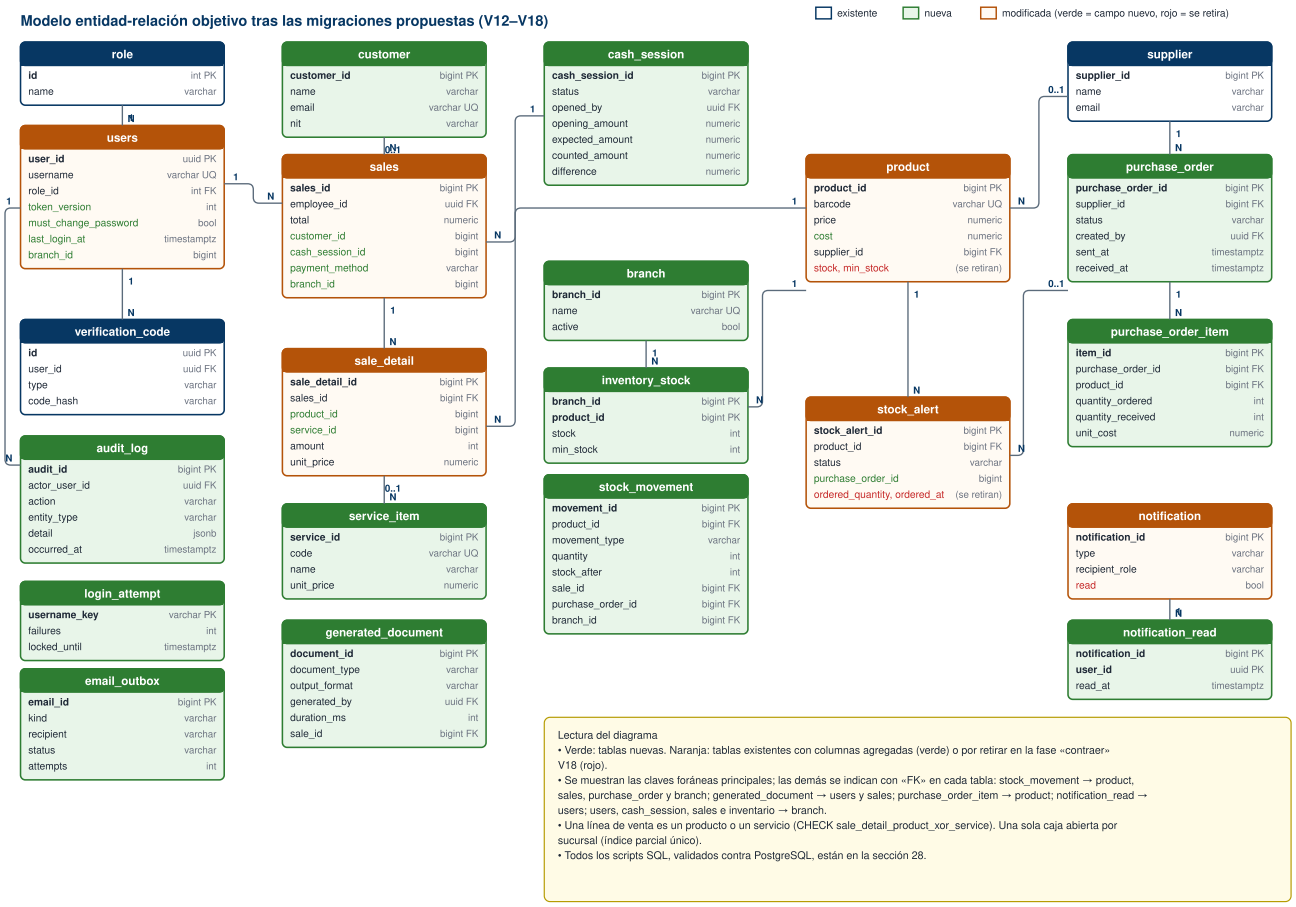


Figura 28.2 — Modelo entidad-relación objetivo tras V12 a V18 (verde: nuevo; naranja: modificado)

Migración	Proceso o historia que habilita	Requisito o hallazgo
V12	Ventas simultáneas seguras y reportes rápidos	RNF-09, H-01 a H-03
V13	Auditoría de acciones, cierre de sesión que revoca, cambio forzado de la contraseña sembrada, correo con reintentos (BPMN-05, BPMN-06)	RF-08, RNF-03, H-04 a H-07
V14	Compras con orden y recepción, kardex de inventario (BPMN-F1)	RF-04, RF-10, H-08 y H-09
V15	Apertura y cierre de caja (BPMN-F2), registro de servicios (BPMN-F3), clientes, medición de RNF-02	RF-07, H-10
V16	Campana de notificaciones por usuario (mejora de HU-54)	RF-06, H-11
V17	Operación multi-sucursal (sección 26.6)	H-12
V18	Limpieza del modelo tras la transición	H-13

29. Tecnologías Nuevas por Utilizar y su Justificación

29.1. Criterios de selección

Toda tecnología nueva debe justificarse con un problema medido o documentado, no con la moda. Se evaluó cada propuesta con los mismos criterios, pensando en un negocio pequeño operado por una sola persona técnica.

Criterio	Pregunta que se hace
Problema real	¿Resuelve un hallazgo concreto de las secciones 25 y 26 (por ejemplo DT-01 o DT-10)?
Costo	¿Es gratuita, de código abierto o con un plan pequeño? ¿El costo crece con el uso?
Operación	¿Puede operarla una sola persona sin convertirse en su trabajo principal? Se prefiere lo administrado
Madurez	¿Tiene comunidad, documentación y mantenimiento activo?
Salida	¿Cuánto cuesta dejarla si deja de convenir (dependencia del proveedor)?
Integración	¿Encaja con Spring Boot, React, PostgreSQL y GitHub Actions sin reescribir?
Seguridad	¿Reduce riesgos o agrega superficie de ataque?

29.2. Línea base: lo que ya se utiliza y se mantiene

Estas tecnologías ya están en producción y no se reemplazan: Spring Boot 3.5 con Java 25, PostgreSQL 17 con Flyway, React 18 con Vite y Tailwind, JWT en cookie HttpOnly con BCrypt y HMAC, BoofCV, PDFBox y Apache POI, Brevo para correo, Docker, GitHub Actions, Railway y Netlify, y las pruebas con JUnit y Vitest. La lista completa con versiones está en la sección 24.2.

29.3. Tecnologías nuevas propuestas

Se agrupan por el horizonte de la hoja de ruta (sección 26.5). Los costos son cualitativos: «gratuito» significa que el plan básico o la edición de código abierto cubre el uso esperado del negocio; confirme siempre los precios vigentes antes de contratar.

Horizonte 1 · 0 a 3 meses: endurecer lo existente

Tecnología	Categoría	Problema que resuelve	Justificación y alternativas	Costo y riesgo
Dependabot, CodeQL y Trivy	DevSecOps	DT-15: dependencias fijas sin vigilancia; no hay análisis de código ni de la imagen Docker	Nativos de GitHub y sin servidor propio. Alternativas: Snyk y SonarCloud (más completos pero con límites de plan)	Gratuito en repositorios públicos; riesgo bajo; hay que atender las alertas
springdoc-openapi	Documentación de API	El catálogo de 24.4 se mantiene a mano y se desactualiza	Genera OpenAPI 3 y Swagger UI desde los controladores; habilita pruebas de contrato. Alternativa: Postman manual	Gratuito; exponer solo en entornos no públicos
Monitor externo (UptimeRobot o Better Stack)	Observabilidad	Solo se mira el health durante el despliegue; una caída nocturna pasa inadvertida	Consulta `/actuator/health` cada minuto y avisa por correo o chat. Alternativa: Prometheus Blackbox (más complejo)	Gratuito en el plan básico; sin riesgo técnico
Respaldo `pg_dump` + GPG + bucket S3 compatible	Continuidad	No hay respaldo propio verificado (25.7)	Estándar, sin agentes; Cloudflare R2 o AWS S3 como destino. Alternativa: solo los respaldos del proveedor (no probados)	Costo de almacenamiento mínimo; riesgo: gestionar la llave de cifrado
JaCoCo	Calidad	No existe medición de cobertura	Plugin de Maven integrado al `verify`; umbral mínimo en el CI. Alternativa: Codecov	Gratuito; sin riesgo
Playwright (E2E)	Pruebas	Los 30 flujos de interfaz probados el 24/10 corrieron a mano, fuera del CI	Pruebas de navegador reales ejecutadas en GitHub Actions. Alternativa: Cypress	Gratuito; las pruebas E2E requieren mantenimiento
Testcontainers	Pruebas	Las pruebas de contexto dependen de un PostgreSQL local o de un servicio del CI	Levanta PostgreSQL en un contenedor desde JUnit, igual en el CI y en la máquina del equipo. Alternativa: servicio de Actions (actual)	Gratuito; necesita Docker local (en algunos equipos no funciona)

Horizonte 2 · 3 a 9 meses: operar a escala

Tecnología	Categoría	Problema que resuelve	Justificación y alternativas	Costo y riesgo
Redis con Spring Data Redis y Bucket4j	Estado compartido	DT-01: contadores en memoria impiden tener varias instancias	Límites de tasa y bloqueos compartidos con vencimiento nativo; caché de reportes. Alternativa: tablas de PostgreSQL (V13), más simple pero más lenta	Plan gratuito o pequeño (Upstash, Railway); riesgo: una dependencia más que operar; degradar a memoria si cae
Micrometer, Prometheus y Grafana	Métricas	DT-10: no se puede medir la latencia, la memoria ni el tiempo de los documentos	Actuator ya está en el proyecto; Micrometer agrega el registro Prometheus; Grafana Cloud tiene plan gratuito. Alternativa: Datadog (costoso)	Gratuito en el plan básico; proteger `/actuator/prometheus`
Sentry	Errores	Los 500 solo se ven al revisar los registros	SDK para Spring Boot y React: agrupa errores, avisa y muestra el contexto. Alternativa: GlitchTip autoalojado	Plan gratuito limitado; filtrar datos personales
Registros JSON con correlación (Logback)	Observabilidad	Los registros de texto libre dificultan buscar un incidente	Un identificador por petición permite seguir una venta de punta a punta. Alternativa: dejar el texto actual	Sin costo; cuidar de no registrar datos personales
Cola de trabajos sobre PostgreSQL (`SKIP LOCKED`) y, después, SQS o RabbitMQ	Asincronía	DT-02 y DT-08: imágenes y correo dentro de la petición	Una tabla de trabajos y un trabajador con `FOR UPDATE SKIP LOCKED` no agrega infraestructura; un broker se justifica con varios nodos. Alternativas: Kafka (excesivo), Spring `@Async` solo (se pierde al reiniciar)	Sin costo adicional al inicio; hay que manejar reintentos y trabajos atascados
Terraform	Infraestructura como código	El enunciado exige IaC; hoy los entornos se arman a mano (DT-11)	Declara Netlify, GitHub (ramas protegidas y environments) y AWS; Railway mediante un proveedor de la comunidad o CLI. Alternativas: Pulumi y OpenTofu	Gratuito; riesgo: estado compartido y proveedor comunitario de Railway
Entorno `staging`	Entrega	DT-11: los cambios se prueban directamente en producción	Un segundo entorno de Railway y despliegue de vista previa en Netlify; el pipeline despliega primero allí	Costo de otro servicio pequeño
k6	Pruebas de carga	Las medidas de capacidad son puntuales	Scripts en JavaScript que reproducen venta, documentos y reportes; se integran al CI. Alternativas: JMeter y Gatling	Gratuito; hay que usar datos de prueba
Gestor de secretos (Doppler o AWS Secrets Manager)	Seguridad	Los secretos viven en variables de tres plataformas	Un único lugar con auditoría y rotación. Alternativa: seguir con las variables de cada plataforma (aceptable hoy)	Plan gratuito o pequeño; dependencia adicional

Horizonte 3 · 9 a 18 meses: crecer

Tecnología	Categoría	Problema que resuelve	Justificación y alternativas	Costo y riesgo
TOTP (RFC 6238) y WebAuthn	Autenticación	El 2FA por correo depende de que el correo funcione y sea rápido	Aplicaciones autenticadoras y llaves de acceso no dependen del correo. Alternativa: SMS (más costoso e inseguro)	Biblioteca de código abierto; hay que prever la recuperación de la cuenta
Réplica de lectura y PgBouncer	Base de datos	Reportes pesados compiten con las ventas; muchas conexiones	Mueve las consultas analíticas a otra instancia. Alternativa: vistas materializadas	Costo de una segunda instancia
Particionado nativo y `pg_cron`	Base de datos	`audit_log` y `sales` crecen sin límite	Particiones mensuales con archivado automático. Alternativa: borrar o exportar a mano	Sin costo; requiere planificar
PWA con Workbox	Frontend	Conexión intermitente en el mostrador o en el móvil	Aplicación instalable con caché del catálogo. Alternativa: aplicación nativa (mucho más costosa)	Sin costo; sincronización de ventas fuera de línea es compleja
AWS (S3, CloudFront, ECS Fargate, RDS)	Nube de propósito general	Exigencia institucional, más control o crecimiento	Mapa de equivalencias en 26.4. Alternativas: Azure y GCP	Costo mayor que las plataformas como servicio; complejidad de operación

29.4. Detalle de las decisiones principales

29.4.1. Estado compartido con Redis

El sistema es sin estado salvo dos contadores (bloqueo de inicio de sesión y recibos por hora). Mientras haya una sola instancia eso es correcto; con dos, cada una contaría por su lado. La solución propuesta es Redis con `spring-`

`boot-starter-data-redis` y Bucket4j: las claves llevan vencimiento (`login:fail:<usuario>`, `receipt:seller:<usuario>`), el bloqueo se comparte entre instancias y la caché de reportes usa el mismo servidor. Si Redis no responde, el servicio degrada a los contadores en memoria actuales y registra una advertencia, de modo que una caída del caché nunca impide iniciar sesión.

29.4.2. Observabilidad con Micrometer

Actuator ya está en el proyecto. Agregar `micrometer-registry-prometheus` y exponer `/actuator/prometheus` protegido (por token o en un puerto interno) entrega métricas de JVM, HTTP y conexiones sin escribir código. Se añaden tres métricas propias: un temporizador por tipo de documento (`documents.generate.duration`, para vigilar RNF-02), un contador de ventas registradas y un contador de correos fallidos. Con ellas se pueden armar los tableros y las alertas de 25.6.

29.4.3. Trabajos asíncronos sin broker

Antes de incorporar RabbitMQ o SQS, una tabla de trabajos resuelve el problema con la infraestructura actual. El API inserta el trabajo y responde 202; un trabajador lo toma con una consulta que evita colisiones entre varios consumidores:

```
SELECT job_id, payload
  FROM job_queue
 WHERE status = 'PENDIENTE' AND run_at <= now()
 ORDER BY run_at
 LIMIT 1
 FOR UPDATE SKIP LOCKED;
```

La misma tabla alimenta la bandeja de correo (`email_outbox`, V13) y la generación pesada de documentos. Cuando el volumen o el número de nodos lo justifiquen, se cambia el transporte por SQS sin tocar la lógica de negocio.

29.4.4. Infraestructura como código y reglas del repositorio

El enunciado pide infraestructura como código. Se propone Terraform para tres frentes: el sitio de Netlify, la configuración de los repositorios de GitHub (ramas protegidas, environments y variables) y, si se migra, la infraestructura de AWS. La protección de ramas es el primer candidato, porque cierra la observación de proceso de 24.6.2:

```
resource "github_branch_protection" "main" {
  repository_id  = github_repository.backend.node_id
  pattern        = "main"
  enforce_admins = true

  required_status_checks {
    strict   = true
    contexts = ["CI"]
  }
  required_pull_request_reviews {
    required_approving_review_count = 1
  }
}
```

29.4.5. Vigilancia de dependencias y de la cadena de suministro

Un archivo `.github/dependabot.yml` abre Pull Requests semanales para las dependencias de Maven, npm, Docker y GitHub Actions; CodeQL analiza el código en cada PR y Trivy escanea la imagen construida antes de desplegarla. Así se atienden a tiempo las versiones fijadas de JJWT, POI y PDFBox, y se mantienen los workflows con acciones actualizadas.

```
version: 2
updates:
  - package-ecosystem: maven
    directory: "/"
    schedule: { interval: weekly }
  - package-ecosystem: docker
    directory: "/"
    schedule: { interval: weekly }
  - package-ecosystem: github-actions
    directory: "/"
    schedule: { interval: weekly }
```

29.5. Tecnologías evaluadas y descartadas

Tecnología	Por qué se evaluó	Por qué se descarta (por ahora)
Microservicios	Escalar componentes por separado	Una sola sucursal y una persona técnica; las transacciones de venta, alerta e inventario comparten datos. Se revisa si un componente (por ejemplo, las imágenes) exige otro ciclo de vida
Kubernetes	Orquestación y autoescalado	La complejidad operativa supera con creces el beneficio; Railway o Fargate cubren el autoescalado necesario
BPMS comercial (Camunda, Bizagi, Bonita)	Ejecutar los procesos modelados	Ya se decidió en la Fase 1: el propio sistema es el BPMS a medida; un motor externo duplicaría reglas que viven en el código
Apache Kafka	Eventos y mensajería	Volumen de decenas de eventos por hora; una tabla de trabajos o SQS bastan
MongoDB u otra base documental	Flexibilidad de esquema	El dominio es relacional y transaccional (ventas, stock); PostgreSQL ya ofrece `jsonb` donde hace falta
Liquibase	Alternativa a Flyway	Flyway ya está integrado y es suficiente; cambiar no aporta valor
Elasticsearch	Búsqueda de productos	La búsqueda por nombre o código de barras sobre miles de filas no lo requiere; un índice de PostgreSQL basta
Reescribir el frontend (Next.js u otro)	Renderizado en servidor	Es un sistema interno detrás de un inicio de sesión; la SPA actual cumple y no requiere posicionamiento web

29.6. Hoja de ruta de adopción

Horizonte	Tecnologías	Esfuerzo estimado	Dependencias
H1	Dependabot, CodeQL, Trivy, springdoc, monitor externo, respaldo con `pg_dump`, JaCoCo, Playwright, Testcontainers	8 a 12 días de trabajo	Ninguna
H2	Redis, Micrometer con Prometheus y Grafana, Sentry, registros JSON, cola de trabajos, Terraform, `staging`, k6, gestor de secretos	25 a 35 días de trabajo	V13; H1 terminado
H3	TOTP y WebAuthn, réplica de lectura, particionado, PWA y, opcionalmente, AWS	30 a 45 días de trabajo	V14 a V17; criterios de activación de 26.8

> **Nota:** las estimaciones de esfuerzo son orientativas para una persona con el contexto del proyecto; deben refinarse al planificar cada sprint de tres semanas.

30. Conclusiones

- El análisis BPM realizado permitió identificar, dentro de la operación aparentemente simple de Multiservicios Wilson, dos procesos con alto potencial de automatización y beneficio inmediato: la gestión de inventario y la generación de fotocopias de DPI.
- El modelado BPMN de ambos procesos en sus versiones AS-IS y TO-BE evidencia con claridad el problema actual, dependencia de la memoria humana y del tiempo manual, la solución propuesta reglas de negocio ejecutadas automáticamente por el sistema.
- El sistema a desarrollar cumplirá a escala de esta miniempresa, el rol de un BPMS: la plataforma que ejecuta y automatiza los procesos modelados, sin necesidad de adoptar una herramienta comercial de propósito general.
- El alcance definido en esta fase es deliberadamente acotado a los procesos de mayor impacto, dejando documentadas las extensiones futuras identificadas durante el análisis de negocio.

- **Fase 3 · puesta en producción.** El sistema opera en la nube (Netlify, Railway y PostgreSQL) con entrega continua desde GitHub Actions, verificación automática posterior al despliegue y un Runbook que explica cómo desplegar, revertir, vigilar y recuperar el servicio, de modo que su operación ya no depende de quien lo desarrolló.
- **Fase 3 · seguridad en capas.** La autenticación se reforzó con verificación en dos pasos por correo, recuperación de contraseña, bloqueo por fuerza bruta, cookies HttpOnly y códigos guardados solo como HMAC. Los riesgos que subsisten (el cierre de sesión no revoca el token y la contraseña sembrada del administrador) están identificados y tienen su migración y su lista de verificación.
- **Fase 3 · datos que evolucionan con control.** El esquema creció de seis a nueve tablas en once migraciones versionadas con Flyway, sin editar jamás una ya aplicada. Se deja un plan de siete migraciones más (V12 a V18), validado sobre PostgreSQL, que sostiene los procesos de compras, caja, servicios y varias sucursales.
- **Fase 3 · trazabilidad.** Cada requisito se puede seguir hasta sus historias de usuario, sus componentes, sus pruebas (251 automatizadas) y sus Pull Requests; las brechas reconocidas (protección de ramas, cobertura, respaldo verificado y entorno de pruebas) figuran en un plan con horizonte y criterio de activación.
- **Fase 3 · crecimiento sin improvisar.** La escalabilidad se plantea por métricas y no por fechas: primero estado compartido, cola de trabajos y observabilidad; después réplicas y sucursales; microservicios y orquestadores quedaron descartados por ahora con su justificación.

31. Enlaces de interés

DiagramasBMPN
Diagramas de Procesos
Diagramas de Casos de Uso
Diagramas UML
Repositorio Github
Manual de Usuario
Repositorio Backend (GitHub)
Repositorio Frontend (GitHub)
Estado del API en producción (health)